
THIẾT KẾ HỆ THỐNG NHÚNG

(EMBEDDED SYSTEMS DESIGN)

cuu duong than cong. com

Tài Liệu Tham Khảo

- Embedded Systems Design: A unified hardware/software introduction – Vahid/Givargis, 1999.
- Designing Embedded Hardware – Jonh Catsoulis, 2005.
- Programming Embedded Systems in C and C++ - Michael Barr, 1999.
- Verilog HDL: A guide to digital design and synthesis – Sarmir Palnitkar, 2003.

CHƯƠNG 1 – BÀI 1

GIỚI THIỆU CHUNG

cuuduongthancong.com

Tổng Quan

- Tổng quan hệ thống nhúng
 - Hệ thống nhúng là gì?
- Yêu cầu về thiết kế – tối ưu các thông số thiết kế
- Các công nghệ cuuduongthancong.com
 - Công nghệ xử lý
 - Công nghệ IC
 - Công nghệ thiết kế

cuuduongthancong.com

Tổng quan hệ thống nhúng

- Các hệ thống tính toán “computing” có mặt ở mọi nơi
- Đa số chúng ta nghĩ đến hệ thống tính toán như là một máy tính
 - PC’s 
 - Laptops 
 - Mainframes
 - Servers
- Nhưng có rất nhiều các hệ thống tính toán khác

Tổng quan hệ thống nhúng

- Hệ thống tính toán nhúng
 - Hệ thống tính toán nhúng trong các thiết bị điện tử
 - Rất khó để định nghĩa. Có thể coi chúng là các hệ thống tính toán ngoài PC.
 - Có hàng tỷ thiết bị được sản xuất mỗi năm, so với số lượng hàng triệu của PC.
 - Có lẽ chiếm đến 50% các thiết bị gia dụng và ô-tô

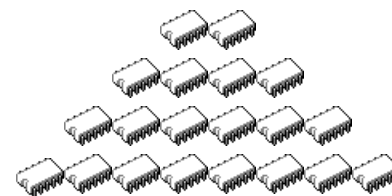
Computers are in here...



and here...



and even here...

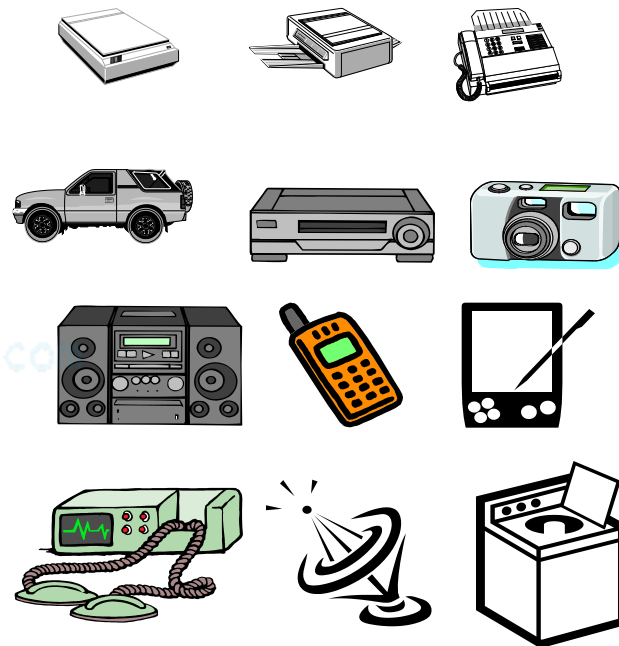


Lots more of these,
though they cost a lot
less each.

Một “danh sách” các hệ thống nhúng

Hệ thống chống bó (Anti-lock brakes)
Tự động điều chỉnh tiêu cự (Auto-focus cameras)
Tự động trả lời (Automatic teller machines)
Thanh toán tự động (Automatic toll systems)
Truyền dẫn tự động (Automatic transmission)
Avionic systems
Xạc bin (Battery chargers)
Máy quay KTS (Camcorders)
Điện thoại di động (Cell phones)
Trạm di động (Cell-phone base stations)
Điện thoại không dây (Cordless phones)
Điều khiển lái (Cruise control)
Camera số (Digital cameras)
Ổ đĩa cứng
Thiết bị đọc thẻ điện tử
Dụng cụ điện tử
Đồ chơi điện tử
Điều khiển nhà máy
Máy Fax
Thiết bị nhận dạng vân tay
Hệ thống an ninh tòa nhà
Hệ thống kiểm tra y tế

Modems
Bộ giải mã MPEG
Card mạng
Định tuyến/chuyển mạch
Thiết bị định vị
Máy photocopy
Máy in
Điện thoại vệ tinh
Máy quét
Máy giặt
Thiết bị nhận dạng giọng nói
Hệ thống thị giác
Hội nghị từ xa
Truyền hình
Bộ điều khiển nhiệt độ
Hệ thống chống trộm
Đầu VCR's, DVDs
Điện thoại có hình
Máy rửa bát
vv....

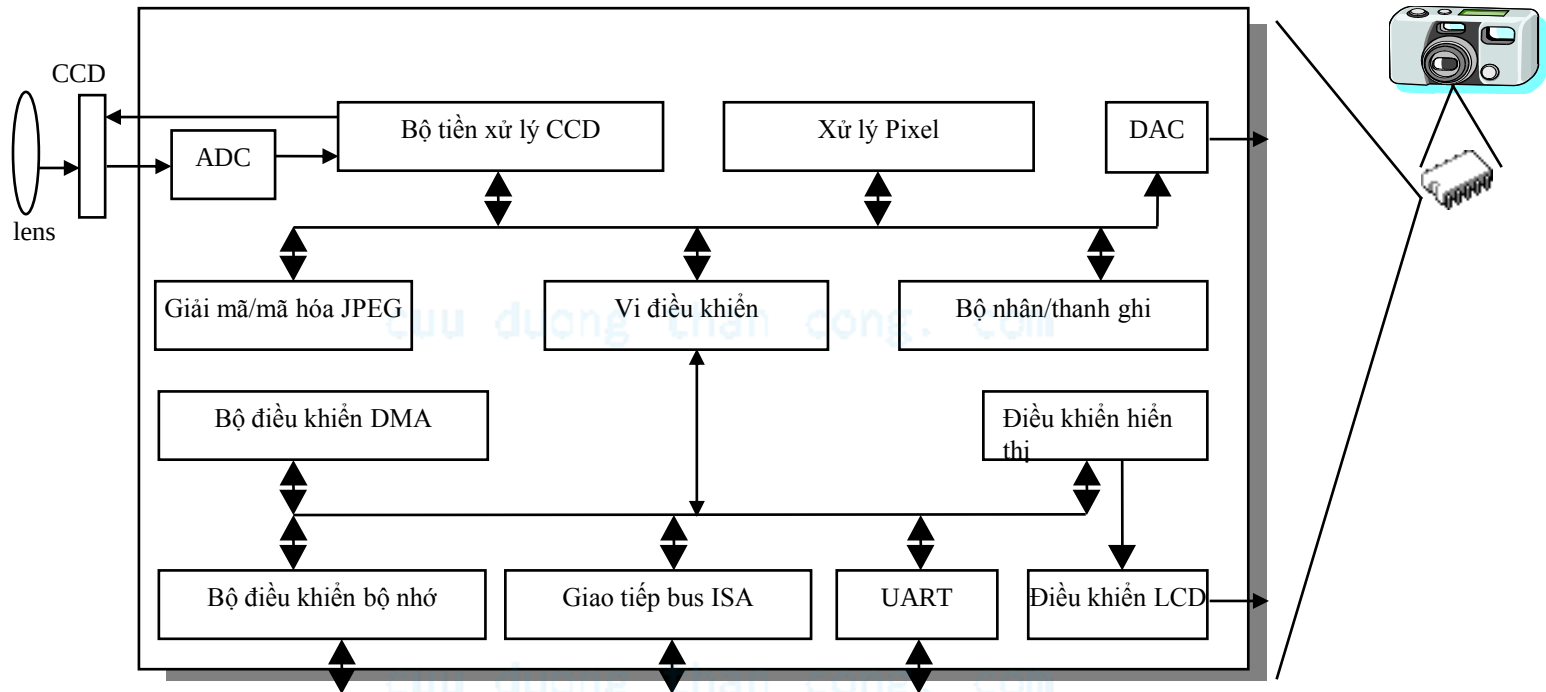


Và rất nhiều thiết bị khác

Một số đặc tính chung của hệ thống nhúng

- Có chức năng đơn lẻ
 - Thực hiện một chương trình đơn lẻ, lặp lại
 - Có nhiều ràng buộc
 - Giá thành thấp, công suất tiêu thụ thấp, nhỏ, nhanh, vv.
 - Tương tác và thời gian thực
 - Tương tác liên tục với những thay đổi trong môi trường xung quanh
 - Phải tính toán kết quả trong một khoảng thời gian thực (real-time) không có hoặc ít trễ
-

Một ví dụ về hệ thống nhúng – Camera số



- Chức năng đơn lẻ -- luôn là một camera số
- Các ràng buộc – giá thấp, công suất thấp, nhỏ và nhanh
- Tương tác và thời gian thực – thời gian thực hiện ngắn

Yêu cầu thiết kế hệ nhúng – tối ưu các thông số thiết kế

- Mục tiêu thiết kế tổng quát:
 - Xây dựng một hệ thống thực hiện các chức năng yêu cầu.
- Các yêu cầu về thiết kế:
 - Tối ưu các thông số thiết kế đồng thời
- Các thông số thiết kế
 - Đặc tính xác định việc thực hiện hệ thống
 - Tối ưu các thông số thiết kế là thách thức chủ yếu trong thiết kế hệ thống nhúng

Yêu cầu thiết kế hệ nhúng – tối ưu các thông số thiết kế

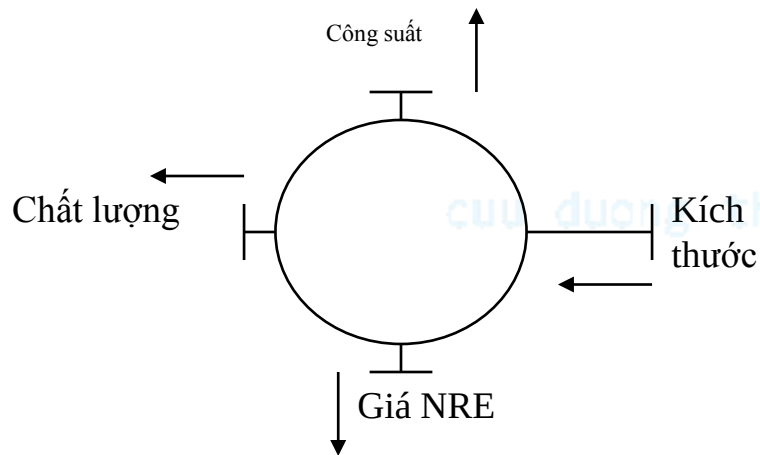
- Các thông số chung
 - Giá của thiết bị: là giá thành sản xuất mỗi sản phẩm, bao gồm giá kỹ thuật
 - Giá (Giá kỹ thuật không được sử dụng lại): Giá thiết kế hệ thống một lần
 - Kích thước: không gian vật lý yêu cầu của hệ thống
 - Chất lượng: thời gian làm việc hoặc tuổi thọ của hệ thống, vv.
 - Công suất: lượng công suất tiêu thụ của hệ thống
 - Độ linh hoạt: khả năng thay đổi các chức năng của hệ thống không làm thay đổi giá kỹ thuật

Yêu cầu thiết kế hệ nhúng – tối ưu các thông số thiết kế

- Các thông số chung (tiếp)
 - Thời gian thử nghiệm: thời gian cần thiết để chế tạo một phiên bản làm việc được
 - Thời gian đưa ra thị trường: thời gian cần thiết để phát triển một hệ thống có thể bán tới khách hàng
 - Khả năng bảo trì: khả năng thay thế và sửa chữa khi có sự cố
 - Độ tin cậy, độ an toàn, vv.

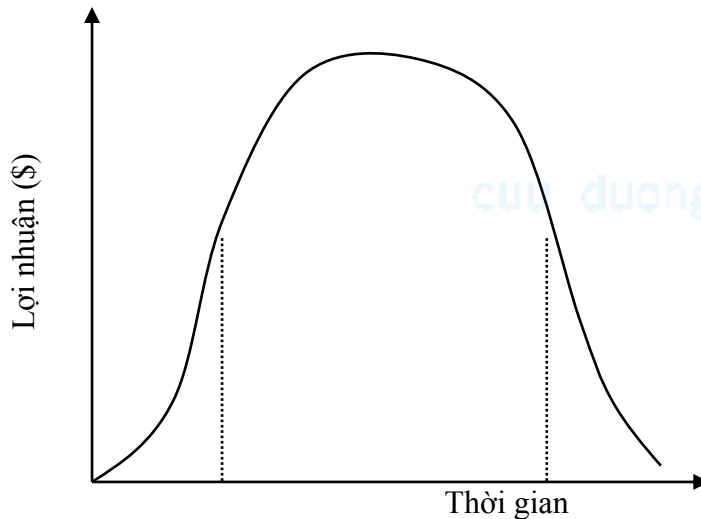
cuu duong than cong. com

Xem xét các thông số thiết kế - cải thiện một thông số có thể làm ảnh hưởng các thông số khác



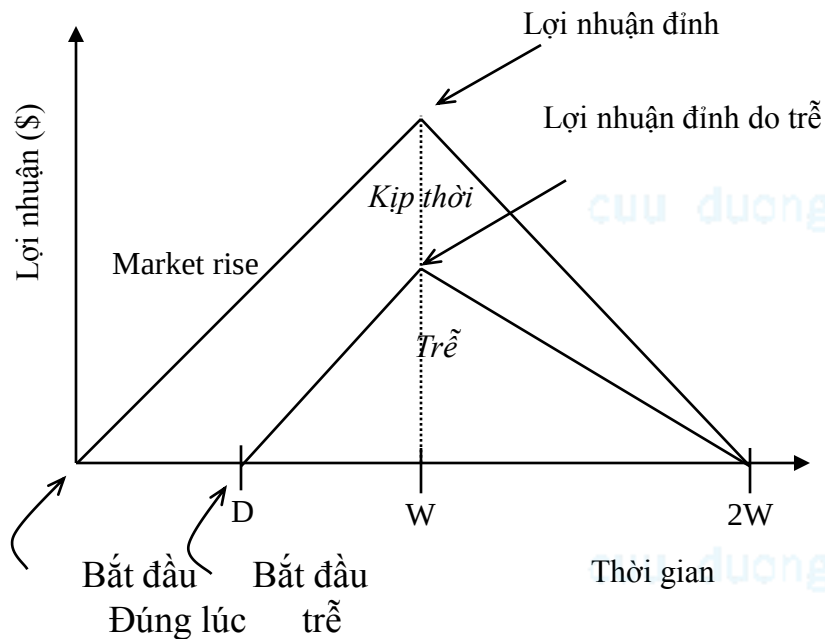
- Yêu cầu kinh nghiệm cả về **phần cứng và phần mềm** để tối ưu quá trình thiết kế
 - Không chỉ đơn thuần là một chuyên gia phần cứng, hoặc phần mềm.
 - Một người thiết kế phải hiểu nhiều công nghệ khác nhau để lựa chọn công nghệ tốt nhất cho một ứng dụng cụ thể.

Thời gian đưa ra thị trường: một thông số thiết kế quan trọng



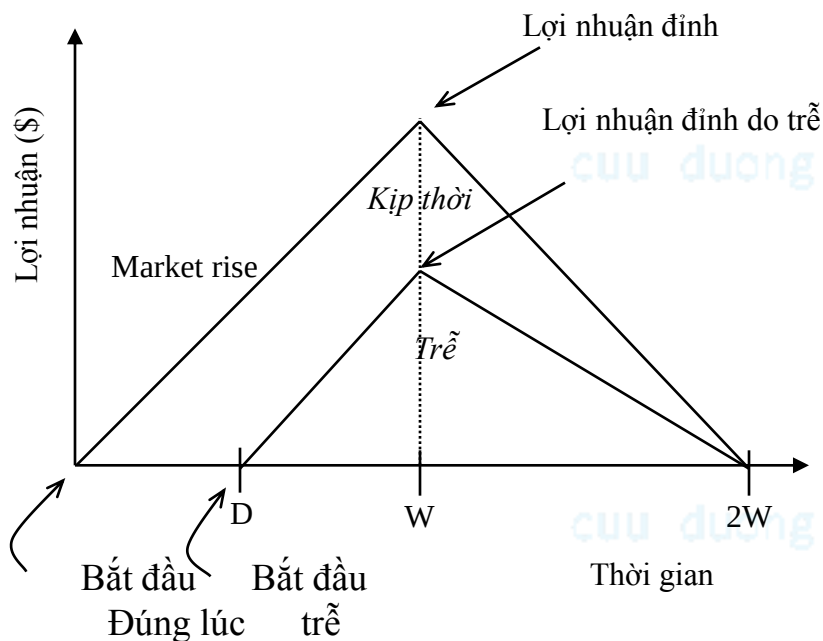
- Thời gian cần thiết để cung cấp sản phẩm tới khách hàng
- “Cửa sổ” thị trường
 - Giai đoạn mà sản phẩm có khả năng bán được số lượng lớn nhất.
- Trung bình thời gian đưa ra thị trường cho một sản phẩm nhúng là 8 tháng
- Kéo dài hơn sẽ tăng giá sản phẩm và làm mất cơ hội cạnh tranh

Các mất mát do đưa ra thị trường trễ



- Mô hình lợi nhuận đơn giản
 - Chu kỳ sống = $2W$, đỉnh tại W
 - Thời gian đưa ra thị trường định nghĩa như 1 hình tam giác, diễn tả sự thâm nhập thị trường
 - Vùng diện tích tam giác biểu thị lợi nhuận
- Chi phí cơ hội
 - Là sự khác nhau giữa vùng đưa ra kịp thời và đưa ra trễ

Các mất mát do đưa ra thị trường trễ (tiếp)



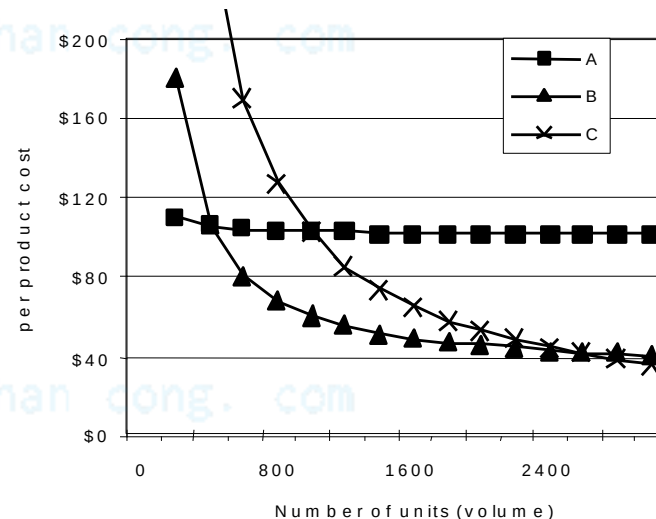
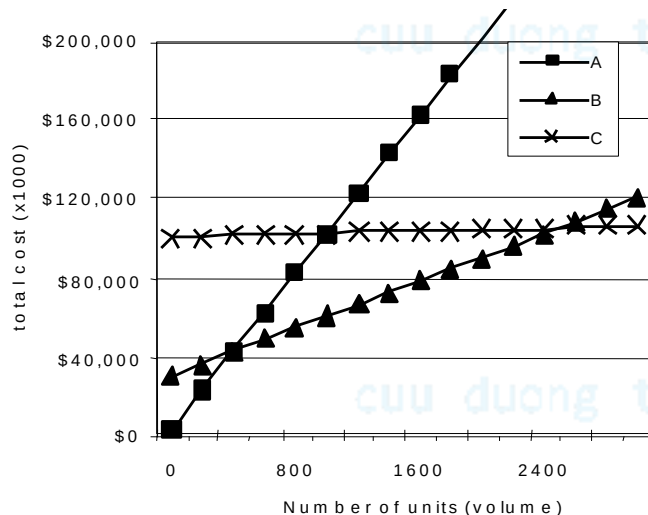
- Diện tích = $1/2 * \text{đáy} * \text{chiều cao}$
 - Đúng hạn = $1/2 * 2W * W$
 - Trễ = $1/2 * (W-D+W)*(W-D)$
- Phần trăm lợi nhuận bị mất = $(D(3W-D)/2W^2)*100\%$
- Ví dụ
 - Chu kỳ sống $2W=52$ tuần, trễ $D=4$ tuần
 - $(4*(3*26-4)/2*26^2) = 22\%$
 - Chu kỳ sống $2W=52$ tuần, trễ $D=10$ tuần
 - $(10*(3*26-10)/2*26^2) = 50\%$
 - Trễ làm giảm lợi nhuận!

Các thông số về giá NRE và giá đơn chiếc

- Giá thành:
 - Giá thành đơn chiếc: lượng chi phí để sản xuất một thiết bị, bao gồm giá NRE
 - Giá NRE (Non-Recurring Engineering cost): Giá thiết kế hệ thống một lần
 - $\text{Giá trị tổng} = \text{giá NRE} + \text{giá đơn chiếc} * \text{số lượng}$
 - $\text{Giá thành sx đơn chiếc} = \text{Giá trị tổng} / \text{số lượng}$
- Ví dụ
 - Giá NRE=\$2000, giá đơn vị=\$100
 - Đối với 10 sản phẩm
 - Tổng giá trị = \$2000 + 10*\$100 = \$3000
 - Giá thành từng sản phẩm = \$2000/10 + \$100 = \$300

Các thông số về giá NRE và giá đơn chiếc

- So sánh các công nghệ về giá
 - Công nghệ A: NRE=\$2,000, đơn giá =\$100
 - Công nghệ B: NRE=\$30,000, đơn giá =\$30
 - Công nghệ C: NRE=\$100,000, đơn giá =\$2



- Ngoài ra, còn phải quan tâm tới thời gian đưa ra thị trường

Thông số chất lượng

- Tránh lạm dụng các thông số
 - Tần số xung nhịp, số lệnh trên giây – không đánh giá tốt chất lượng
 - Ví dụ camera số – một người sử dụng quan tâm tốc độ xử lý ảnh, không phải tốc độ xung nhịp hoặc số lệnh trên giây
- Trễ (thời gian đáp ứng)
 - Thời gian giữa khởi đầu và kết thúc một tác vụ
 - VD, Camera's A và B xử lý hình ảnh trong 0.25 giây
- Dung lượng, lưu lượng
 - Số tác vụ trên giây, VD Camera A xử lý 4 ảnh trên giây

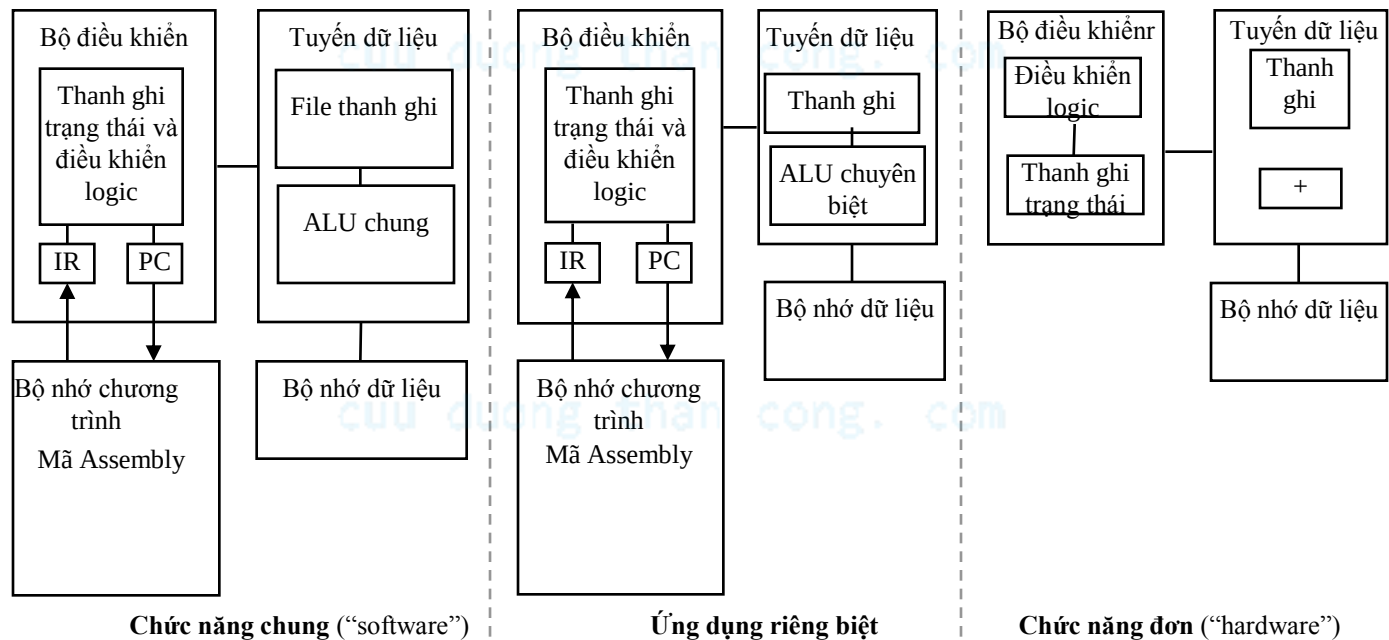
Ba công nghệ chìa khóa của hệ thống nhúng

- Công nghệ
 - Công nghệ ám chỉ việc thực hiện một tác vụ, sử dụng quá trình kỹ thuật, phương pháp và hiểu biết.
- Ba công nghệ chìa khóa đối với hệ thống nhúng
 - Công nghệ xử lý (processor technology)
 - Công nghệ IC (IC technology)
 - Công nghệ thiết kế (design technology)

cuu duong than cong. com

Công nghệ xử lý

- Kiến trúc của các thiết bị tính toán sử dụng để thực hiện một chức năng yêu cầu
- Bộ xử lý không yêu cầu phải lập trình lại
 - “Bộ xử lý” không giống với GPP



Công nghệ xử lý

- Bộ xử lý thay đổi tùy thuộc vào vấn đề mà nó xử lý



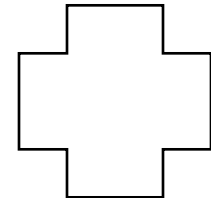
Chức năng mong
muốn



Bộ xử lý chức
năng chung



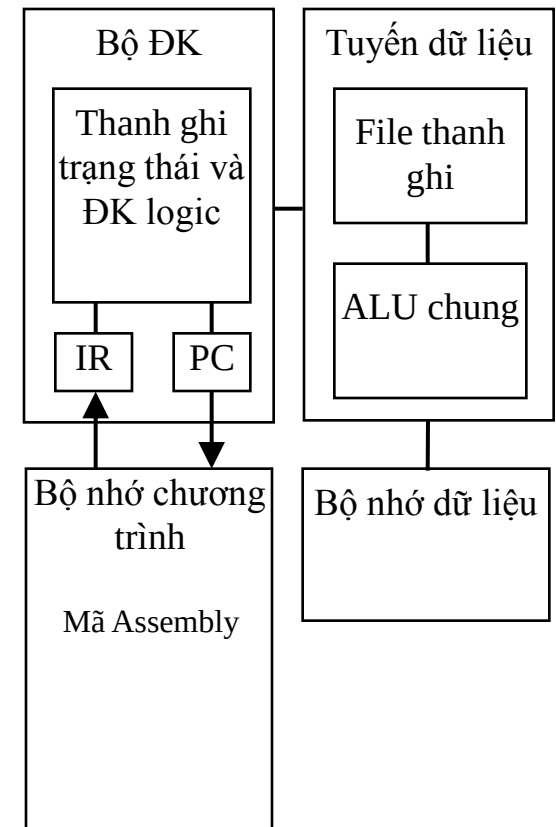
Bộ xử lý chuyên biệt



Bộ xử lý chức
năng đơn

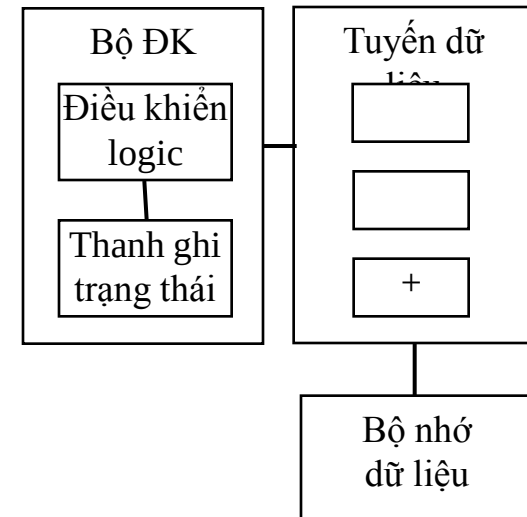
Bộ xử lý chức năng chung (General purpose processors)

- Là các thiết bị có thể lập trình sử dụng cho nhiều ứng dụng khác nhau
 - Đôi khi còn được gọi là “microprocessor”
- Đặc điểm
 - Có bộ nhớ chương trình
 - Có nhiều thanh ghi và đơn vị tính toán ALU
- Lợi ích
 - Giá thành đưa sản phẩm ra thị trường và giá thành NRE thấp
 - Độ linh hoạt cao
- Nổi tiếng nhất là bộ xử lý “Pentium”, tuy nhiên cũng có rất nhiều loại khác



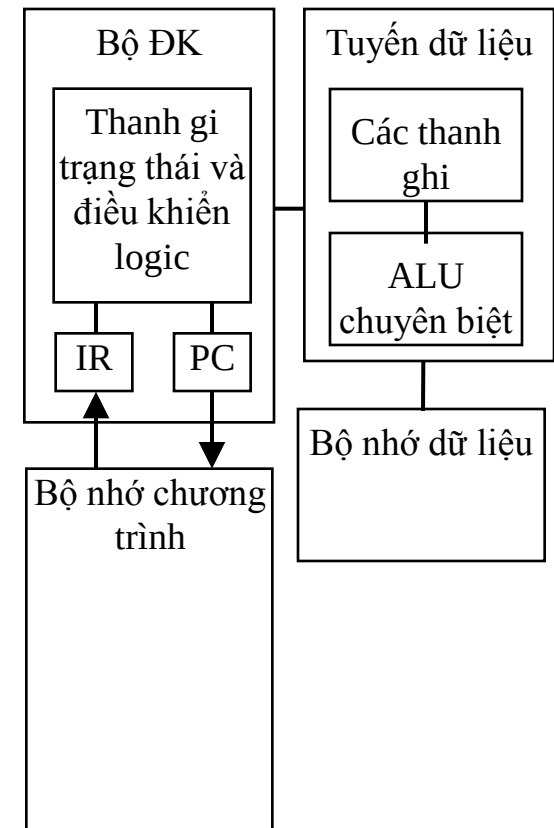
Bộ xử lý chức năng đơn (Single purpose processors)

- Là mạch số được thiết kế để thực hiện chính xác một chương trình
 - VD các thiết bị điều khiển ngoại vi
- Đặc điểm
 - Chỉ chứa các phần tử cần thiết cho một chương trình duy nhất
 - Không có bộ nhớ chương trình
- Lợi ích
 - Nhanh
 - Công suất thấp
 - Kích thước nhỏ



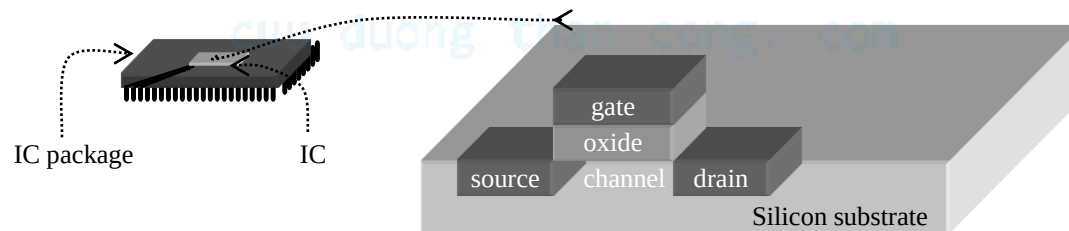
Bộ xử lý ứng dụng chuyên biệt (Application-specific processors)

- Bộ xử lý có thể lập trình được và tối ưu cho một loại ứng dụng cụ thể có nhiều đặc tính chung
 - Nó là sự cân bằng (dung hòa giữa GP processors và SP processors)
- Đặc điểm
 - Có bộ nhớ chương trình
 - Tuyến dữ liệu được thiết kế tối ưu
 - Có các đơn vị chức năng đặc biệt
- Lợi ích
 - Có độ linh hoạt nhất định, chất lượng, kích thước và công suất tốt



Công nghệ IC

- Là công nghệ thực hiện các cổng logic số và được tích hợp trên một thiết bị (IC)
 - IC: mạch tích hợp, hoặc “chip”
 - Công nghệ IC biến đổi tùy thuộc vào thiết kế cho một ứng dụng cụ thể
 - IC thường bao gồm một số lớp (10 lớp hoặc lớn hơn)
 - Công nghệ IC khác nhau ở khía cạnh ai xây dựng các lớp và khi nào chúng được xây dựng (thứ tự)



Công nghệ IC

- Có 3 công nghệ IC điển hình
 - VLSI (very large scale integrated)
 - ASIC (application specific IC - IC chức năng chuyên biệt)
 - PLD (programmable logic devices - thiết bị logic khả lập trình)

cuu duong than cong. com

Chuyên dụng – đầy đủ/Full-custom VLSI

- Tất cả các lớp được tối ưu cho việc thực hiện một hệ thống nhúng
 - Cách bố trí các transistors
 - Kích thước/số lượng các transistors
 - Kết nối
- Lợi ích
 - Chất lượng tốt, kích thước nhỏ, công suất thấp
- Hạn chế
 - Giá NRE cao, thời gian đưa ra thị trường lâu

Bán chuyên dụng (semi-custom)

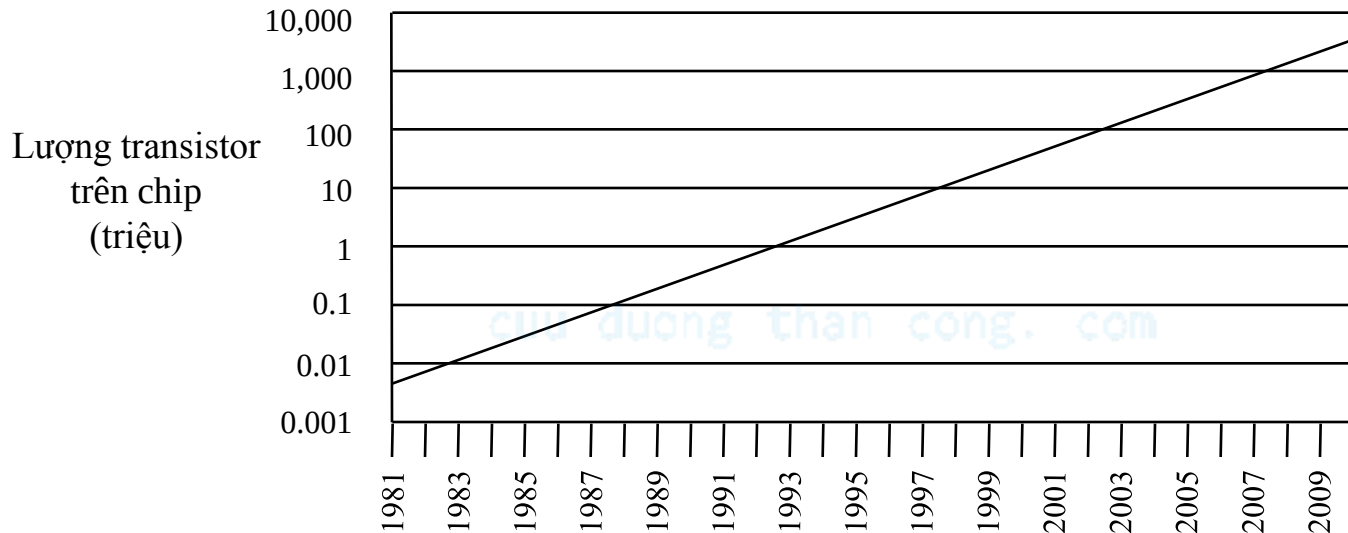
- Các lớp mức thấp được thiết kế một phần hay toàn bộ
 - Người thiết kế được quyền thay đổi kết nối hoặc thêm/bớt các khối
- Lợi ích
 - Chất lượng tốt, kích thước nhỏ, giá NRE giảm so với full-custom
- Nhược điểm
 - Vẫn cần hàng tuần hoặc hàng tháng để thiết kế

PLD (Thiết bị logic khả trình)

- Tất cả các lớp đã có sẵn
 - Người thiết kế có thể mua một IC
 - Các kết nối trên IC được thực hiện bằng cách lập trình
 - Nổi tiếng nhất là FPGA (Field-Programmable Gate Array)
- Lợi ích
 - Giá NRE thấp
- Nhược điểm
 - Kích thước lớn, đắt, tiêu thụ nhiều công suất và tốc độ chậm

Luật Moore

- Sự phát triển của hệ nhúng
 - Dự đoán đưa ra năm 1965 bởi người đồng sáng lập Intel
- Số lượng transistor tích hợp trên vi mạch tăng gấp đôi sau mỗi 18 tháng**



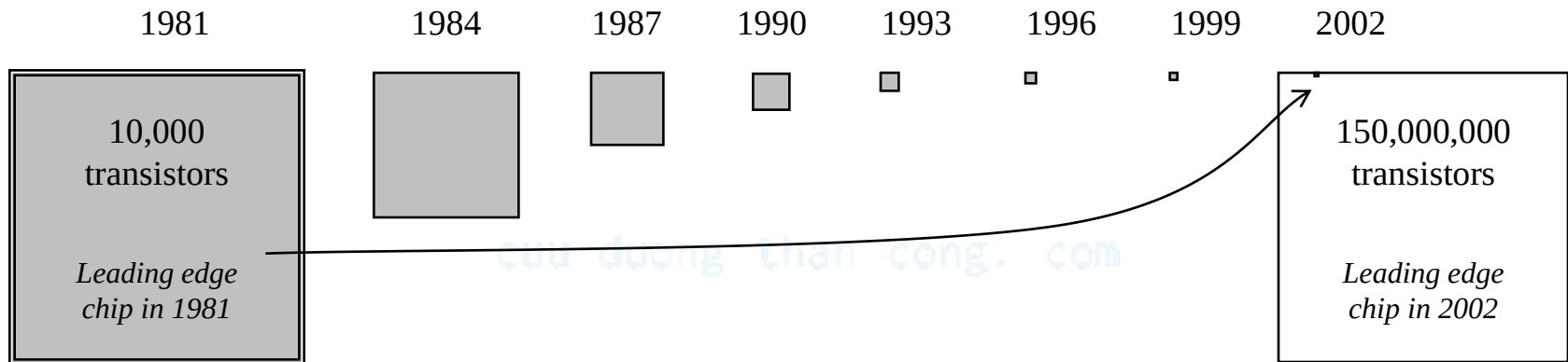
Luật Moore

- Điều ngạc nhiên
 - Tốc độ tăng trưởng này rất khó tưởng tượng, nhiều người ban đầu không tin tưởng

cuu duong than cong. com

cuu duong than cong. com

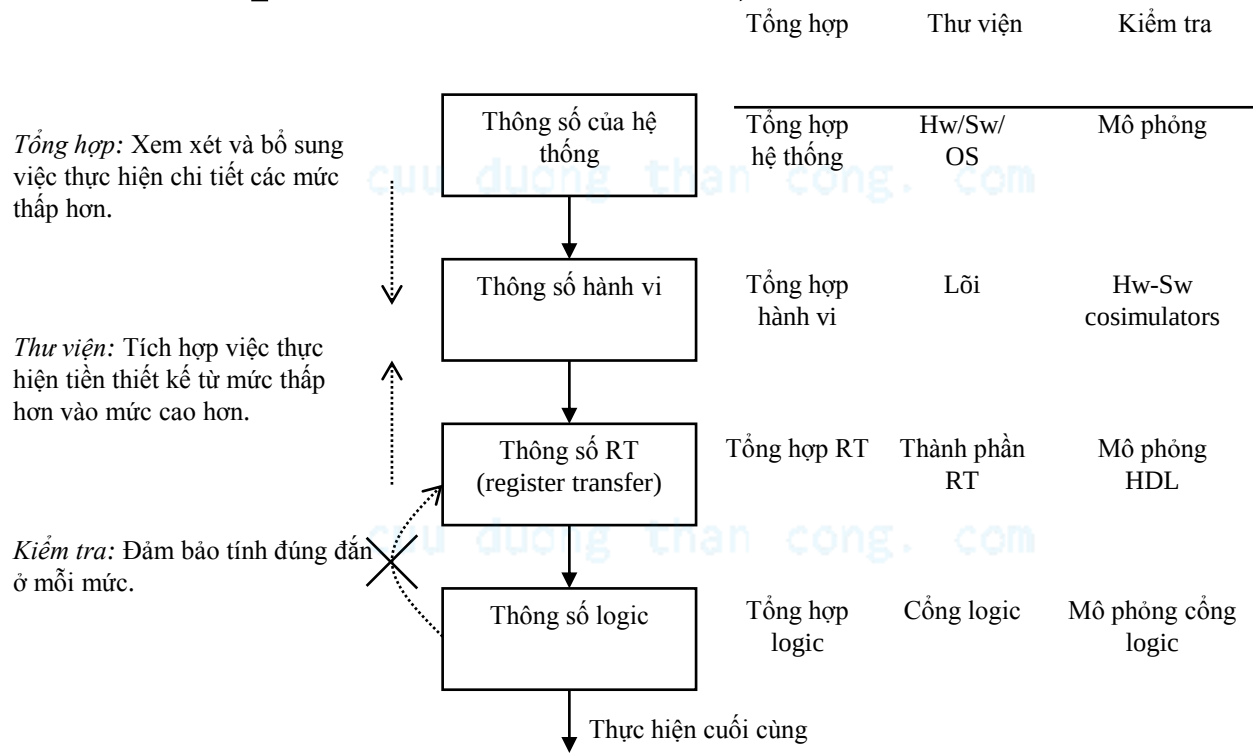
Minh họa đồ thị luật Moore



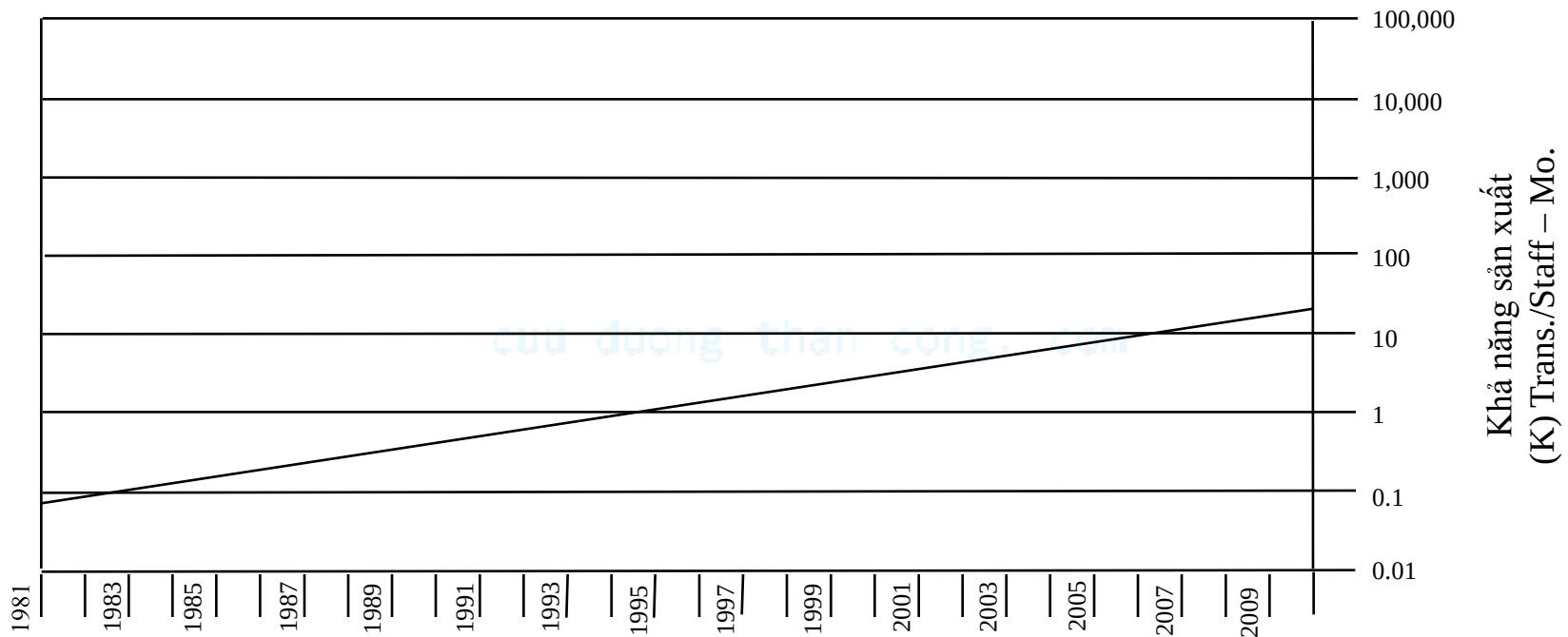
- Một chip năm 2002 có thể chứa khoảng 15,000 chips sản xuất năm 1981 (với cùng kích thước)

Công nghệ thiết kế

- Cách chúng ta biến ý tưởng thiết kế thành hiện thực (thực hiện quá trình thiết kế)



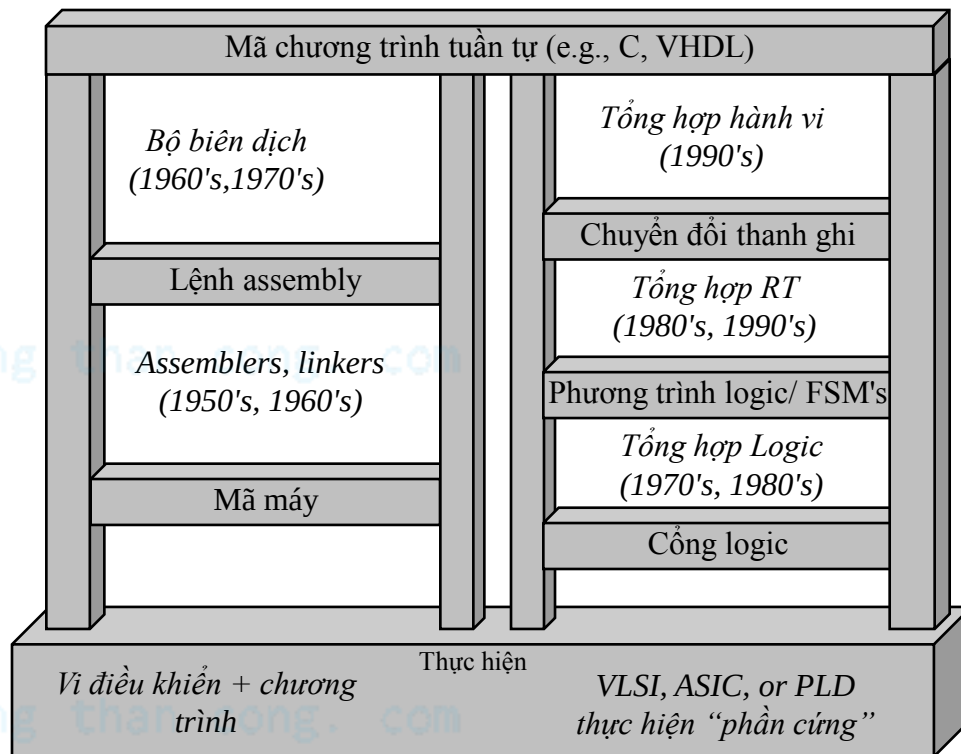
Công nghệ thiết kế tăng theo hàm số mũ



- Tăng theo hàm mũ trong nhiều thập kỷ qua

Đồng thiết kế (co-design)

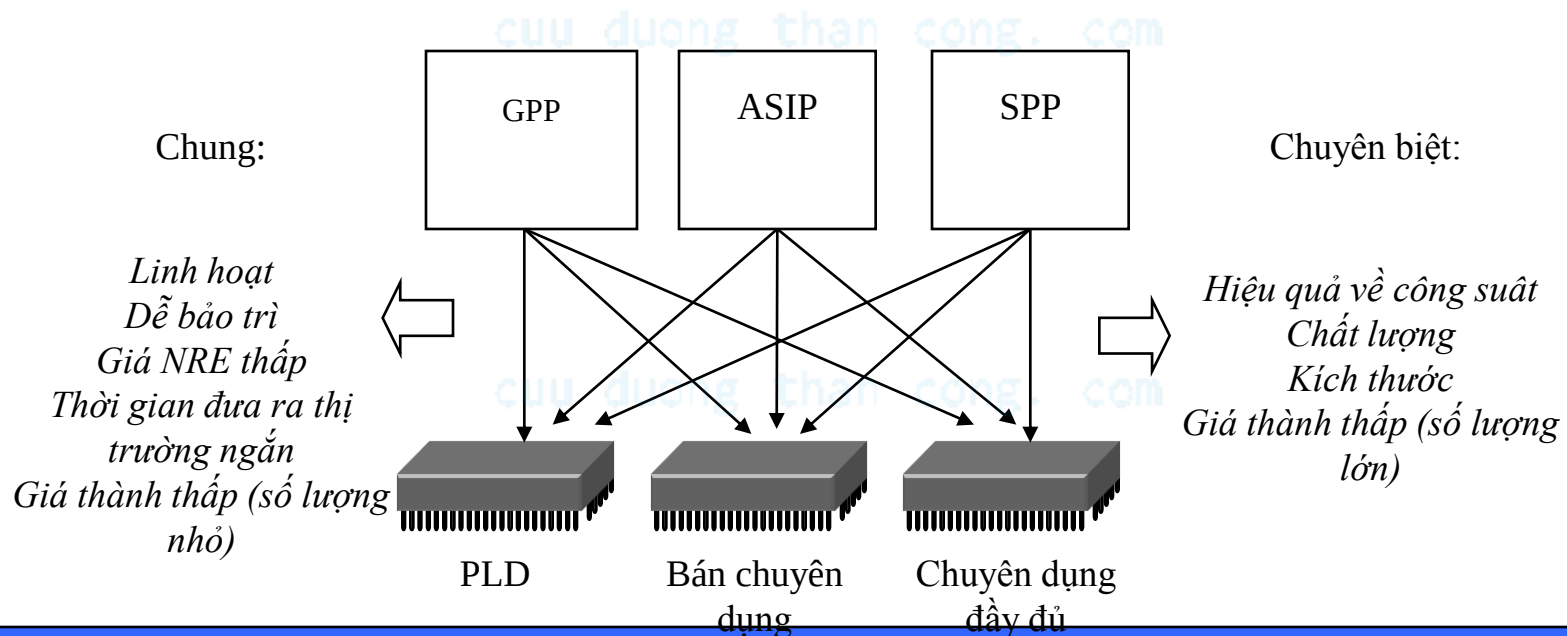
- Trong quá khứ:
 - Công nghệ thiết kế phần cứng và phần mềm rất khác nhau
 - Công nghệ thiết kế gần đây cho phép tổng hợp việc thiết kế HW và SW
- Đồng thiết kế phần cứng/phần mềm



Việc lựa chọn HW hay SW cho các chức năng nhất định là sự lựa chọn cân bằng giữa nhiều thông số thiết kế, như chất lượng, công suất, kích thước, giá thành,...

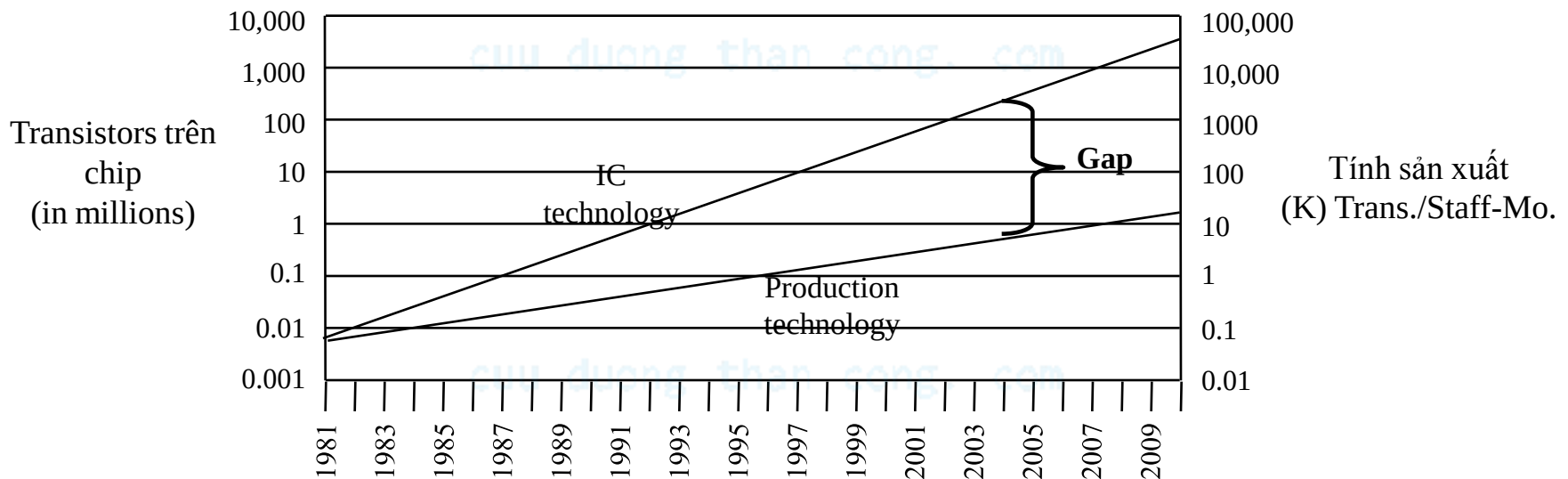
Tính độc lập của bộ xử lý và công nghệ IC

- Cân bằng cơ bản
 - Chung hay chuyên biệt
 - Tập trung vào công nghệ xử lý hay công nghệ IC
 - Hai công nghệ có tính độc lập với nhau



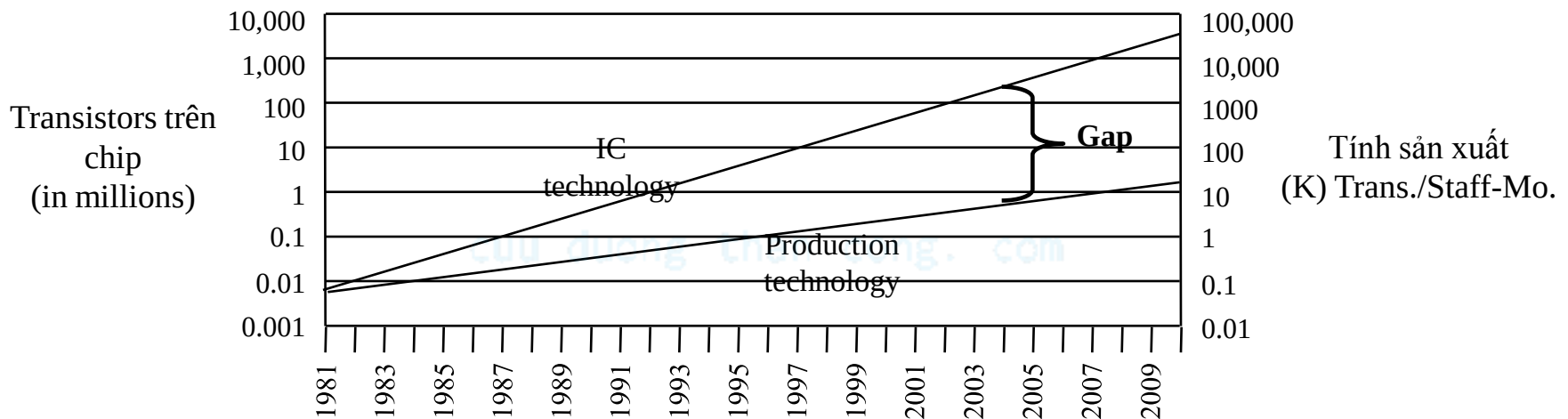
Khoảng trống thiết kế sản phẩm

- Trong khi các nhà thiết kế sản phẩm đã tăng trưởng ở một tốc độ ấn tượng trong thời gian qua, tuy nhiên tốc độ này không theo kịp tốc độ tích hợp chip, tạo ra một khoảng trống



Khoảng trống thiết kế sản phẩm

- Năm 1981, các chip yêu cầu 100 tháng lao động (người thiết kế)
 - 10,000 transistors / 100 transistors/month
- Năm 2002, các chip yêu cầu 30,000 tháng lao động (người thiết kế)
 - 150,000,000 / 5000 transistors/month
- Giá người thiết kế tăng từ \$1M tới \$300M



Tóm tắt

- Hệ thống nhúng có mặt ở mọi nơi
 - Thách thức chính: tối ưu các thông số thiết kế
 - Các thông số thiết kế có quan hệ ràng buộc với nhau
 - Vì vậy, có hiểu biết chung về cả phần cứng và phần mềm là rất cần thiết để tăng tính sản xuất của hệ nhúng
 - Ba công nghệ chìa khóa đối với hệ nhúng
 - Công nghệ bộ xử lý: Chức năng chung, chức năng đơn hay chức năng chuyên biệt
 - Công nghệ IC: Chuyên dụng đầy đủ, bán chuyên dụng, PLD
 - Công nghệ thiết kế: Biên dịch/tổng hợp, thư viện/IP, kiểm tra/thử nghiệm
-

Reference

- This document is used for educational purposes only
- This document contains material from other sources (incl. textbook, lecture notes, application notes, ...)
- Our thanks go to the contributors, references

CHƯƠNG 2: CẤU TRÚC PHẦN CỨNG HỆ THỐNG NHÚNG

Bài 2: Bộ xử lý chức năng đơn chuyên dụng (Custom single-purpose processors)

cuu duong than cong. com

Tổng quan

- Giới thiệu
- Mạch tổ hợp
- Mạch tuần tự
- Thiết kế bộ xử lý chức năng đơn
- Thiết kế bộ xử lý chức năng đơn chuyên dụng thời gian thực

Giới thiệu

*** Các loại vi điều khiển trên thị trường hiện nay:**

- **Freescale 68HC11 (8-bit)**
- **Intel 8051**
- **STMicroelectronics STM8S (8-bit), ST10 (16-bit) và STM32 (32-bit)**
- **Atmel AVR (8-bit), AVR32 (32-bit), và AT91SAM (32-bit)**
- **Freescale ColdFire (32-bit) và S08 (8-bit)**
- **Hitachi H8 (8-bit), Hitachi SuperH (32-bit)**
- **MIPS (32-bit PIC32)**
- **PIC (8-bit PIC16, PIC18, 16-bit dsPIC33 / PIC24)**
- **PowerPC ISE**
- **PSoC (Programmable System-on-Chip)**
- **Texas Instruments Microcontrollers MSP430 (16-bit), C2000 (32-bit), và Stellaris (32-bit)**
- **Toshiba TLCS-870 (8-bit/16-bit)**
- **Zilog eZ8 (16-bit), eZ80 (8-bit)**
- **Philips Semiconductors LPC2000, LPC900, LPC700**

Giới thiệu

- * Ứng dụng các loại vi xử lý và vi điều khiển được sử dụng trên thị trường Việt Nam hiện nay.
 - Có thể nói việc sử dụng các loại vi điều khiển và vi xử lý trong các thiết bị điện tử tự động ở Việt Nam rất đa dạng, phong phú tùy vào yêu cầu kỹ thuật và giá thành sản phẩm.
 - Đối với các thiết bị như các máy ATM, máy giặt thường sử dụng vi điều khiển 8051, các bộ điều khiển trong robot công nghiệp, trong hệ thống ô tô thường sử dụng PIC, AVR, PSoC, còn trong điện thoại sử dụng các chip ARM...

cuu duong than cong. com

Giới thiệu

1. Vi điều khiển 8051.

- Intel 8051 - là vi điều khiển đơn tinh thể kiến trúc Harvard, lần đầu tiên được sản xuất bởi Intel năm 1980, để dùng trong các hệ thống nhúng. Trong những năm 1980 và đầu những năm 1990 đã rất nổi tiếng. Tuy nhiên hiện tại đã cũ và được thay thế bằng các thiết bị hiện đại hơn, với các lõi phối hợp 8051, được sản xuất bởi hơn 20 nhà sản xuất độc lập như Atmel, Maxim IC (công ty con của Dallas Semiconductor), NXP Semiconductors (Philips Semiconductor trước đây), Winbond, Silicon Laboratories, Texas Instruments và Cypress Semiconductor. Tên gọi chính thức của họ vi điều khiển Intel 8051 - MCS 51.
- Những vi điều khiển Intel 8051 được sản xuất với việc dùng công nghệ MOSFET, những những bản sau, chứa kí hiệu “C” trong tên, như 80C51, dùng công nghệ CMOS và yêu cầu công suất thấp, hơn những cái MOSFET trước (điều này cho phép trang bị cho các thiết bị với nguồn là pin).

cuu duong than cong. com

Giới thiệu

1. Vi điều khiển 8051.

* Các thông số kỹ thuật:

8 bit ALU, 8 bit thanh ghi.

8 bit dữ liệu bus

16 bit địa chỉ bus vì vậy không gian bộ nhớ tối đa cho ROM và RAM lên tới 64 kb

Bộ nhớ dữ liệu SRAM 128 bytes

Bộ nhớ chương trình ROM 4 kb.

32 chân vào/ra đa hướng.

Giao tiếp nối tiếp UART.

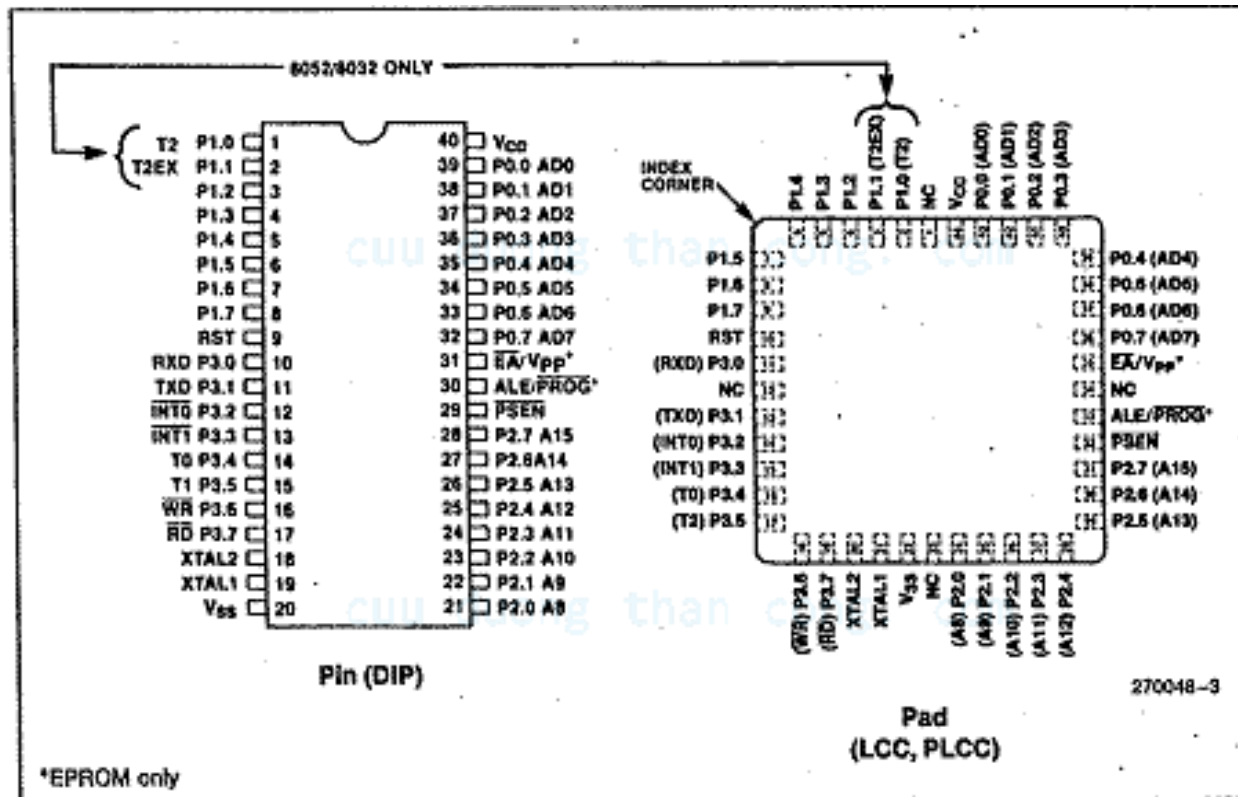
Hai bộ timer/counter 16 bit.

Hai ngắt ngoài.

Giới thiệu

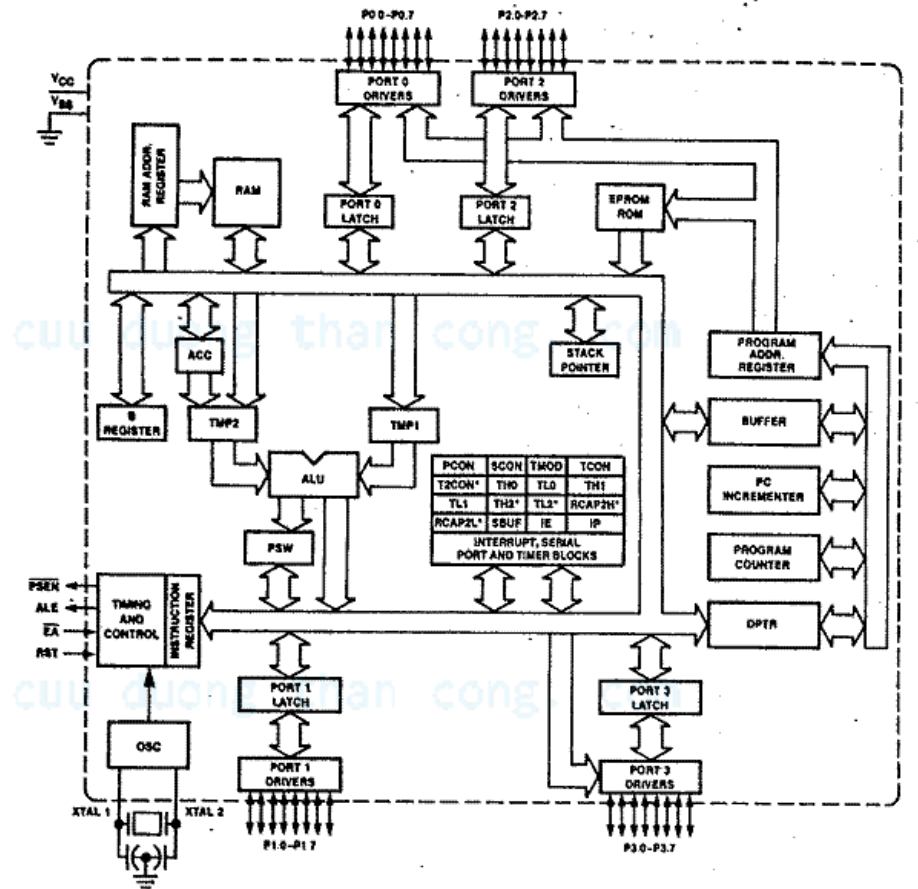
1. Vi điều khiển 8051.

Sơ đồ chân của 8051



Giới thiệu

1. Vi điều khiển 8051. Sơ đồ khối điều khiển:



Giới thiệu

1. Vi điều khiển 8051.

Lập trình cho 8051:

Các nhà sản xuất 8051 đều hỗ trợ ngôn ngữ lập trình Assembler tuy nhiên ngôn ngữ này thường ít được dùng cho những ứng dụng lớn do tính phù hợp của nó, vì vậy trong các ứng dụng thực tế hay sử dụng ngôn ngữ C. Ngoài ra còn một số ngôn ngữ khác được phát triển cho 8051 như Pascal, Basic, Forth.

cuu duong than cong. com

cuu duong than cong. com

Giới thiệu

2. Vi điều khiển AVR.

Là dòng vi điều khiển do hãng Atmel sản xuất có nhiều loại AVR như:

- 32-bit AVR UC3.
- 8/16-bit AVR XMEGA.
- 8-bit mega AVR.
- 8-bit tiny AVR.

•Vi điều khiển Atmega 16:

Là vi điều khiển 8 bit với tiêu thụ điện năng thấp dựa trên kiến trúc RISC (Reduced Instruction Set Computer). Vào ra Analog – digital và ngược lại. Với công nghệ này cho phép các lệnh thực thi chỉ trong một chu kỳ xung nhịp, vì thế tốc độ xử lý dữ liệu có thể đạt đến 1 triệu lệnh trên giây ở tần số 1Mhz. Vi điều khiển này cho phép người thiết kế có thể tối ưu hoá chế độ tiêu thụ năng lượng mà vẫn đảm bảo tốc độ xử lý.

Lỗi AVR có tập lệnh phong phú với số lượng với 32 thanh ghi làm việc chung với nhau. Tất cả 32 thanh ghi đều được nối trực tiếp với ALU (Arithmetic Logic Unit), cho phép 2 thanh ghi truy cập độc lập trong một chỉ lệnh đơn trong một chu kỳ xung nhịp. Kiến trúc đạt được có tốc độ xử lý nhanh gấp 10 lần vi điều khiển dạng CISC (Complex Instruction Set Computer) thông thường.

Giới thiệu

2. Vi điều khiển AVR.

Thông số kỹ thuật:

- Bộ nhớ:

Flash 16KB

EEPROM 512 Byte

SRAM 1KB.

- Ngoại vi:

Hai timer 8 bit

Một timer 16 bit

Bộ counter với tần số riêng

Bốn bộ điều chế độ rộng xung PWM.

Tám kênh ADC 10 bit.

USART.

Giao tiếp SPI, Giao diện I2C.

Watchdog timer.

Bộ so sánh tương tự trên chip.

Giới thiệu

2. Vi điều khiển AVR.

Thông số kỹ thuật:

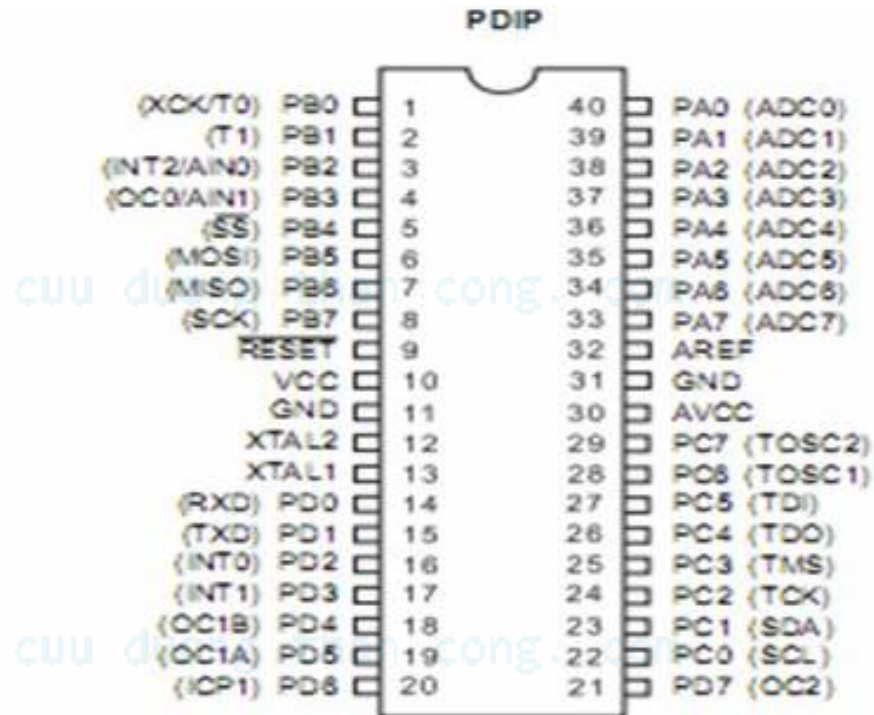
Tính năng:

- Tập lệnh gồm 131 lệnh, hầu hết thực hiện trong một chu kỳ máy.
- Xử lý 16 triệu lệnh ở tần số 16 MHZ.
- 32 chân vào/ra có thể lập trình được.
- Sáu chế độ sleep .
- 40 pin kiểu PDIP, 44 pin kiểu TQFP và kiểu QFL/MLF.
- 32 thanh ghi 8 bit đa dụng.
- Ngắt trong và ngắt ngoài.
- Điện áp hoạt động từ 2,7-5,5V cho Atmega 16A.

Giới thiệu

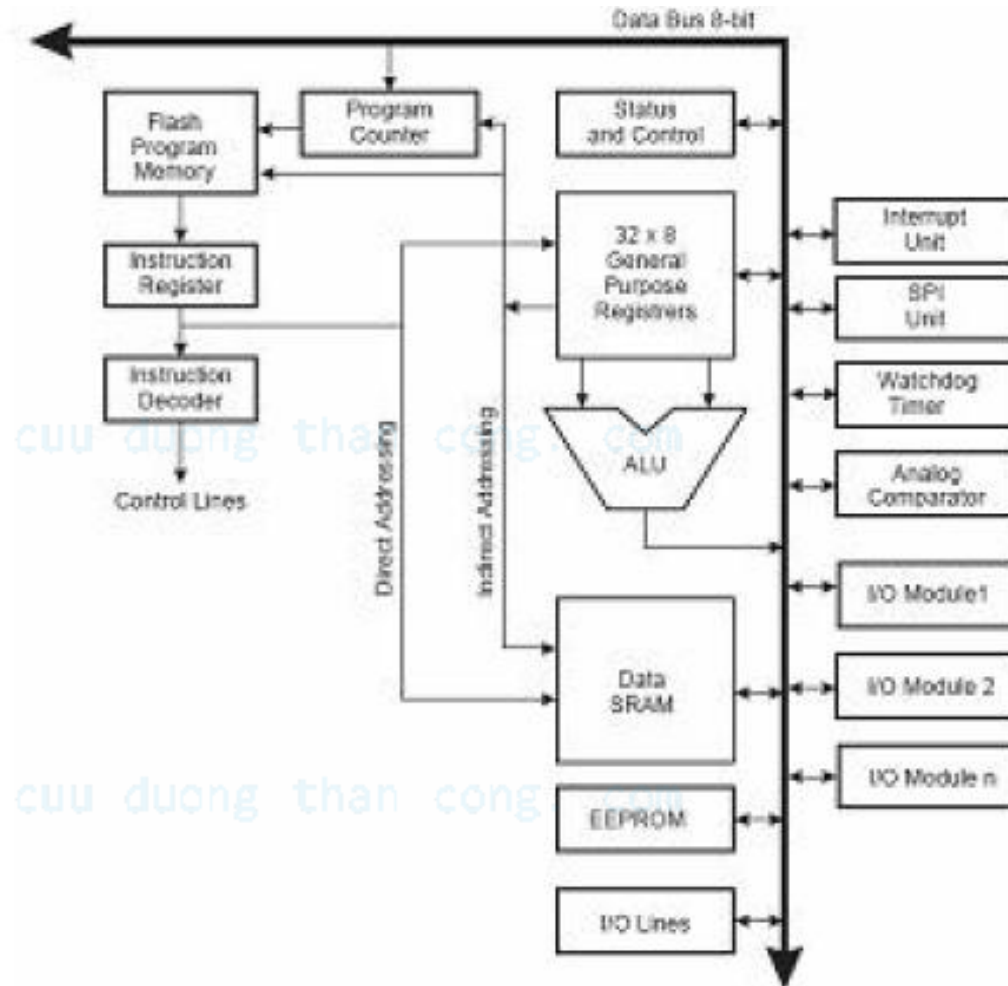
2. Vi điều khiển AVR.

Sơ đồ chân



Giới thiệu

2. Vi điều khiển AVR. Sơ đồ khối điều khiển



Giới thiệu

2. Vi điều khiển AVR.

Atmega 16 được hỗ trợ đầy đủ phần mềm và công cụ phát triển hệ thống bao gồm:

Trình dịch Assembly như AVR studio của Atmel, Trình dịch C như win AVR, CodeVisionAVR C, ICCAVR. C - CMPPILER của GNU... Trình dịch C đã được nhiều người dùng và đánh giá tương đối mạnh, dễ tiếp cận đối với những người bắt đầu tìm hiểu AVR, đó là trình dịch CodeVisionAVR C. Phần mềm này hỗ trợ nhiều ứng dụng và có nhiều hàm có sẵn nên việc lập trình tốt hơn.

cuu duong than cong. com

cuu duong than cong. com

Giới thiệu

3. Vi điều khiển PIC.

PIC là một họ vi điều khiển RISC được sản xuất bởi công ty Microchip Technology. Dòng PIC đầu tiên là PIC1650 được phát triển bởi Microelectronics Division thuộc General Instrument.

PIC bắt nguồn là chữ viết tắt của "Programmable Intelligent Computer" (Máy tính khả trình thông minh). Là vi điều khiển với kiến trúc RISC thực thi một lệnh với một chu kỳ máy (bằng bốn chu kỳ của bộ dao động). Ngày nay có nhiều dòng PIC được sản xuất với hàng loạt các mô đun ngoại vi tích hợp sẵn như ADC, PWM, USART, SPI...với bộ nhớ chương trình từ 512 word đến 32 Kword.

Các họ vi điều khiển PIC:

- Họ 8 bit: PIC 10/ PIC 12/ PIC 16/ PIC 18
- Họ 16 bit: PIC 24F/ PIC 24H/ dsPIC 30/ dsPIC 33
- Họ 32 bit: PIC 32.

• Comparison of PIC families

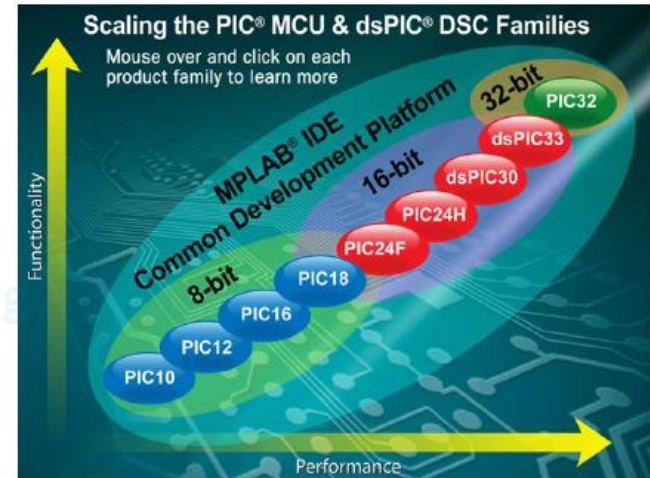
PIC family	Stack size (words)	Instruction word size	Number of instructions	Interrupt vectors
12CXXX/12FXXX	2	12- or 14-bit	33	None
16C5XX/16F5XX	2	12-bit	33	None
16CXXX/16FXXX	8	14-bit	35	1
17CXXX	16	16-bit	58, including hardware multiply	4
18CXXX/18FXXX	32	16-bit	75, including hardware multiply	2 (prioritised)

Giới thiệu

•3. Vi điều khiển PIC.

Thông số kỹ thuật

- Chân vào/ra I/O có thể lập trình được.
- Flash và ROM có thể tùy chọn từ 256 byte đến 512 Kbyte
- Bộ dao động bên trong.
- 8/16/32 bit Timers.
- Bộ nhớ EEPROM nội
- Chuẩn giao tiếp nối tiếp đồng bộ và không đồng bộ USART
- MSSP Peripheral cho giao tiếp I2C và SPI
- Các chế độ so sánh, bắt giữ và điều chế độ rộng xung PWM.
- Bộ so sánh điện áp.
- Bộ chuyển đổi ADC (tần số có thể lên tới 1 MHz).
- Hỗ trợ các giao thức USB, CAN, Ethernet.
- Mô đun điều khiển động cơ, mô đun đọc encoder.
- Hỗ trợ bộ nhớ ngoài.
- DSP những tính năng xử lý tín hiệu số (dsPIC)



Giới thiệu

•3. Vi điều khiển PIC.

Thông số kỹ thuật

Device number	No. of pins*	Clock speed	Memory (K = Kbytes, i.e. 1024 bytes)	Peripherals/special features
16F84A	18	DC to 20MHz	1K program memory, 68 bytes RAM, 64 bytes EEPROM	1 8-bit timer 1 5-bit parallel port 1 8-bit parallel port
16LF84A	As above	As above	As above	As above, with extended supply voltage range
16F84A-04	As above	DC to 4MHz	As above	As above
16F873A	28	DC to 20MHz	4K program memory, 192 bytes RAM, 128 bytes EEPROM	3 parallel ports, 3 counter/timers, 2 capture/compare/PWM modules, 2 serial communication modules, 5 10-bit ADC channels, 2 analog comparators
16F874A	40	DC to 20MHz	4K program memory, 192 bytes RAM, 128 bytes EEPROM	5 parallel ports, 3 counter/timers, 2 capture/compare/PWM modules, 2 serial communication modules, 8 10-bit ADC channels, 2 analog comparators

Giới thiệu

•3. Vi điều khiển PIC.

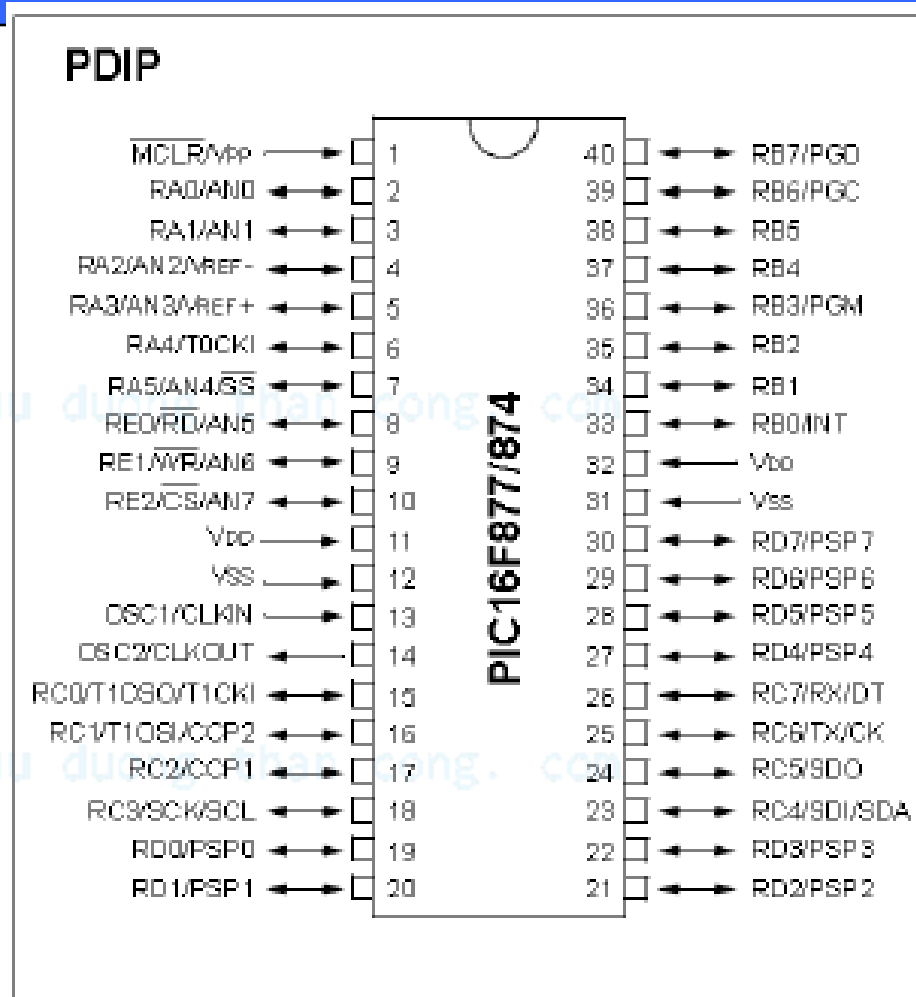
Thông số kỹ thuật

16F876A	28	DC to 20 MHz	8K program memory 368 bytes RAM, 256 bytes EEPROM	3 parallel ports, 3 counter/timers, 2 capture/compare/PWM modules, 2 serial communication modules, 5 10-bit ADC channels, 2 analog comparators
16F877A	40	DC to 20 MHz	8K program memory 368 bytes RAM, 256 bytes EEPROM	5 parallel ports, 3 counter/timers, 2 capture/compare/PWM modules, 2 serial communication modules, 8 10-bit ADC channels, 2 analog comparators

Giới thiệu

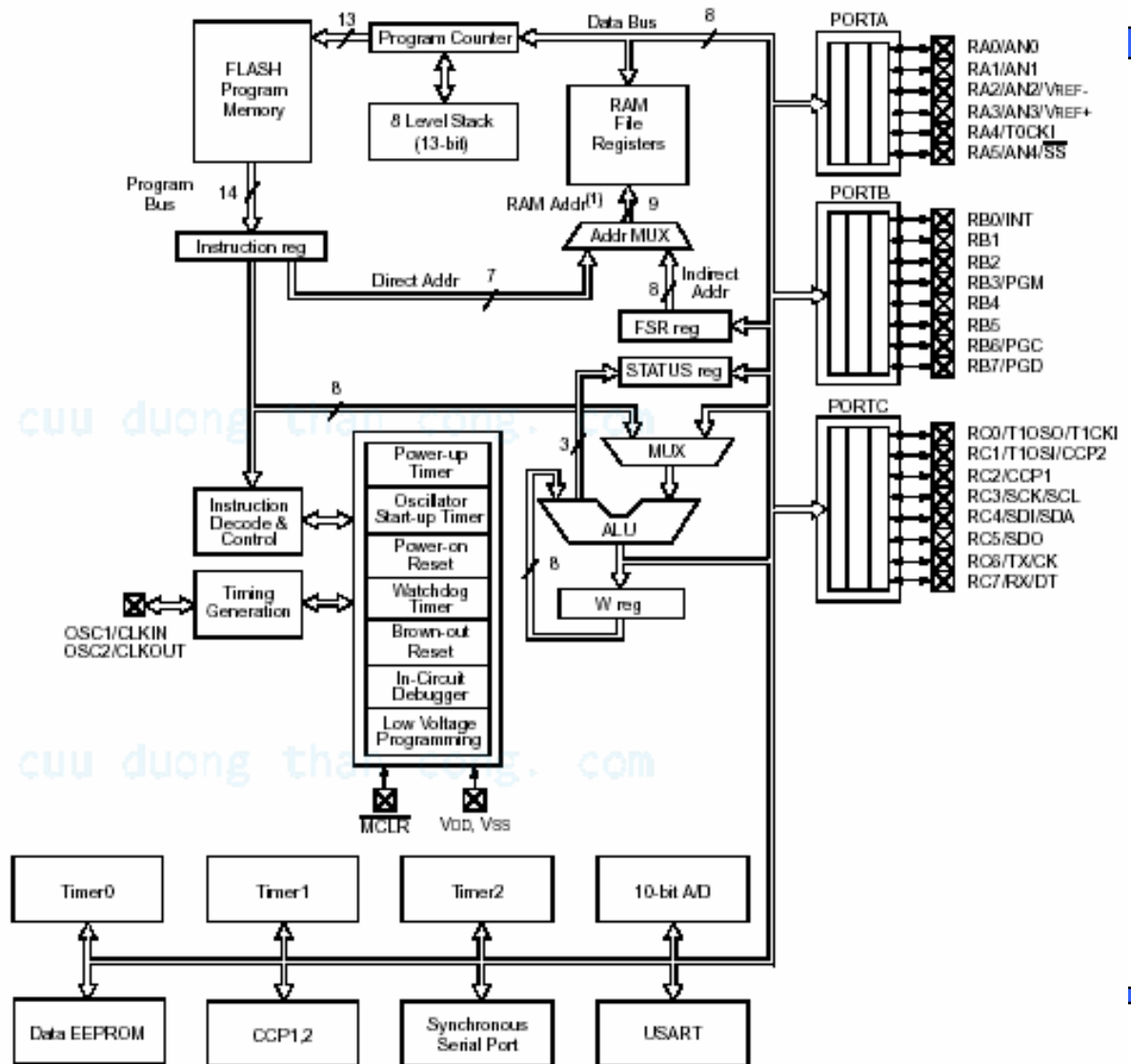
3. Vi điều khiển PIC.

Sơ đồ bố trí chân



Giới thiệu

3. Vi điều khiển PIC. Sơ đồ khối chức năng



Giới thiệu

3. Vi điều khiển PIC.

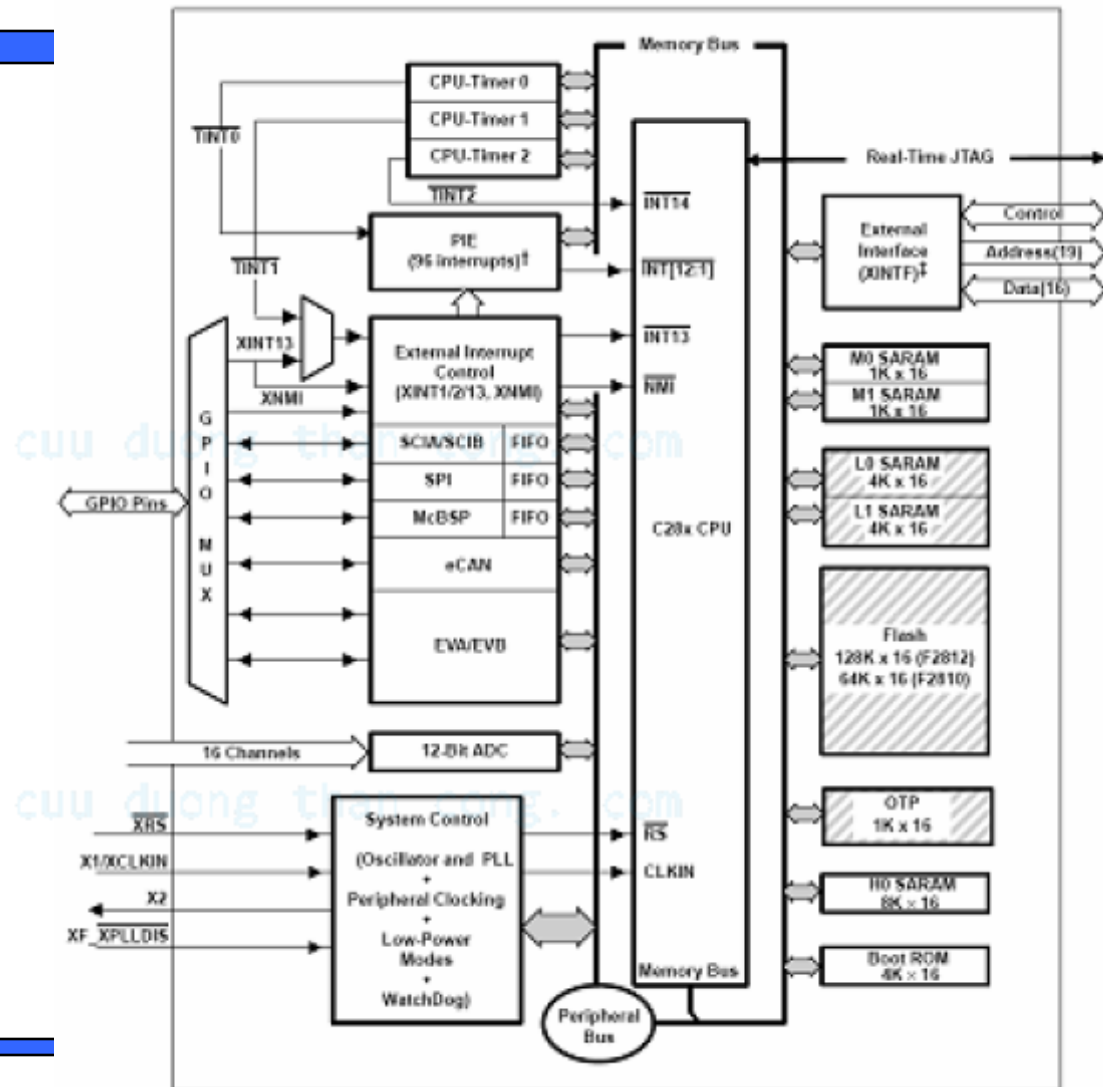
Lập trình cho PIC

Hãng Microchip cung cấp môi trường lập trình MPLAB nó bao gồm phần mềm mô phỏng, trình dịch ASM, liên kết và gỡ rối. Ngoài ra hãng này cũng bán trình biên dịch C cho các dòng PIC18 và dsPIC tích hợp trong MPLAB.

Ngoài ra còn một số công ty khác cung cấp trình biên dịch C, PASCAL, BASIC cho PIC đó có thể là phần mềm thương mại hoặc phần mềm mã nguồn mở.

Giới thiệu

4. Chip DSP



Giới thiệu

5. Vi điều khiển ARM.

Cấu trúc ARM (viết tắt từ tên gốc là Acorn RISC Machine) là một loại cấu trúc vi xử lý 32-bit kiểu RISC được sử dụng rộng rãi trong các thiết kế nhúng. Được phát triển lần đầu trong một dự án của công ty máy tính Acorn. Do có đặc điểm tiết kiệm năng lượng, các bộ CPU ARM chiếm ưu thế trong các sản phẩm điện tử di động, mà với các sản phẩm này việc tiêu tán công suất thấp là một mục tiêu thiết kế quan trọng hàng đầu.

Ngày nay, hơn 75% CPU nhúng 32-bit là thuộc họ ARM, điều này khiến ARM trở thành cấu trúc 32-bit được sản xuất nhiều nhất trên thế giới. CPU ARM được tìm thấy khắp nơi trong các sản phẩm thương mại điện tử, từ thiết bị cầm tay (PDA, điện thoại di động, máy đa phương tiện, máy trò chơi cầm tay, và máy tính cầm tay) cho đến các thiết bị ngoại vi máy tính (ổ đĩa cứng, bộ định tuyến để bàn.). Một nhánh nổi tiếng của họ ARM là các vi xử lý Xscale của Intel.

cuu duong than cong. com

Giới thiệu

5. Vi điều khiển ARM.

Giới thiệu về vi điều khiển LPC2148:

Là dòng vi điều khiển ARM được sản xuất bởi hãng Philips.

- Vi điều khiển 16/32-bit ARM7TDMI-S
- 40k RAM tĩnh (8k +32k), 512k flash
- Tích hợp USB 2.0
- Hỗ trợ hai bộ ADC 10 bit
- Một bộ DAC 10 bit
- 2 bộ timer 32 bit, 6 ngõ điều chế độ rộng xung
- Đồng hồ thời gian thực hỗ trợ tần số 32kHz
- Khả năng thiết lập chế độ ưu tiên và định địa chỉ cho ngắt
- 45 chân GPIO vào ra đa dụng
- 9 chân ngắt ngoài (tích cực cạnh hoặc tích cực mức)
- CPU clock đạt tối đa 60MHz thông qua bộ PLL lập trình được
- Xung PLCK hoạt động độc lập.



LPC1754FBD80



AT91SAM7S64-AU



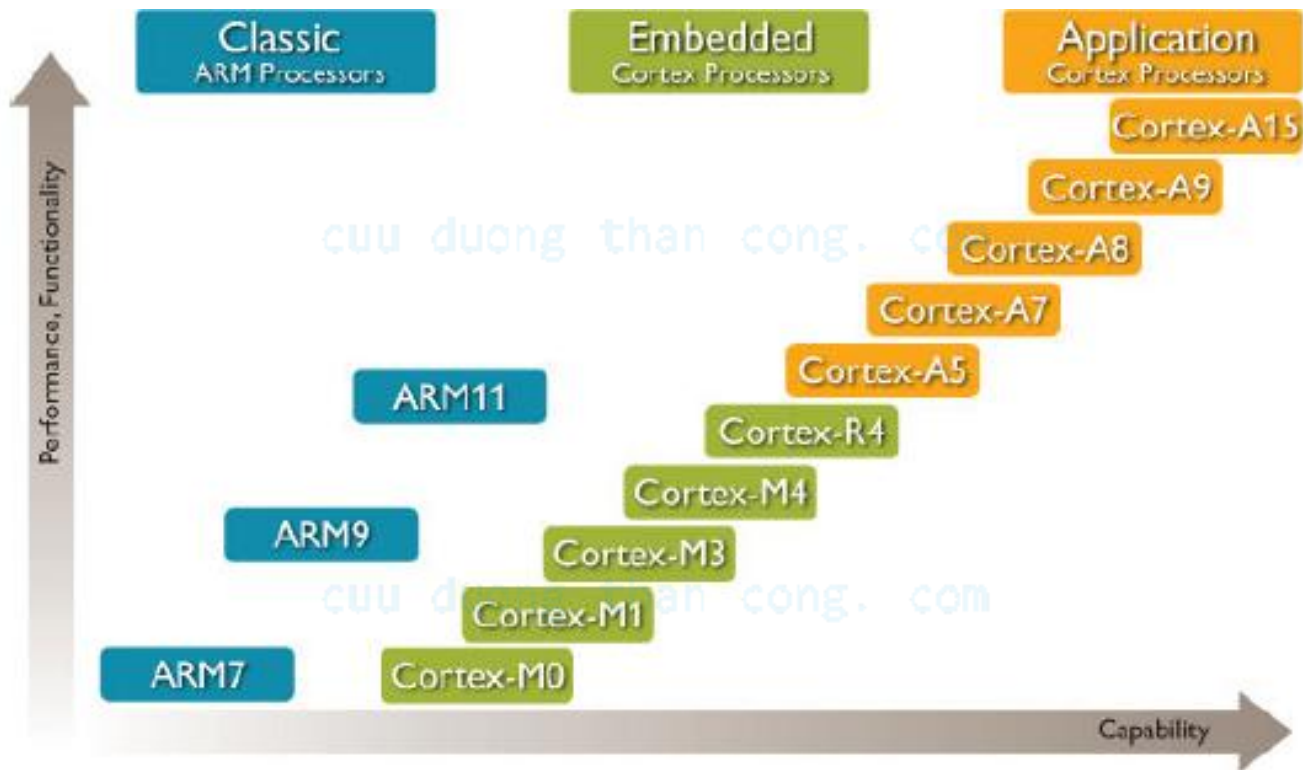
STM32F103RCT6



LM3S3749

Giới thiệu

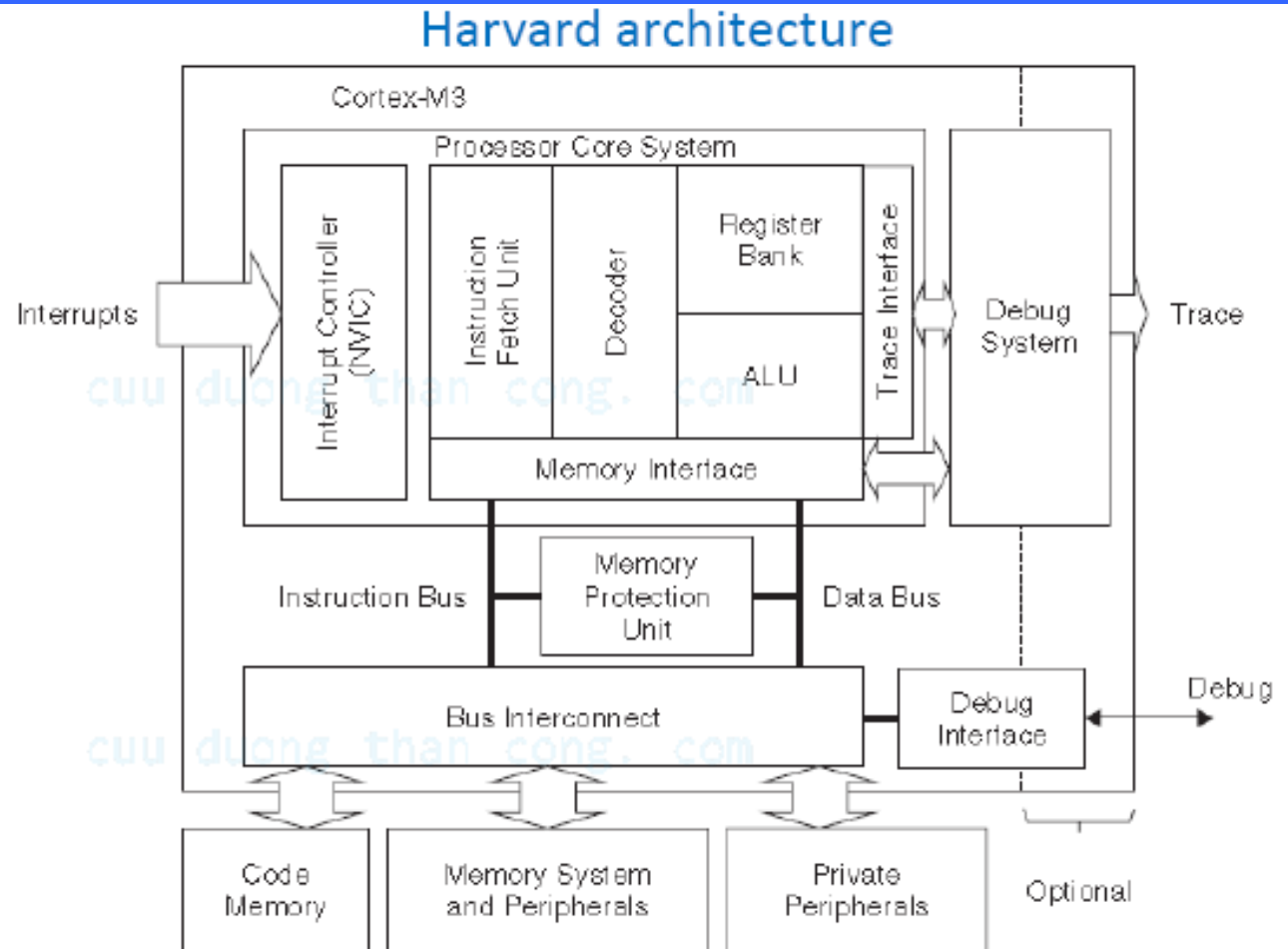
5. Vi điều khiển ARM.



Giới thiệu

5. Vi điều khiển

Sơ đồ kiến trúc



Giới thiệu

5. Vi điều khiển ARM.

Ngôn ngữ lập trình chính cho ARM hiện nay là ngôn ngữ C. Các trình biên dịch cho ARM thường được dùng:

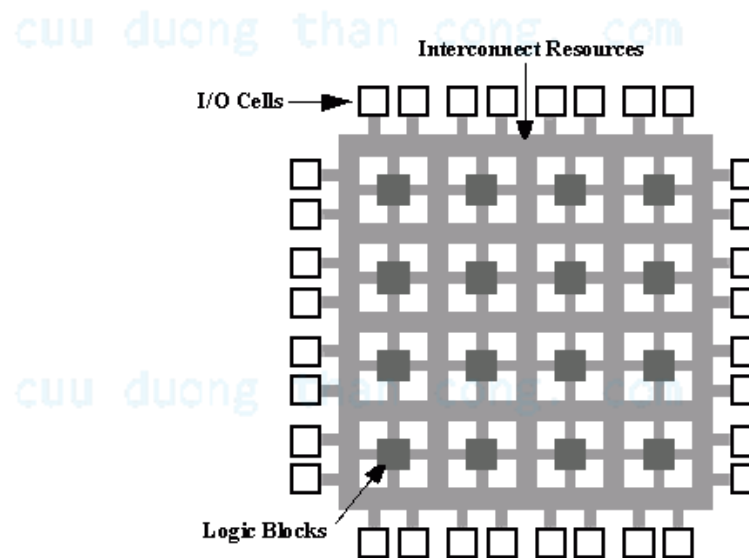
- Keil ARM.
- IAR.
- HTPICC for ARM.
- ImageCraft ICCV7 for ARM

Giới thiệu

6. FPGA và các ứng dụng nhúng

Hệ thống nhúng (embedded system) bắt đầu từ các họ Vi điều khiển 8051 rồi đến PIC, AVR và cao hơn nữa là họ ARM, AVR32 hay PSoC. Với FPGA tiếp cận hoàn toàn khác.

Vậy bao nhiêu công sức, kinh nghiệm về vi điều khiển và cả những công trình nghiên cứu bị bỏ rơi? thực ra FPGA chỉ phát huy sức mạnh của nó khi được ghép nối với vi điều khiển. Đó cũng chính là mục đích và tư tưởng thiết kế của co-design.



Giới thiệu

6. FPGA và các ứng dụng nhúng

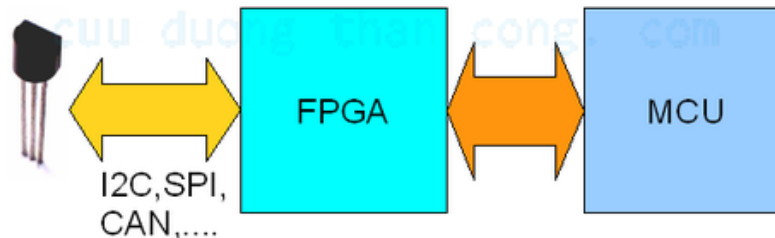
Co-design kết hợp năng lực về phần cứng của FPGA với ưu thế xử lý phần mềm của Vi điều khiển để tạo nên một hệ thống đầy sức mạnh.

Ví dụ : Muốn thiết kế một ứng dụng đo nhiệt độ phòng với cảm biến nhiệt có giao tiếp I²C.

Nếu chỉ dùng MCU thông thường không có giao tiếp I²C thì sẽ gặp rất nhiều khó khăn (Phải lập trình ngắt, bắt sườn, mức của xung,...).

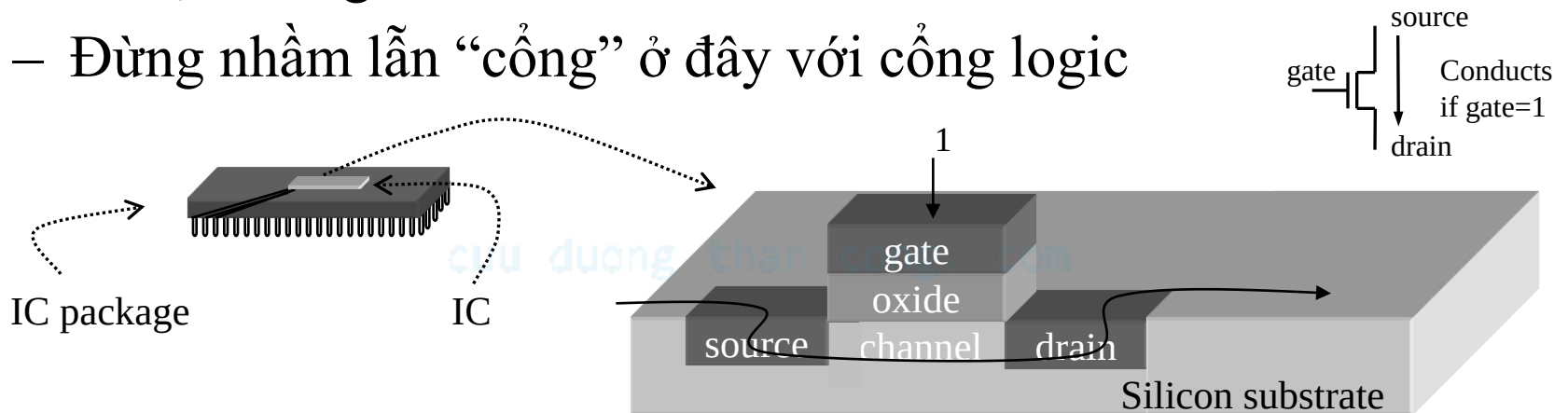
Còn nếu chỉ sử dụng FPGA trong ứng dụng này cũng không ổn vì lúc đó bạn sẽ gặp khó khăn nhất định trong các tính toán số học.

Ví dụ cảm biến đo nhiệt độ bằng đơn vị độ F, trong khi ta muốn hiển thị độ C, mà muốn thực hiện các phép toán cộng trừ nhân chia để chuyển đổi độ F với độ C bằng FPGA là không hề đơn giản. Trong trường hợp này ta thiết kế theo phương thức co-design. FPGA phụ trách giao tiếp với cảm biến I²C và trả về các số liệu thô để MCU thực hiện các tính toán số học.



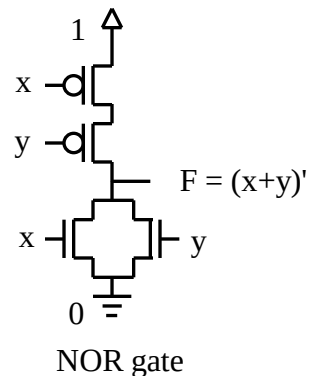
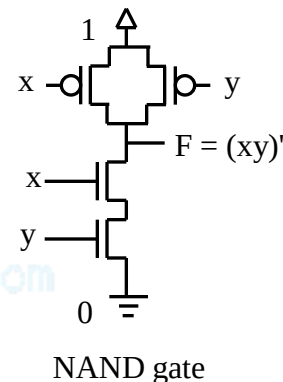
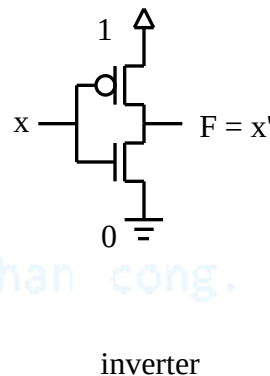
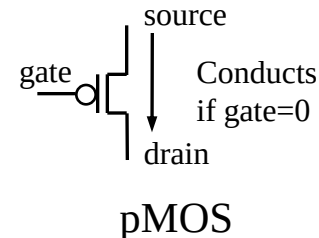
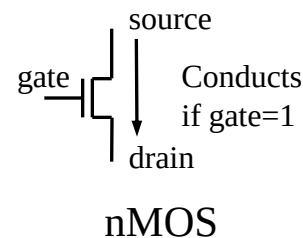
Transistor CMOS trên silicon

- Transistor
 - Là phần điện tử cơ bản trên mạch số
 - Hoạt động như một chuyển mạch on/off
 - Điện áp ở cực “cổng” điều khiển dòng giữa cực nguồn và cực máng
 - Đừng nhầm lẫn “cổng” ở đây với cổng logic

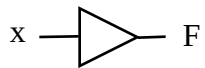


Thực hiện transistor CMOS

- CMOS - Complementary Metal Oxide Semiconductor
- Chúng ta quan tâm đến mức logic
 - Điện hình 0 là 0V, 1 là 5V
- Hai kiểu CMOS cơ bản
 - nMOS dẫn nếu “công” = 1
 - pMOS dẫn nếu “công” = 0
 - Vì thế gọi là “complementary”
- Các cổng cơ bản
 - Đảo, NAND, NOR

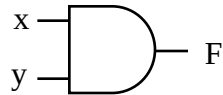


Các cổng logic cơ bản



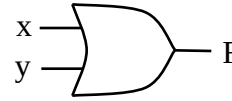
x	F
0	0
1	1

$F = x$
Driver



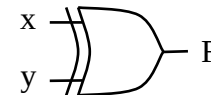
x	y	F
0	0	0
0	1	0
1	0	0
1	1	1

$F = x y$
AND



x	y	F
0	0	0
0	1	1
1	0	1
1	1	1

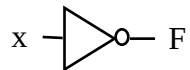
$F = x + y$
OR



x	y	F
0	0	0
0	1	1
1	0	1
1	1	0

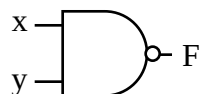
$F = x \oplus y$
XOR

cuu duong than cong. com



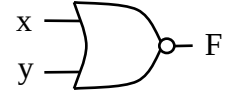
x	F
0	1
1	0

$F = x'$
Inverter



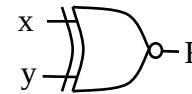
x	y	F
0	0	1
0	1	1
1	0	1
1	1	0

$F = (x y)'$
NAND



x	y	F
0	0	1
0	1	0
1	0	0
1	1	0

$F = (x + y)'$
NOR



x	y	F
0	0	1
0	1	0
1	0	0
1	1	1

$F = x \odot y$
XNOR

cuu duong than cong. com

Thiết kế logic tổ hợp

A) Mô tả vấn đề

y là 1 nếu a là 1, hoặc b và c là 1. z là 1 nếu b hoặc c là 1, nhưng không phải cả hai, hoặc nếu tất cả là 1.

B) Bảng chân lý

Inputs			Outputs	
a	b	c	y	z
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	1	0
1	0	1	1	1
1	1	0	1	1
1	1	1	1	1

C) Phương trình đầu ra

$$y = a'bc + ab'c' + ab'c + abc' + abc$$

$$z = a'b'c + a'bc' + ab'c + abc' + abc$$

D) Phương trình đầu ra tối thiểu

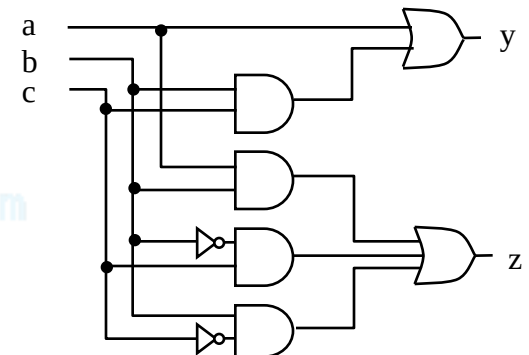
y	a	bc			
		00	01	11	10
0	0	0	0	1	0
1	1	1	1	1	1

$$y = a + bc$$

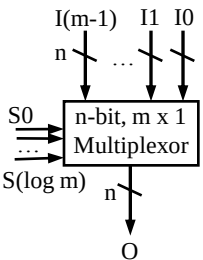
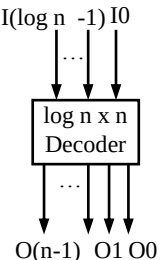
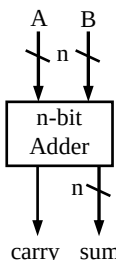
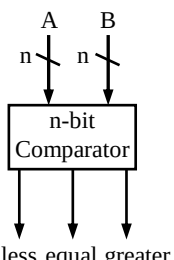
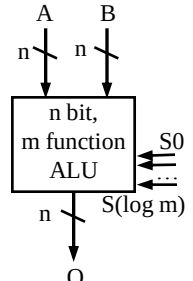
z	a	bc			
		00	01	11	10
0	0	0	1	0	1
1	1	0	1	1	1

$$z = ab + b'c + bc'$$

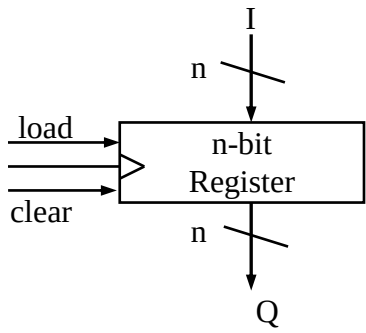
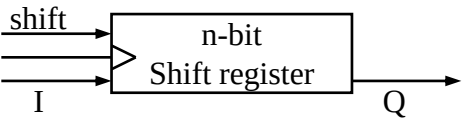
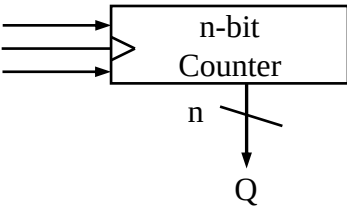
E) Mạch logic



Các phần tử mạch tổ hợp

				
$O =$ I_0 nếu $S=0..00$ I_1 nếu $S=0..01$ $\dots$ $I_{(m-1)}$ nếu $S=1..11$	$O_0 = 1$ nếu $I=0..00$ $O_1 = 1$ nếu $I=0..01$ $\dots$ $O_{(n-1)} = 1$ nếu $I=1..11$	$sum = A + B$ (n bit đầu) $carry = \text{bit thứ } (n+1) \text{ của } A+B$	$Less = 1$ nếu $A < B$ $equal = 1$ nếu $A = B$ $greater = 1$ nếu $A > B$	$O = A \text{ op } B$ op được xác định bởi S.
	Với đầu vào enable $e \rightarrow$ tất cả các bit đầu ra là 0 nếu $e=0$	Với đầu vào Carry-in $C_i \rightarrow$ $sum = A + B + C_i$		Có thể có các đầu ra trạng thái.

Các phần tử mạch tuần tự

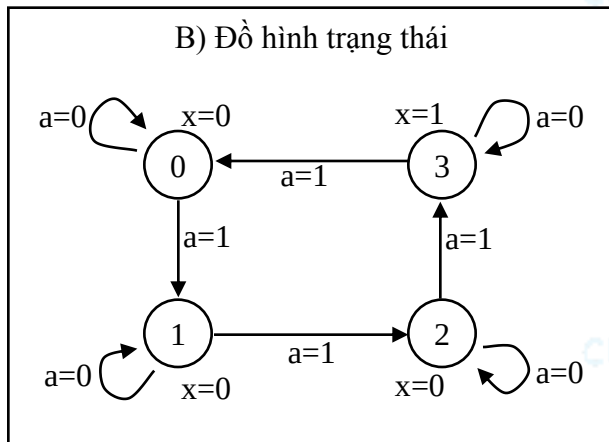
		
Q = 0 nếu clear=1, I nếu load=1 and clock=1, Q(previous) nếu khác.	Q = lsb - Nội dung được dịch - I được lưu trong msb	Q = 0 nếu clear=1, Q(prev)+1 nếu count=1 và clock=1.

Thiết kế mạch tuần tự

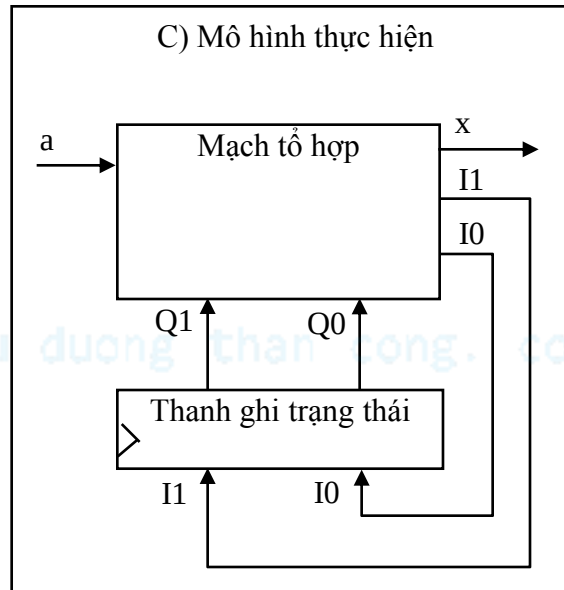
A) Mô tả vấn đề

Giả sử ta muốn xây dựng 1 bộ chia xung clock. Đầu ra có tín hiệu “1” sau mỗi 4 xung clock.

B) Đồ hình trạng thái



C) Mô hình thực hiện



D) Bảng trạng thái (Moore-type)

Inputs			Outputs		
Q1	Q0	a	I1	I0	x
0	0	0	0	0	0
0	0	1	0	1	
0	1	0	0	1	0
0	1	1	1	0	
1	0	0	1	0	0
1	0	1	1	1	
1	1	0	1	1	1
1	1	1	0	0	

- Cho mô hình thực hiện này
 - Vấn đề thiết kế mạch tuần tự trở thành thiết kế mạch tổ hợp

Thiết kế mạch tuần tự (cont.)

E) Phương trình đầu ra tối thiểu

I1

	Q1Q0			
a	00	01	11	10
0	0	0	1	1
1	0	1	0	1

$$I1 = Q1'Q0a + Q1a' + Q1Q0'$$

I0

	Q1Q0			
a	00	01	11	10
0	0	1	1	0
1	1	0	0	1

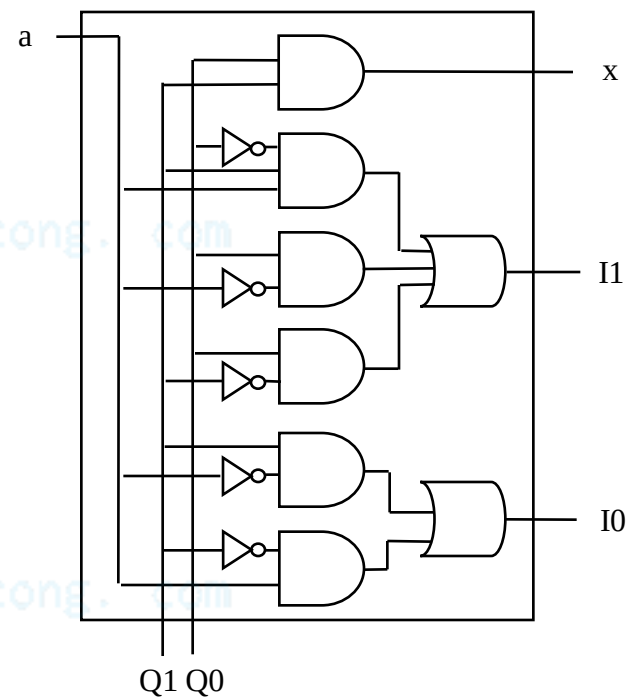
$$I0 = Q0a' + Q0'a$$

x

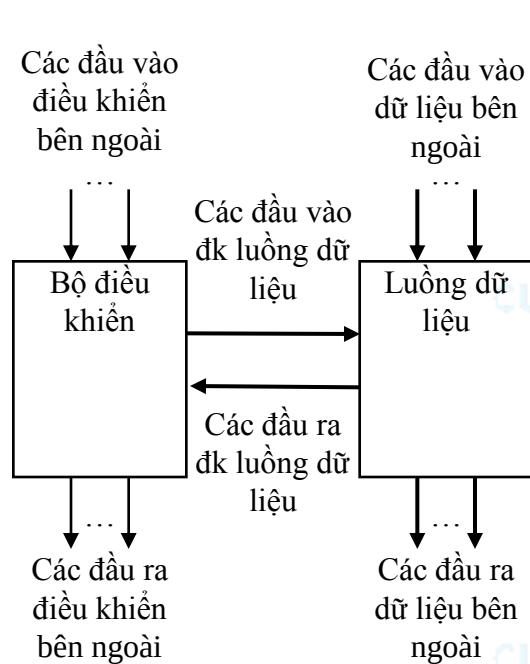
	Q1Q0			
a	00	01	11	10
0	0	0	1	0
1	0	0	1	0

$$x = Q1Q0$$

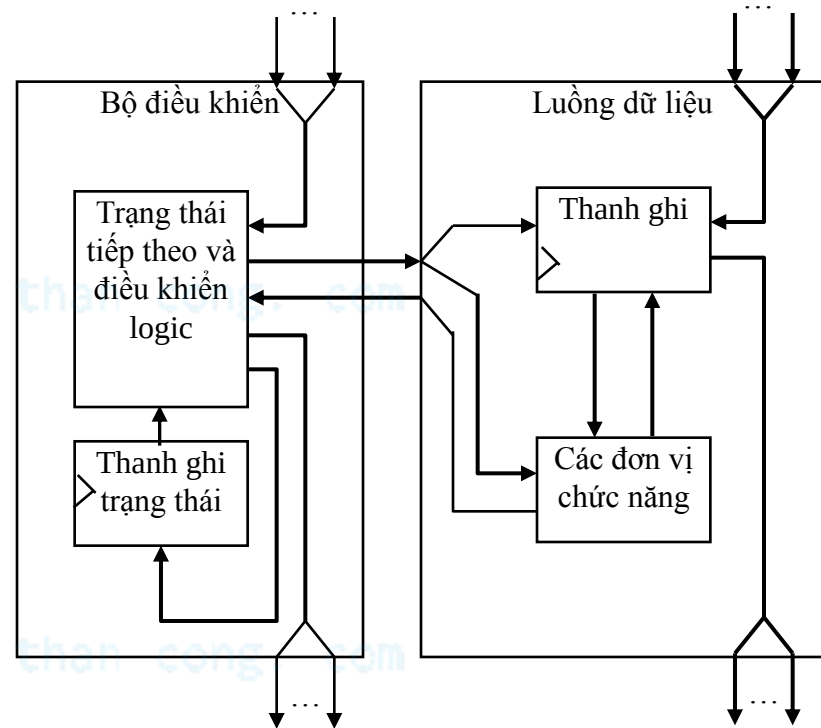
F) Mạch tổ hợp



Mô hình cơ bản của bộ xử lý chức năng đơn chuyên biệt



Bộ điều khiển và luồng dữ liệu



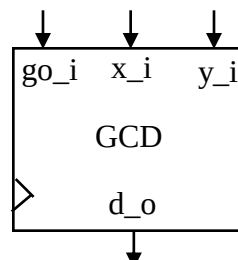
Nhìn bên trong bộ điều khiển và luồng dữ liệu

Ví dụ bộ xử lý chức năng đơn chuyên biệt

Tìm ước số chung lớn nhất

- Đầu tiên tạo ra thật toán
- Biến đổi thuật toán thành giản đồ trạng thái
 - Thường gọi là FSMD (finite-state machine with datapath)
 - Có thể sử dụng biểu tượng để thực hiện việc chuyển đổi

(a) Sơ đồ khối

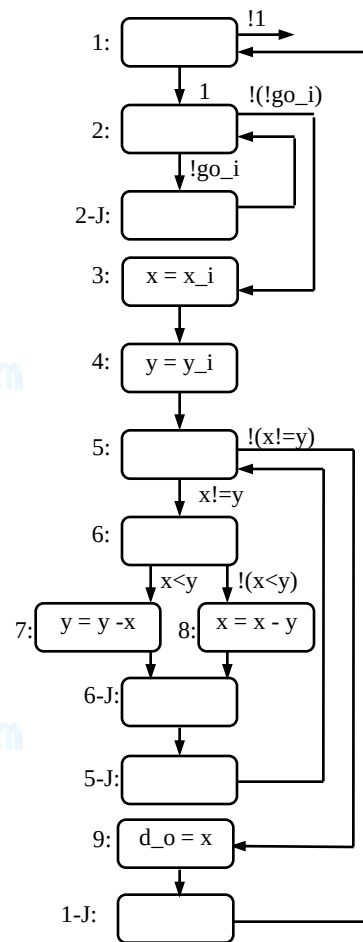


(b) Chức năng yêu cầu

```

0: int x, y;
1: while (1) {
2:   while (!go_i);
3:   x = x_i;
4:   y = y_i;
5:   while (x != y) {
6:     if (x < y)
7:       y = y - x;
8:     else
9:       x = x - y;
10:  }
11:  d_o = x;
12: }
    
```

(c) Giản đồ trạng thái

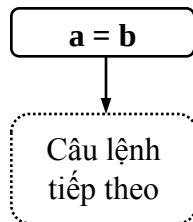


Biểu tượng giản đồ trạng thái

Câu lệnh gán

a = b

Câu lệnh tiếp
theo

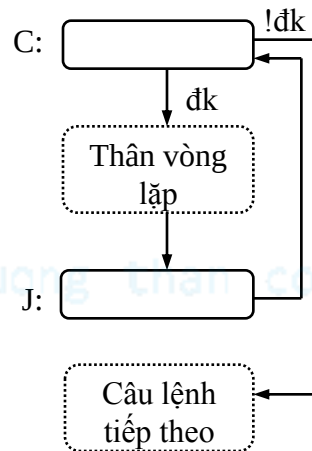


Câu lệnh lặp

while (đk)

```
{  
    thân vòng lặp  
}
```

Câu lệnh tiếp theo



Câu lệnh rẽ nhánh

if (c1)

c1 stmts

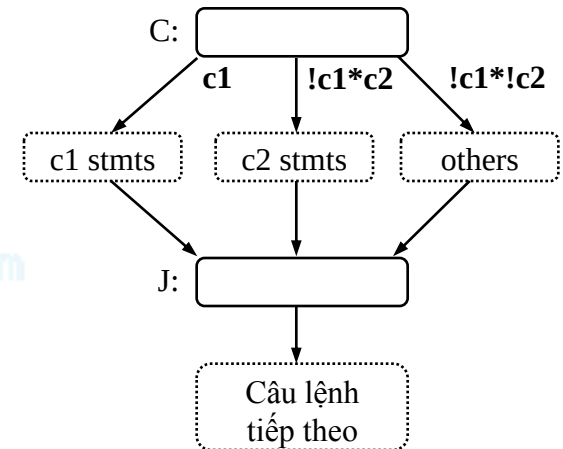
else if c2

c2 stmts

else

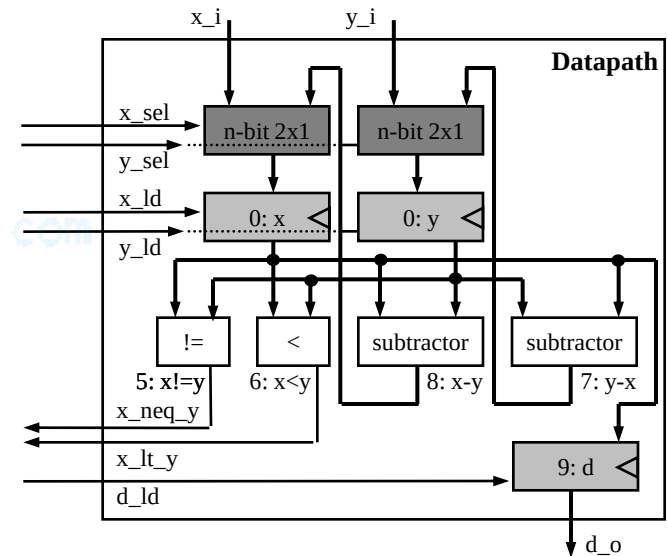
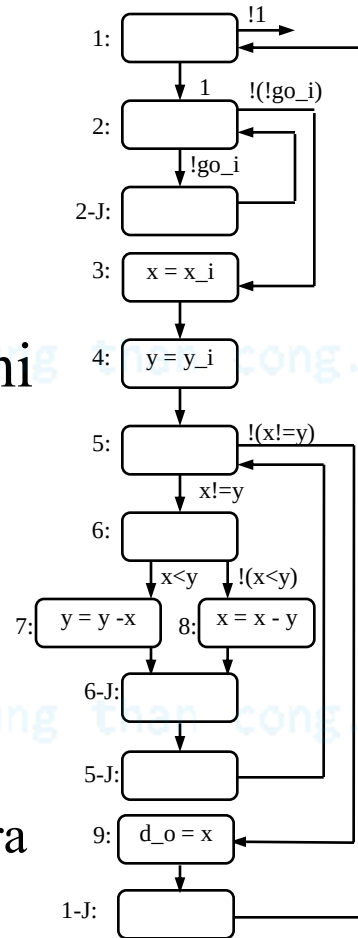
other stmts

Câu lệnh tiếp theo

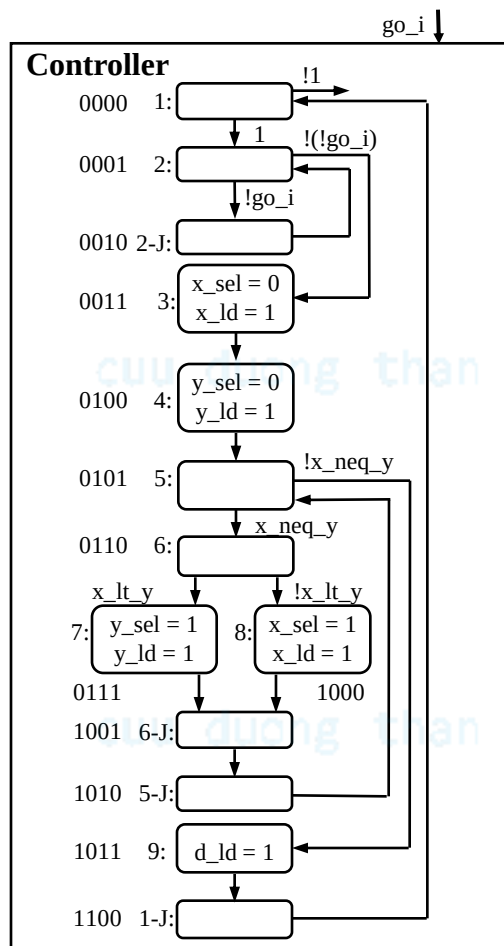
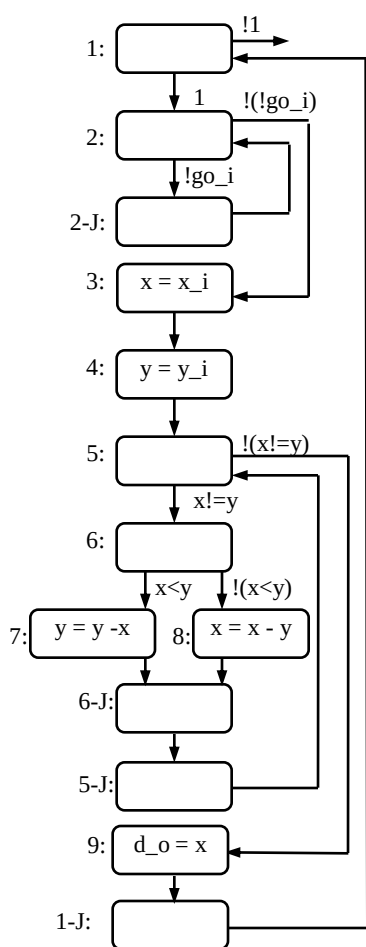


Tạo ra tuyến dữ liệu

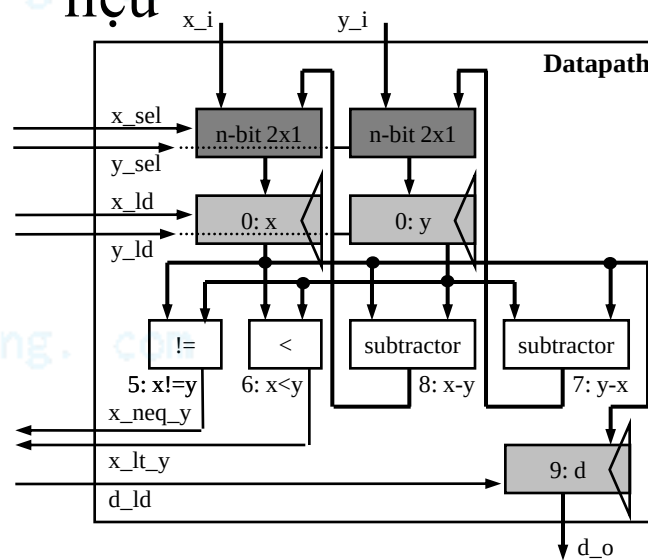
- Tạo ra một thanh ghi cho biến được khai báo
- Tạo ra một hàm cho mỗi thuật toán tính toán
- Kết nối các cổng, thanh ghi và hàm
 - Dựa trên việc đọc và ghi
- Tạo ra bộ nhận dạng đồng nhất
 - Đối với mỗi luồng dữ liệu, điều khiển đầu vào và đầu ra



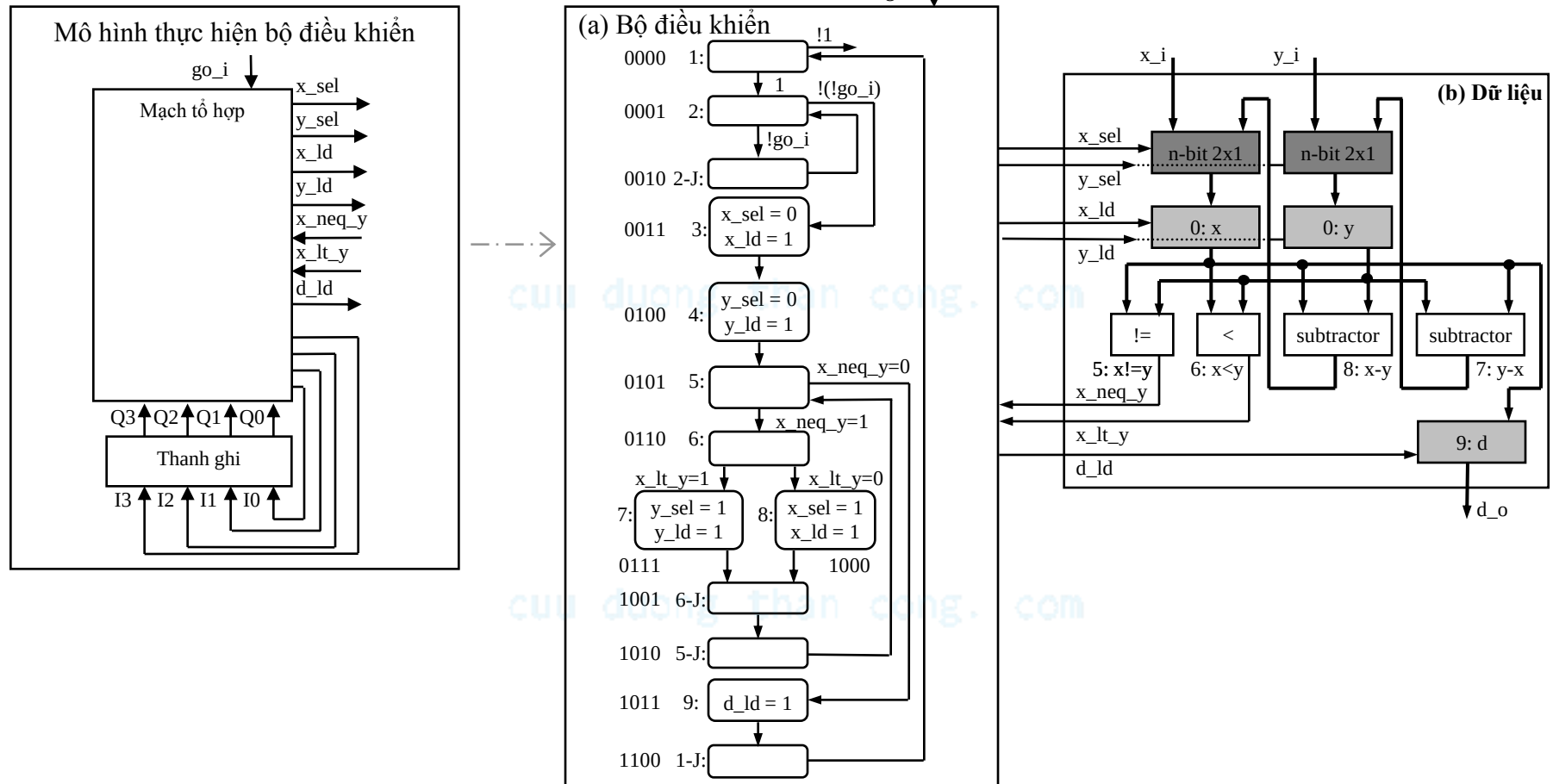
Tạo ra biểu đồ trạng thái (FSM) cho bộ điều khiển



- Có cùng cấu trúc như FSMD
- Thay thế các phép tính phức tạp bằng luồng dữ liệu



Tách thành bộ điều khiển và luồng dữ liệu

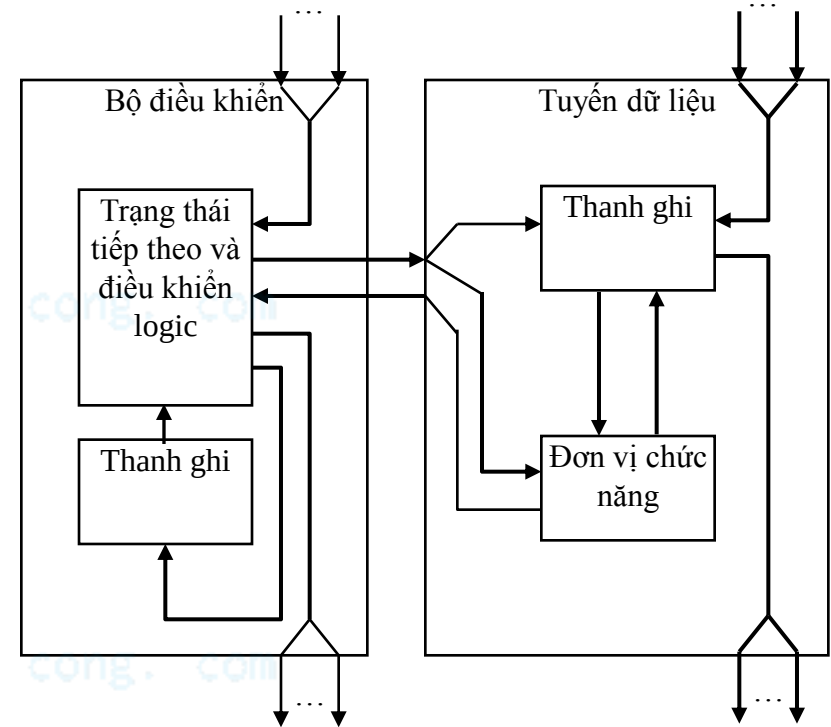


Bảng trạng thái điều khiển cho ví dụ Tìm ước số chung lớn nhất (GCD)

Inputs							Outputs								
Q3	Q2	Q1	Q0	x_neq_y	x_lt_y	go_i	I3	I2	I1	I0	x_sel	y_sel	x_ld	y_ld	d_ld
0	0	0	0	*	*	*	0	0	0	1	X	X	0	0	0
0	0	0	1	*	*	0	0	0	1	0	X	X	0	0	0
0	0	0	1	*	*	1	0	0	1	1	X	X	0	0	0
0	0	1	0	*	*	*	0	0	0	1	X	X	0	0	0
0	0	1	1	*	*	*	0	1	0	0	0	X	1	0	0
0	1	0	0	*	*	*	0	1	0	1	X	0	0	1	0
0	1	0	1	0	*	*	1	0	1	1	X	X	0	0	0
0	1	0	1	1	*	*	0	1	1	0	X	X	0	0	0
0	1	1	0	*	0	*	1	0	0	0	X	X	0	0	0
0	1	1	0	*	1	*	0	1	1	1	X	X	0	0	0
0	1	1	1	*	*	*	1	0	0	1	X	1	0	1	0
1	0	0	0	*	*	*	1	0	0	1	1	X	1	0	0
1	0	0	1	*	*	*	1	0	1	0	X	X	0	0	0
1	0	1	0	*	*	*	0	1	0	1	X	X	0	0	0
1	0	1	1	*	*	*	1	1	0	0	X	X	0	0	1
1	1	0	0	*	*	*	0	0	0	0	X	X	0	0	0
1	1	0	1	*	*	*	0	0	0	0	X	X	0	0	0
1	1	1	0	*	*	*	0	0	0	0	X	X	0	0	0
1	1	1	1	*	*	*	0	0	0	0	X	X	0	0	0

Hoàn thiện thiết kế bộ xử lý chức năng đơn chuyên biệt GCD

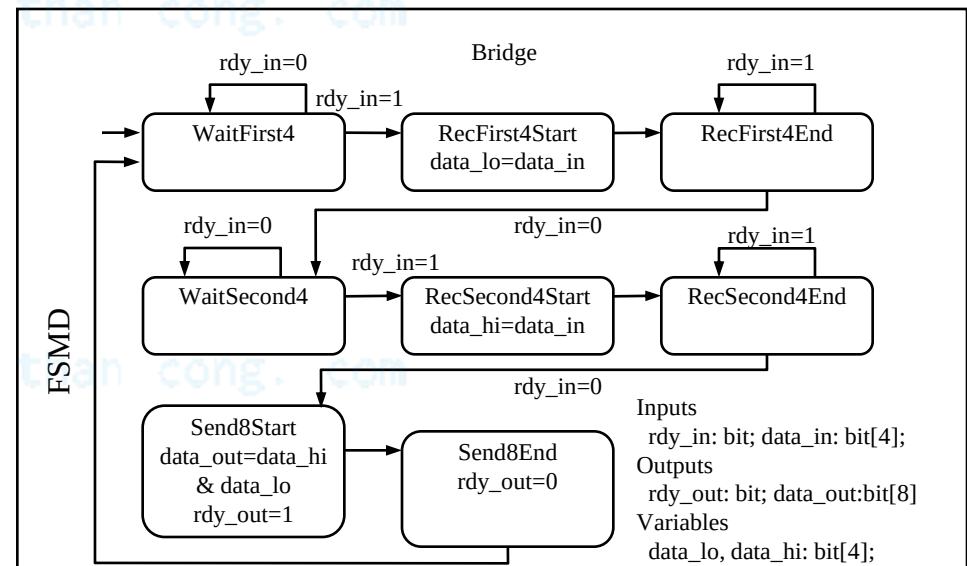
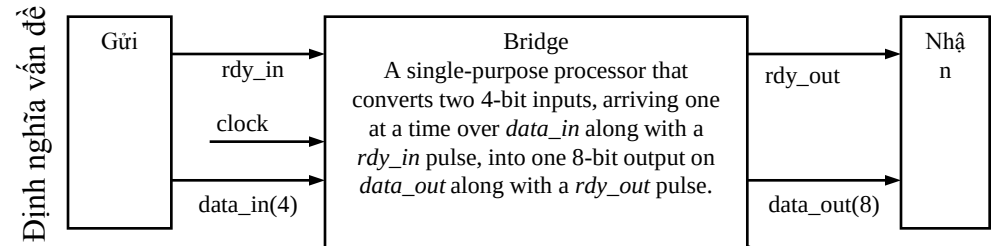
- Chúng ta đã tạo ra tuyến dữ liệu
- Chúng ta đã có bảng trạng thái và điều khiển logic
 - Bộ phận bên trái là thiết kế logic
- Không phải là một thiết kế tối ưu, nhưng chúng ta đã thực hiện các bước thiết kế cơ bản



Bên trong bộ điều khiển và tuyến dữ liệu

Thiết kế bộ xử lý chức năng đơn chuyên biệt mức chuyển đổi thanh ghi (RT)

- Chúng ta thường bắt đầu với một giản đồ trạng thái
 - Khác với thuật toán
 - Việc lặp lại chu trình thường chỉ chú ý đến chức năng
- Ví dụ
 - Một bộ chuyển đổi bus biến đổi một bus 4-bit sang bus 8-bit
 - Bắt đầu với FSMD
 - Thường được biết như mức chuyển đổi thanh ghi (RT)
 - Bài tập: hoàn thành thiết kế



Tối ưu bộ xử lý chức năng đơn

- Tối ưu là nhiệm vụ tạo ra các giá trị của thông số thiết kế phù hợp nhất có thể
- Các cơ hội tối ưu
 - Chương trình nguồn
 - FSMD
 - Tuyến dữ liệu
 - FSM

Tối ưu chương trình nguồn

- Phân tích thuộc tính của chương trình và tìm ra các khu vực có thể cải tiến được
 - Số phép tính
 - Kích thước biến
 - Độ phức tạp về thời gian và không gian
 - Các toán hạng sử dụng
 - Ví dụ phép nhân và phép chia rất phức tạp về độ tính toán

Tối ưu chương trình nguồn (cont')

original program

```
0: int x, y;
1: while (1) {
2:   while (!go_i);
3:   x = x_i;
4:   y = y_i;
5:   while (x != y) {
6:     if (x < y)
7:       y = y - x;
8:     else
9:       x = x - y;
10:  }
11:  d_o = x;
12: }
```

replace the subtraction
operation(s) with modulo
operation in order to speed
up program

optimized program

```
0: int x, y, r;
1: while (1) {
2:   while (!go_i);
3:   // x must be the larger number
4:   if (x_i >= y_i) {
5:     x=x_i;
6:     y=y_i;
7:   }
8:   else {
9:     x=y_i;
10:    y=x_i;
11:  }
12:  while (y != 0) {
13:    r = x % y;
14:    x = y;
15:    y = r;
16:  }
17:  d_o = x;
18: }
```

GCD(42, 8) - 9 iterations to complete the loop

x and y values evaluated as follows : (42, 8), (43, 8),
(26,8), (18,8), (10, 8), (2,8), (2,6), (2,4), (2,2).

GCD(42,8) - 3 iterations to complete the loop

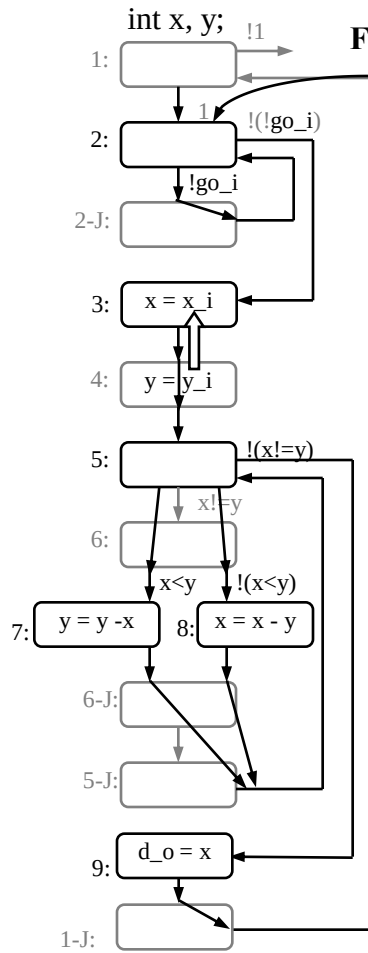
x and y values evaluated as follows: (42, 8), (8,2),
(2,0)

Tối ưu FSMD

- Các trường hợp có thể cải tiến
 - Ghép trạng thái
 - Trạng thái không có biến đổi (chuyển trạng thái) có thể bỏ đi
 - Trạng thái với các hoạt động độc lập có thể ghép
 - Tách trạng thái
 - Các trạng thái yêu cầu các toán hạng phức tạp ($a*b*c*d$) có thể tách thành các trạng thái nhỏ hơn để giảm kích thước phần cứng
 - Lập lịch

Tối ưu FSMD (cont.)

FSMD ban đầu



Loại bỏ trạng thái 1 – quá trình chuyển có giá trị không đổi

Ghép trạng thái 2 và trạng thái 2J – không có hoạt động lặp trong nó

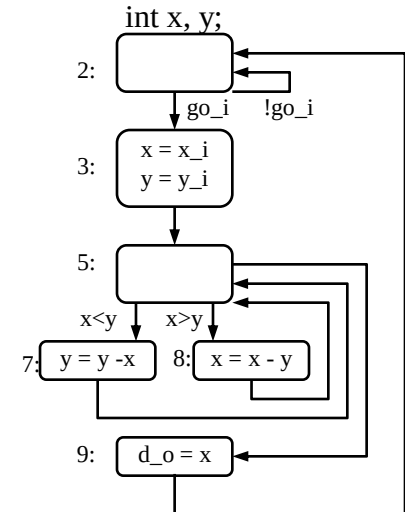
Ghép trạng thái 3 và 4 – hoạt động gán độc lập với các hoạt động khác

Ghép trạng thái 5 và 6 – việc chuyển sang trạng thái 6 có thể thực hiện ở trạng thái 5

Loại bỏ trạng thái 5J và 6J – việc chuyển từ mỗi trạng thái có thể thực hiện từ trạng thái 7 hoặc 8, tương ứng

Loại bỏ trạng thái 1-J – chuyển từ trạng thái 1-J có thể thực hiện trực tiếp từ trạng thái 9

FSMD tối ưu



Tối ưu tuyến dữ liệu

- Chia sẻ các bộ phận chức năng
 - Ghép từng chức năng, như thực hiện trong phần trước, là không cần thiết
 - Nếu có cùng hoạt động xảy ra trong các trạng thái khác nhau, chúng có thể chia sẻ một đơn vị chức năng đơn
- Các bộ phận đa chức năng
 - ALUs hỗ trợ một loạt các hoạt động, nó có thể chia sẻ các hoạt động trong các trạng thái khác nhau

Tối ưu FSM

- Mã hóa trạng thái
 - Là nhiệm vụ gán một mẫu bit duy nhất tới mỗi trạng thái trong một FSM
 - Kích thước của thanh ghi trạng thái và mạch tổ hợp biến đổi
 - Có thể được xem xét như một vấn đề sắp xếp
- Tối ưu trạng thái
 - Là nhiệm vụ ghép các trạng thái tương đồng thành một trạng thái duy nhất
 - Trạng thái tương đồng nếu kết hợp tất cả các đầu vào, hai trạng thái tạo ra cùng đầu ra và cùng chuyển tới trạng thái kế tiếp

Tóm tắt

- Bộ xử lý chức năng đơn chuyên biệt
 - Kỹ thuật thiết kế trực tiếp
 - Có thể được xây dựng để thực hiện các thuật toán
 - Thường bắt đầu với FSMD
 - Công cụ CAD có thể trợ giúp đắc lực

Reference

- This document is used for educational purposes only
- This document contains material from other sources (incl. textbook, lecture notes, application notes, ...)
- Our thanks go to the contributors, references

CHƯƠNG 2: CẤU TRÚC PHẦN CỨNG HỆ THỐNG NHÚNG

Bài 3: Bộ xử lý chức năng đơn tiêu chuẩn - Thiết bị ngoại vi

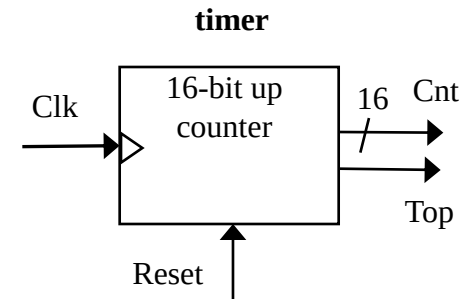
cuu duong than cong. com

Giới thiệu

- Bộ xử lý chức năng đơn
 - Thực hiện các nhiệm vụ tính toán nhất định
 - Bộ xử lý chức năng đơn chuyên biệt
 - Thiết kế cho một nhiệm vụ duy nhất
 - *Bộ xử lý chức năng đơn “tiêu chuẩn”*
 - “Off-the-shelf” -- Thiết kế trước cho một nhiệm vụ chung
 - VD: ngoại vi
 - Truyền thông nối tiếp
 - ADC

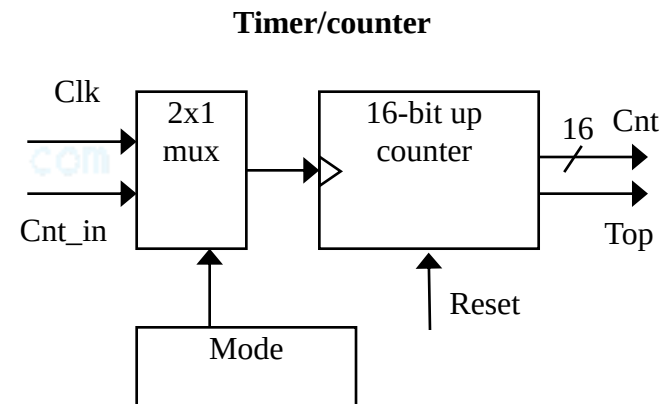
Timers, counters, watchdog timers

- Bộ định thời - Timer: dùng đo khoảng thời gian
 - Để phát ra các sự kiện đầu ra định thời
 - VD: giữ cho đèn xanh sáng 10 s
 - Để đo các sự kiện đầu vào
 - VD: đo tốc độ xe
- Dựa trên việc đếm xung đồng hồ
 - VD: giả sử chu kỳ Clk là 10 ns
 - Và chúng ta đếm được 20,000 Clk
 - Như vậy, 200 microsec đã trôi qua
 - Bộ đếm 16-bit sẽ đếm tới $65,535 \times 10 \text{ ns} = 655.35 \text{ microsec.}$, độ phân giải = 10 ns
 - Top: biểu thị đạt đến số đếm cực đại, quay lại



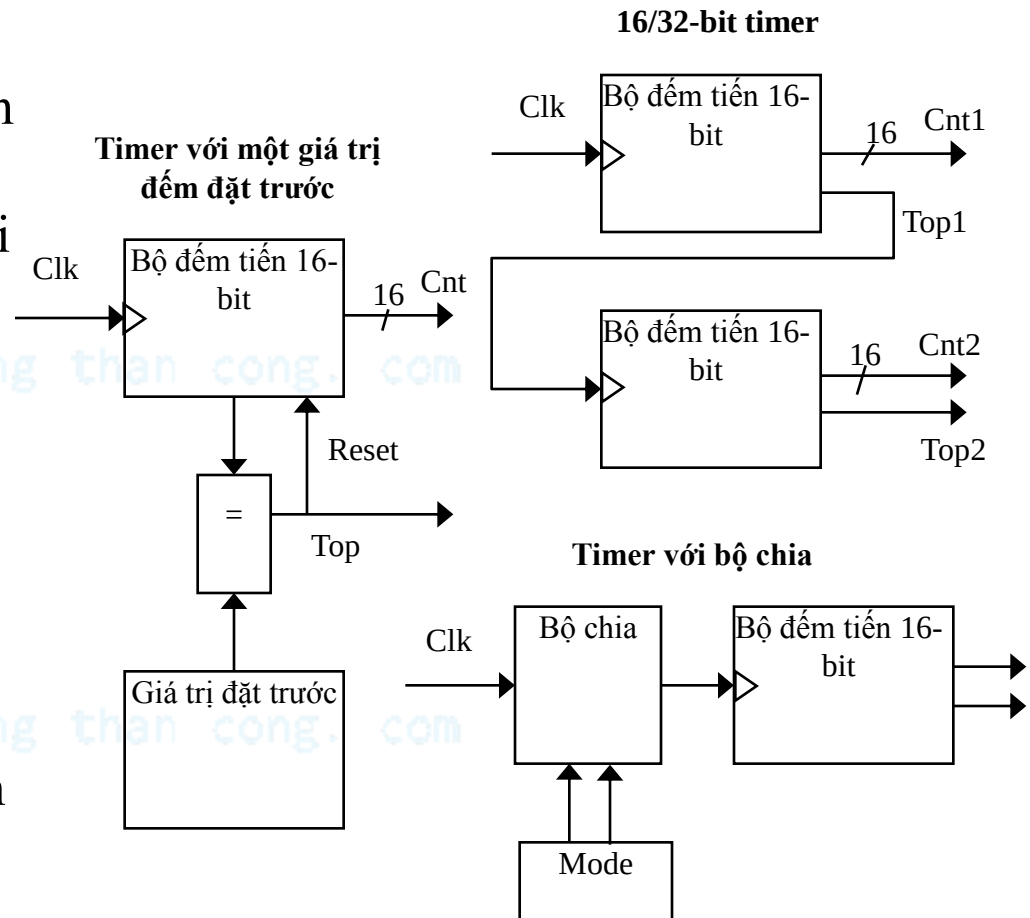
Bộ đếm - Counters

- Counter: giống một timer, nhưng đếm xung trên một tín hiệu đầu vào thay vì xung clk
 - VD: đếm số ô tô chạy qua một cảm biến
 - Đôi khi ta có thể cấu hình thiết bị như một timer hoặc counter

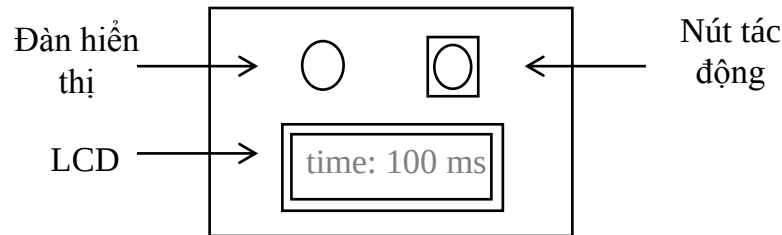


Cấu trúc timer khác

- Timer theo khoảng
 - Biểu thị khi khoảng thời gian yêu cầu trôi qua
 - Chúng ta đặt giá trị đếm cuối cùng cho giá trị yêu cầu
 - $Số\ xung\ clk = khoảng\ thời\ gian\ yêu\ cầu / chu\ kỳ\ đồng\ hồ$
- Bộ đếm ghép
- Bộ chia
 - Chia xung đồng hồ
 - Tăng khoảng thời gian, giảm độ phân giải



Ví dụ: Timer tác động



- Đo khoảng thời gian giữa trạng thái đèn sáng và người dùng bấm nút
 - Timer 16-bit, chu kỳ clk là 83.33 ns, counter tăng sau mỗi 6 chu kỳ đồng hồ
 - Độ phân giải = $6 \times 83.33 = 0.5$ microsec.
 - Khoảng tg = 65535×0.5 microsec = 32.77 millisec
 - Muốn chương trình đếm millisec., vì vậy khởi đầu bộ đếm $65535 - 1000/0.5 = 63535$

```
/* main.c */

#define MS_INIT 63535
void main(void){
    int count_milliseconds = 0;

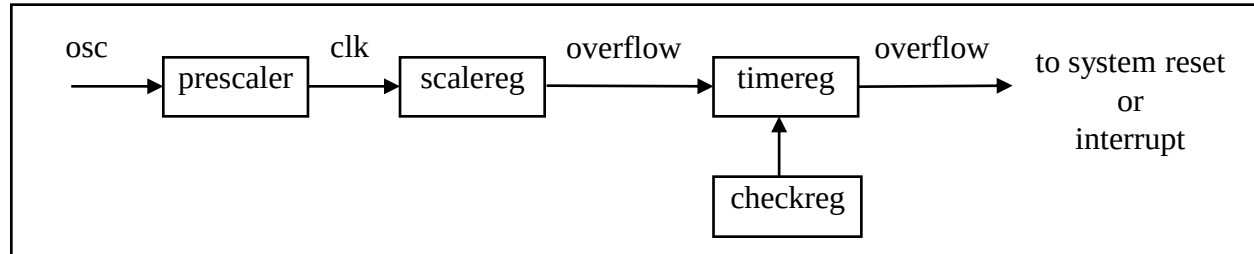
    configure timer mode
    set Cnt to MS_INIT

    wait a random amount of time
    turn on indicator light
    start timer

    while (user has not pushed reaction button){
        if(Top) {
            stop timer
            set Cnt to MS_INIT
            start timer
            reset Top
            count_milliseconds++;
        }
    }
    turn light off
    printf("time: %i ms", count_milliseconds);
}
```

Watchdog timer

- Phải reset timer sau mỗi khoảng thời gian X, nếu không timer sẽ phát ra một tín hiệu
- Sử dụng thông thường: xác định lỗi, hoặc tự reset
- Sử dụng khác: timeouts
 - VD: máy ATM
 - 16-bit timer, độ phân giải 2 ms
 - *Giá trị timereg* = $2 * (2^{16} - 1) - X = 131070 - X$
 - Nếu 2 phút, X = 120,000 ms.

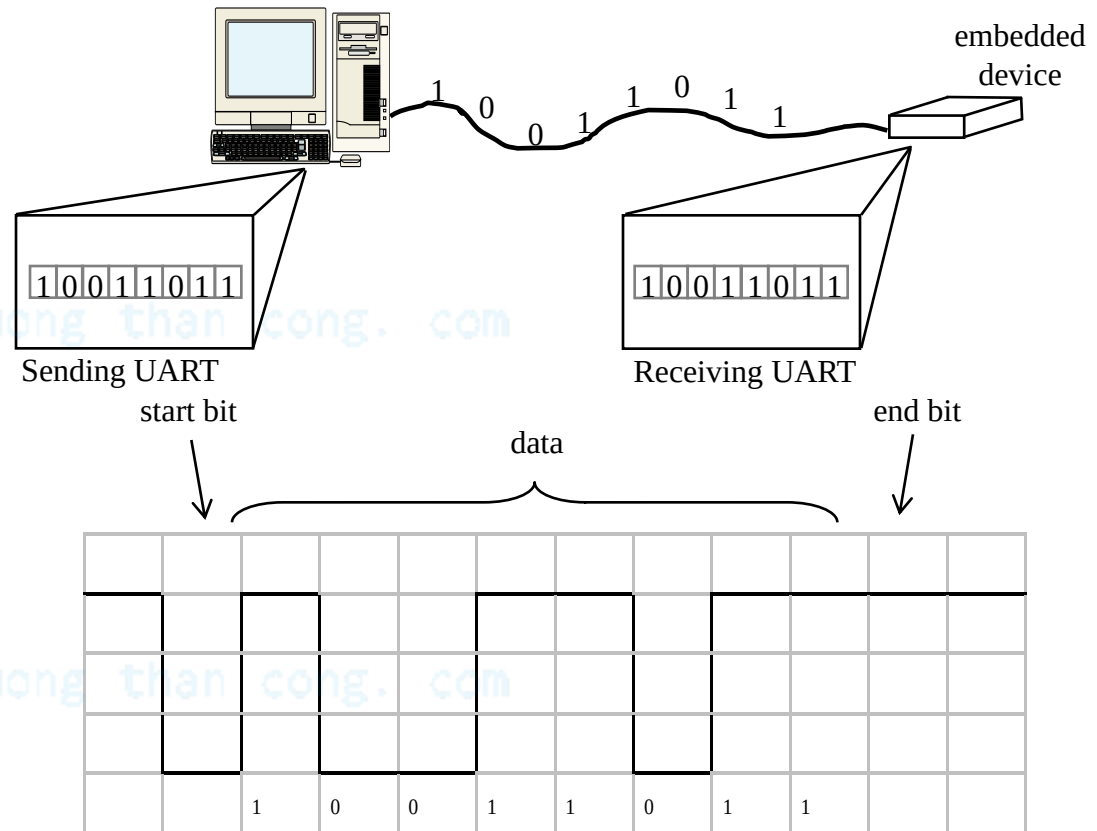


```
/* main.c */  
  
main(){  
    wait until card inserted  
    call watchdog_reset_routine  
  
    while(transaction in progress){  
        if(button pressed){  
            perform corresponding action  
            call watchdog_reset_routine  
        }  
    }  
  
    /* if watchdog_reset_routine not called every  
    < 2 minutes, interrupt_service_routine is  
    called */  
}
```

```
watchdog_reset_routine(){  
    /* checkreg is set so we can load value into  
    timereg. Zero is loaded into scalereg and  
    11070 is loaded into timereg */  
  
    checkreg = 1  
    scalereg = 0  
    timereg = 11070  
}  
  
void interrupt_service_routine(){  
    eject card  
    reset screen  
}
```

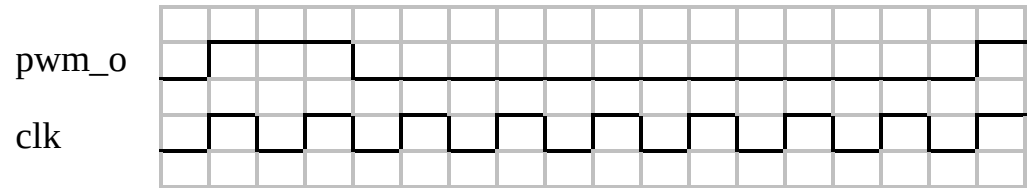

Truyền thông nối tiếp dùng UARTs

- UART: Universal Asynchronous Receiver Transmitter
 - Lấy dữ liệu song song và truyền nối tiếp
 - Nhận dữ liệu nối tiếp và truyền song song
- Chẩn lẻ: Thêm bít cho các kiểm tra đơn giản
- Bit bắt đầu, bit kết thúc
- Baud rate
 - Độ thay đổi tín hiệu trên giây
 - Tốc độ bit thường cao hơn Baud rate

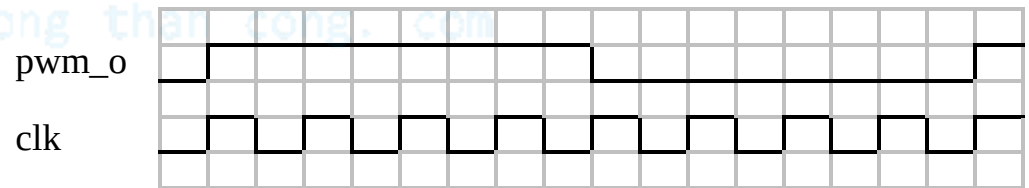


Điều xung PWM

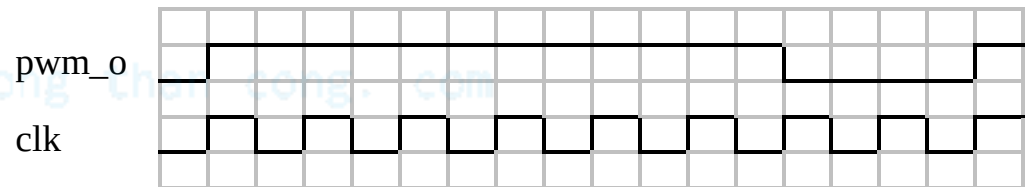
- Phát xung với thời gian cao/thấp nhất định
- Duty cycle: % thời gian cao
 - Xung vuông: 50% duty cycle
- Sử dụng thông thường: điều khiển điện áp trung bình cấp cho thiết bị điện
 - Đơn giản hơn bộ biến đổi DC-DC hoặc ADC
 - Điều khiển tốc độ động cơ DC, đèn
- Sử dụng khác: lệnh được mã hóa, phía thu sử dụng timer để giải mã



25% duty cycle – average pwm_o is 1.25V

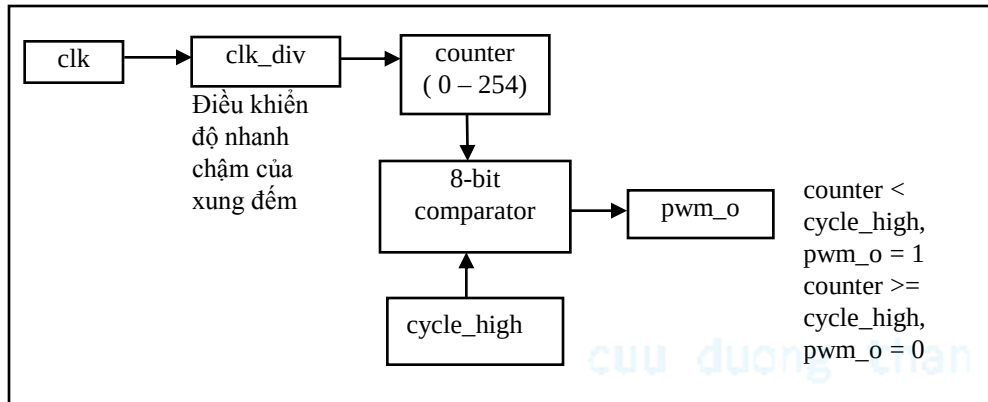


50% duty cycle – average pwm_o is 2.5V.



75% duty cycle – average pwm_o is 3.75V.

Điều khiển động cơ DC với PWM



Cấu trúc bên trong của PWM

Input Voltage	% of Maximum Voltage Applied	RPM of DC Motor
0	0	0
2.5	50	1840
3.75	75	6900
5.0	100	9200

Mối quan hệ giữa điện áp đặt và tốc độ động cơ DC

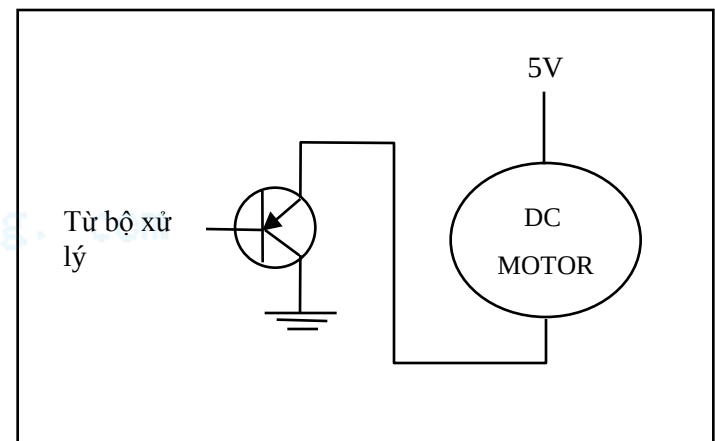
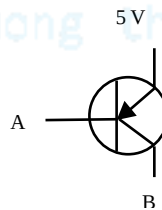
```

void main(void) {

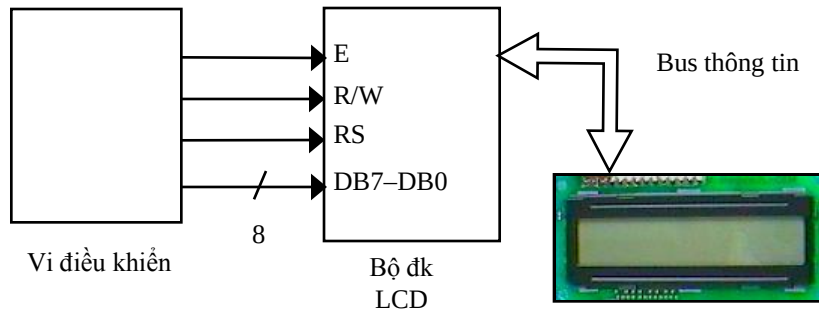
    /* controls period */
    PWMP = 0xff;
    /* controls duty cycle */
    PWM1 = 0x7f;

    while(1) {};
}
    
```

Chỉ với PWM không thể điều khiển động cơ DC, một cách thực hiện được chỉ ra dưới đây sử dụng một transistor MJE3055T.



Bộ điều khiển LCD



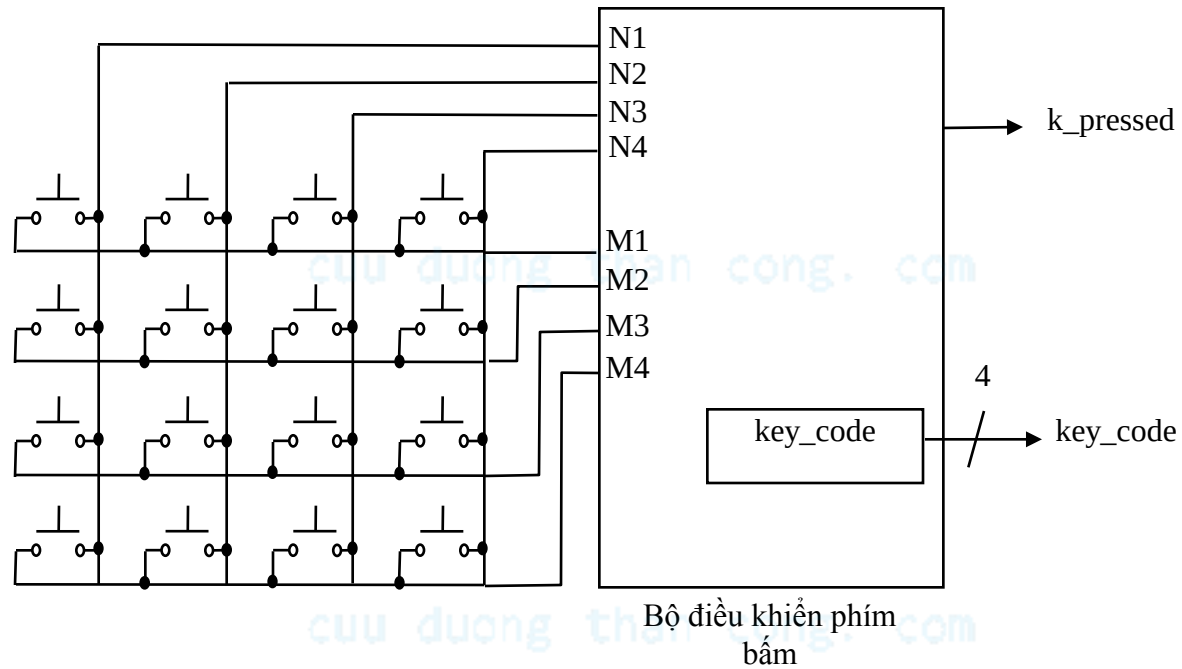
```
void WriteChar(char c){
    RS = 1;                /* indicate data being sent */
    DATA_BUS = c;         /* send data to LCD */
    EnableLCD(45);         /* toggle the LCD with appropriate delay */
}
```

cuuduongthancong.com

CODES	
I/D = 1 cursor moves left	DL = 1 8-bit
I/D = 0 cursor moves right	DL = 0 4-bit
S = 1 with display shift	N = 1 2 rows
S/C = 1 display shift	N = 0 1 row
S/C = 0 cursor movement	F = 1 5x10 dots
R/L = 1 shift to right	F = 0 5x7 dots
R/L = 0 shift to left	

RS	R/W	DB ₇	DB ₆	DB ₅	DB ₄	DB ₃	DB ₂	DB ₁	DB ₀	Description
0	0	0	0	0	0	0	0	0	1	Clears all display, return cursor home
0	0	0	0	0	0	0	0	1	*	Returns cursor home
0	0	0	0	0	0	0	1	I/D	S	Sets cursor move direction and/or specifies not to shift display
0	0	0	0	0	0	1	D	C	B	ON/OFF of all display(D), cursor ON/OFF (C), and blink position (B)
0	0	0	0	0	1	S/C	R/L	*	*	Move cursor and shifts display
0	0	0	0	1	DL	N	F	*	*	Sets interface data length, number of display lines, and character font
1	0	WRITE DATA								Writes Data

Bộ điều khiển phím bấm

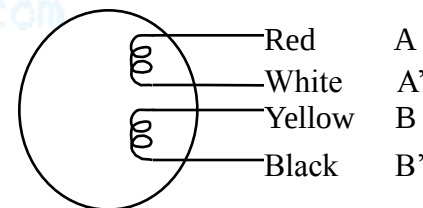
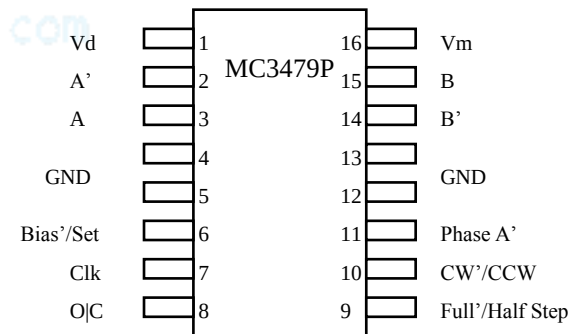


N=4, M=4

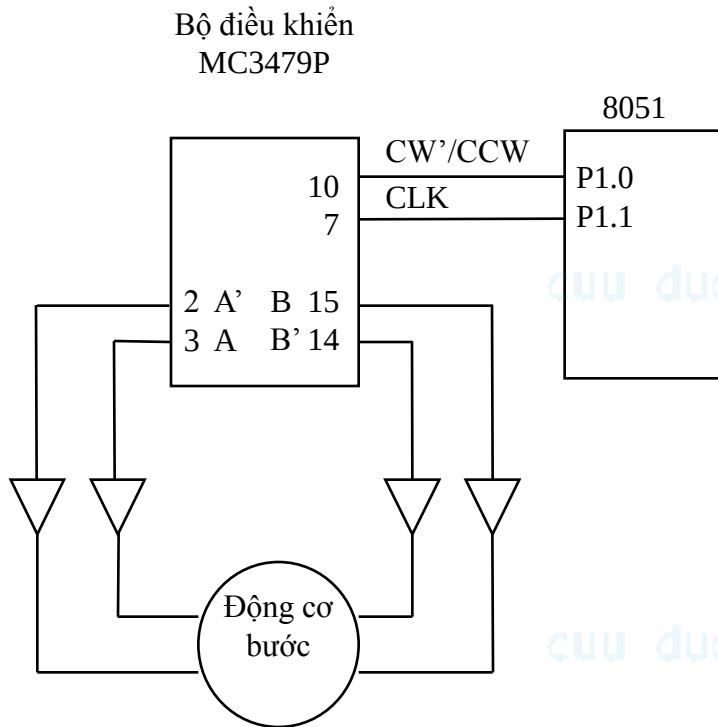
Bộ điều khiển động cơ bước

- Động cơ bước: quay một góc cố định khi cung cấp một tín hiệu “bước”
 - Ngược lại, động cơ DC chỉ quay khi có công suất đặt vào
- Hoạt động quay đạt được bằng cách cung cấp một tuần tự điện áp cho các cuộn dây
- Bộ điều khiển sẽ thực hiện chức năng này

Sequence	A	B	A'	B'
1	+	+	-	-
2	-	+	+	-
3	-	-	+	+
4	+	-	-	+
5	+	+	-	-



Động cơ bước với bộ điều khiển (driver)



```
/* main.c */
```

```
sbit clk=P1^1;
```

```
sbit cw=P1^0;
```

```
void delay(void){  
    int i, j;  
    for (i=0; i<1000; i++)  
        for (j=0; j<50; j++)  
            i = i + 0;  
}
```

```
void main(void){
```

```
    /*turn the motor forward */
```

```
    cw=0;          /* set direction */
```

```
    clk=0;         /* pulse clock */
```

```
    delay();
```

```
    clk=1;
```

```
    /*turn the motor backwards */
```

```
    cw=1;          /* set direction */
```

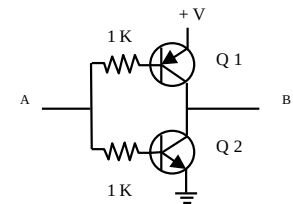
```
    clk=0;         /* pulse clock */
```

```
    delay();
```

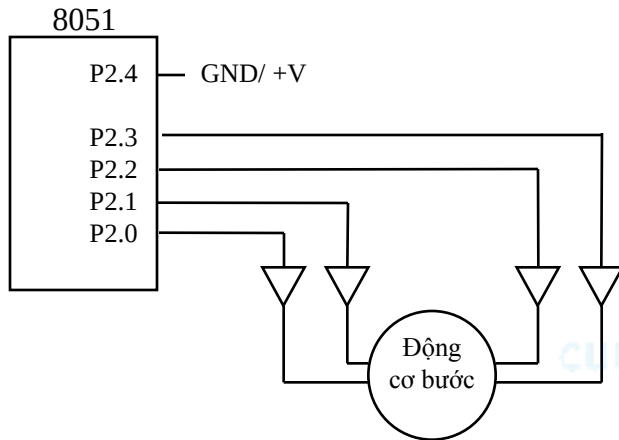
```
    clk=1;
```

```
}
```

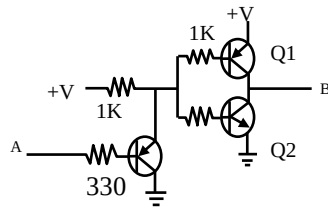
Các chân đầu ra của bộ điều khiển động cơ bước không cung cấp đủ dòng để điều khiển động cơ. Để khuếch đại dòng, cần có một bộ đệm. Một cách thực hiện được chỉ ra trên hình bên phải. Q1 là một transistor NPN MJE3055T và Q2 là một transistor PNP MJE2955T. A kết nối tới VDK 8051 và B kết nối tới động cơ bước.



Động cơ bước không bộ điều khiển (driver)



Một cách để thực hiện bộ đếm như chỉ ra bên dưới. Bản thân 8051 không thể điều khiển động cơ bước, vì vậy một vài transistors được thêm vào để tăng dòng cho động cơ bước. Q1 là MJE3055T NPN Q3 là MJE2955T PNP. A kết nối với 8051 và B kết nối với động cơ bước.



```
/*main.c*/
sbit notA=P2^0;
sbit isA=P2^1;
sbit notB=P2^2;
sbit isB=P2^3;
sbit dir=P2^4;

void delay(){
    int a, b;
    for(a=0; a<5000; a++){
        for(b=0; b<10000; b++){
            a=a+0;
        }
    }
}
```

```
void move(int dir, int steps) {
    int y, z;
    /* clockwise movement */
    if(dir == 1){
        for(y=0; y<=steps; y++){
            for(z=0; z<=19; z++){
                isA=lookup[z];
                isB=lookup[z+1];
                notA=lookup[z+2];
                notB=lookup[z+3];
                delay();
            }
        }
    }
}
```

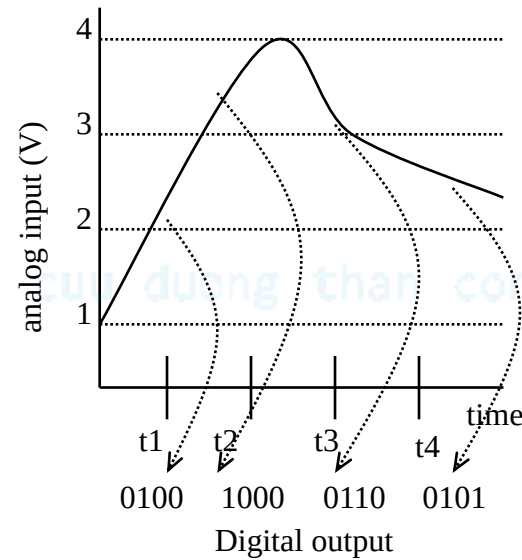
```
/* counter clockwise movement */
if(dir==0){
    for(y=0; y<=step; y++){
        for(z=19; z>=0; z - 4){
            isA=lookup[z];
            isB=lookup[z-1];
            notA=lookup[z - 2];
            notB=lookup[z-3];
            delay( );
        }
    }
}

void main() {
    int z;
    int lookup[20] = {
        1, 1, 0, 0,
        0, 1, 1, 0,
        0, 0, 1, 1,
        1, 0, 0, 1,
        1, 1, 0, 0 };
    while(1){
        /*move forward, 15 degrees (2 steps) */
        move(1, 2);
        /* move backwards, 7.5 degrees (1step)*/
        move(0, 1);
    }
}
```

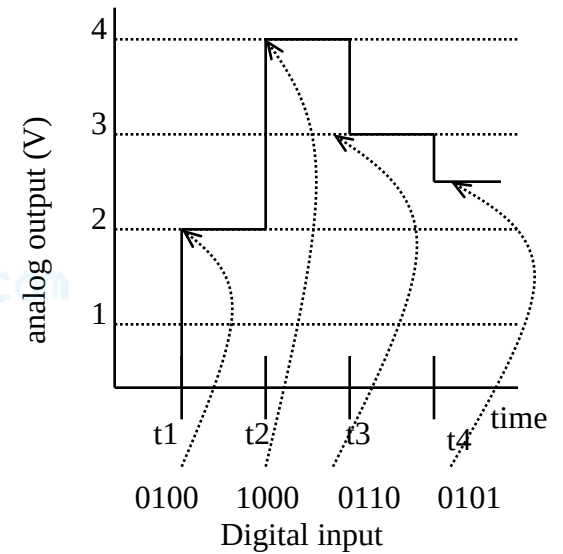

ADC

$V_{\max} = 7.5V$	1111
7.0V	1110
6.5V	1101
6.0V	1100
5.5V	1011
5.0V	1010
4.5V	1001
4.0V	1000
3.5V	0111
3.0V	0110
2.5V	0101
2.0V	0100
1.5V	0011
1.0V	0010
0.5V	0001
0V	0000

Tỷ lệ



Tương tự sang số



Số sang tương tự

DAC dùng xấp xỉ nối tiếp

Cho một tín hiệu tương tự đầu vào điện áp từ 0 đến 15 volts, và một bộ mã hóa số 8-bit, tính toán mã cho giá trị 5 volts.

$$5/15 = d/(2^8-1)$$

$$d = 85$$

Encoding: 01010101

Phương pháp xấp xỉ nối tiếp

$$\frac{1}{2}(V_{\max} - V_{\min}) = 7.5 \text{ volts}$$
$$V_{\max} = 7.5 \text{ volts.}$$

0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---

$$\frac{1}{2}(5.63 + 4.69) = 5.16 \text{ volts}$$
$$V_{\max} = 5.16 \text{ volts.}$$

0	1	0	1	0	0	0	0
---	---	---	---	---	---	---	---

$$\frac{1}{2}(7.5 + 0) = 3.75 \text{ volts}$$
$$V_{\min} = 3.75 \text{ volts.}$$

0	1	0	0	0	0	0	0
---	---	---	---	---	---	---	---

$$\frac{1}{2}(5.16 + 4.69) = 4.93 \text{ volts}$$
$$V_{\min} = 4.93 \text{ volts.}$$

0	1	0	1	0	1	0	0
---	---	---	---	---	---	---	---

$$\frac{1}{2}(7.5 + 3.75) = 5.63 \text{ volts}$$
$$V_{\max} = 5.63 \text{ volts}$$

0	1	0	0	0	0	0	0
---	---	---	---	---	---	---	---

$$\frac{1}{2}(5.16 + 4.93) = 5.05 \text{ volts}$$
$$V_{\max} = 5.05 \text{ volts.}$$

0	1	0	1	0	1	0	0
---	---	---	---	---	---	---	---

$$\frac{1}{2}(5.63 + 3.75) = 4.69 \text{ volts}$$
$$V_{\min} = 4.69 \text{ volts.}$$

0	1	0	1	0	0	0	0
---	---	---	---	---	---	---	---

$$\frac{1}{2}(5.05 + 4.93) = 4.99 \text{ volts}$$

0	1	0	1	0	1	0	1
---	---	---	---	---	---	---	---

Reference

- This document is used for educational purposes only
- This document contains material from other sources (incl. textbook, lecture notes, application notes, ...)
- Our thanks go to the contributors, references

CHƯƠNG 2: CẤU TRÚC PHẦN CỨNG HỆ THỐNG NHÚNG

Bài 4: Bộ nhớ

cuu duong than cong. com

cuu duong than cong. com

Tổng quan

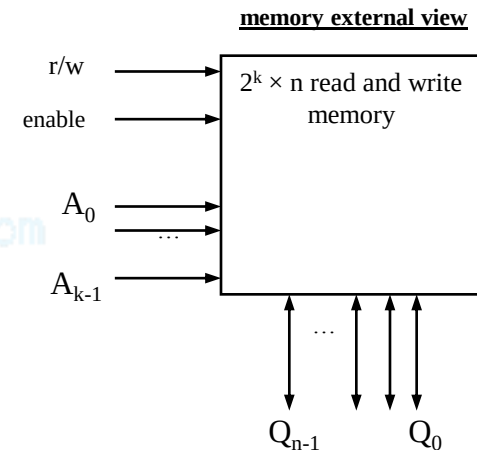
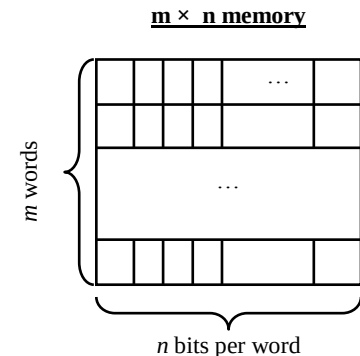
- Khả năng ghi dữ liệu và chất lượng lưu trữ của bộ nhớ
- Các kiểu bộ nhớ chung
- Ghép bộ nhớ
- Phân cấp bộ nhớ và Cache
- RAM cải tiến

Giới thiệu

- Chức năng của các hệ nhúng
 - Xử lý
 - Bộ xử lý
 - Biến đổi dữ liệu
 - Lưu trữ
 - Bộ nhớ
 - Khôi phục dữ liệu
 - Truyền thông
 - Bus
 - Chuyển dữ liệu

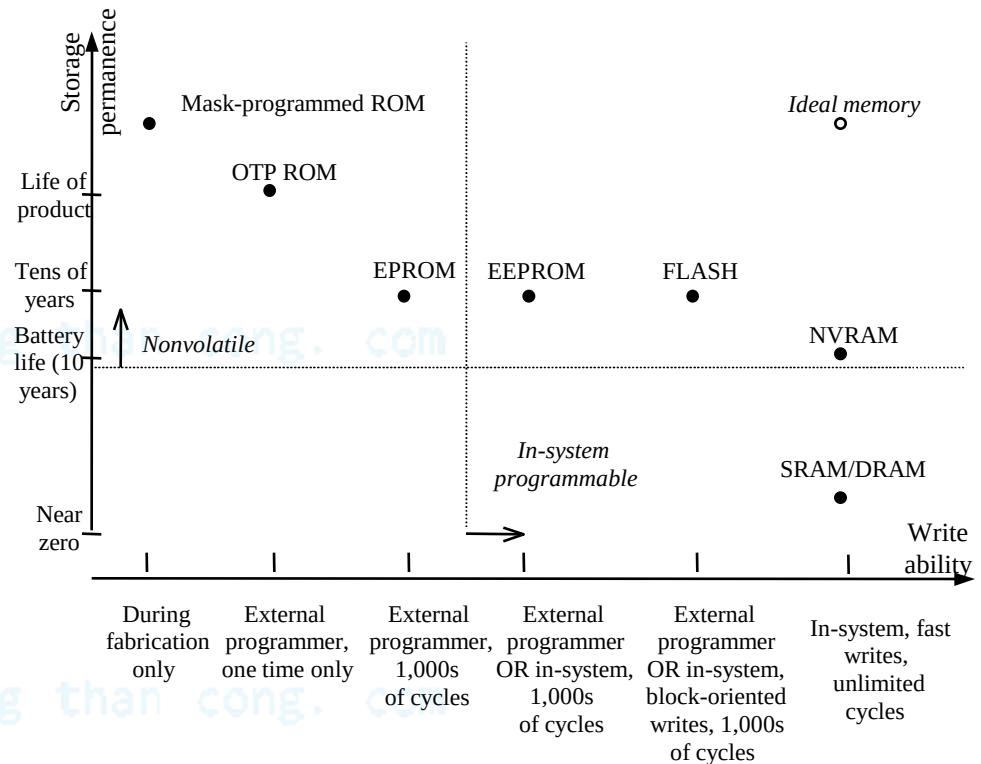
Bộ nhớ: khái niệm cơ bản

- Lưu trữ số lượng bit lớn
 - $m \times n$: m từ mỗi từ n bit
 - $k = \log_2(m)$ tín hiệu địa chỉ đầu vào
 - Hoặc $m = 2^k$ từ
 - VD: bộ nhớ 4,096 x 8:
 - 32,768 bits
 - 12 tín hiệu địa chỉ
 - 8 tín hiệu I/O
- Truy cập bộ nhớ
 - r/w: lựa chọn đọc hoặc ghi
 - enable: chỉ cho phép khi tích cực
 - multiport: đa truy cập tới nhiều vị trí khác nhau đồng thời



Khả năng ghi/chất lượng lưu trữ

- Phân biệt ROM/RAM truyền thống
 - ROM
 - Chỉ đọc, bit được lưu trữ không cần nguồn
 - RAM
 - Đọc và ghi, mất dữ liệu nếu không có nguồn
- Phân biệt truyền thống đã thay đổi
 - ROMs cải tiến có thể ghi
 - e.g., EEPROM
 - RAMs có thể lưu trữ không cần nguồn
 - e.g., NVRAM
- Khả năng ghi
 - Tốc độ để một bộ nhớ được ghi
- Chất lượng lưu trữ
 - Khả năng mà bộ nhớ lưu trữ dữ liệu sau khi nó được ghi



Khả năng ghi và chất lượng lưu trữ của bộ nhớ

Khả năng ghi

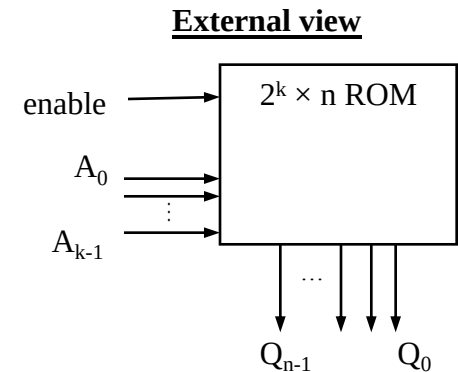
- Phạm vi của “khả năng ghi”
 - Mức cao
 - Bộ xử lý ghi vào bộ nhớ một cách đơn giản và nhanh chóng
 - e.g., RAM
 - Mức trung bình
 - Bộ xử lý ghi vào bộ nhớ, nhưng chậm
 - e.g., FLASH, EEPROM
 - Mức thấp hơn
 - Các thiết bị đặc biệt, “bộ lập trình” phải được sử dụng để ghi bộ nhớ
 - e.g., EPROM, OTP ROM
 - Mức thấp
 - bits chỉ được lưu trữ trong quá trình sản xuất
 - VD: ROM lập trình được bằng mặt nạ
- Bộ nhớ có thể lập trình được trong hệ thống
 - Có thể được ghi bởi bộ xử lý trong hệ thống nhúng
 - Các bộ nhớ loại này có khả năng ghi cao hoặc trung bình

Chất lượng lưu trữ

- Phạm vi chất lượng lưu trữ
 - Loại cao
 - Loại không bao giờ mất bit
 - VD: mask-programmed ROM
 - Loại trung bình
 - Có khả năng lưu trữ bit nhiều ngày, nhiều tháng, hoặc nhiều năm sau khi tắt nguồn
 - VD: NVRAM
 - Loại trung bình thấp
 - Có khả năng lưu trữ bit khi có nguồn cung cấp
 - VD: SRAM
 - Loại thấp
 - Mất bit gần như ngay sau khi được ghi
 - VD: DRAM
- Bộ nhớ không thay đổi được
 - Lưu trữ bit ngay cả khi không được cấp nguồn
 - Chất lượng lưu trữ cao hoặc trung bình

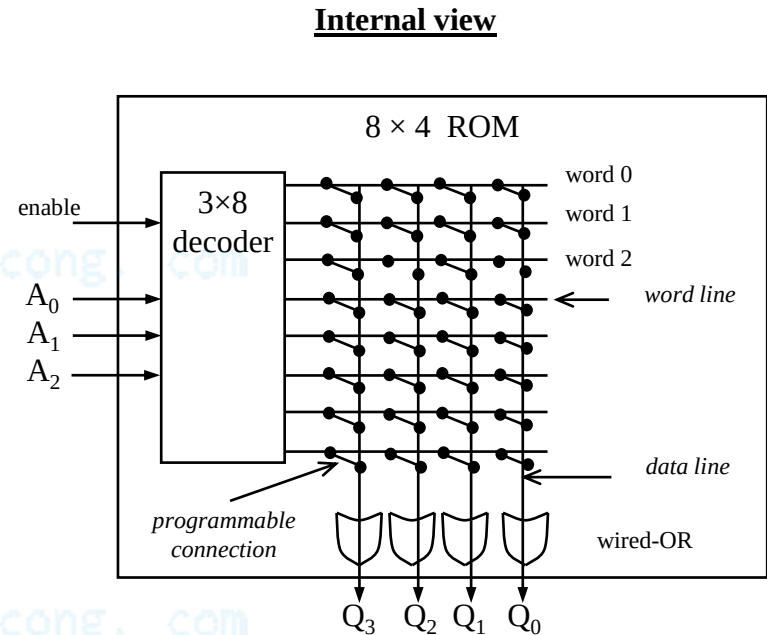
ROM: Bộ nhớ “chỉ đọc”

- Là bộ nhớ không thay đổi được
- Có thể đọc nhưng không thể ghi bởi một bộ xử lý trong hệ thống nhúng
- Thường được ghi bằng cách “lập trình”, trước khi tích hợp trong hệ thống nhúng
- Sử dụng
 - Lưu trữ chương trình phần mềm cho bộ xử lý chức năng chung
 - Lệnh trong chương trình có thể là 1 hoặc nhiều từ nhớ trong ROM
 - Lưu trữ các dữ liệu cố định
 - Thực hiện các mạch tổ hợp



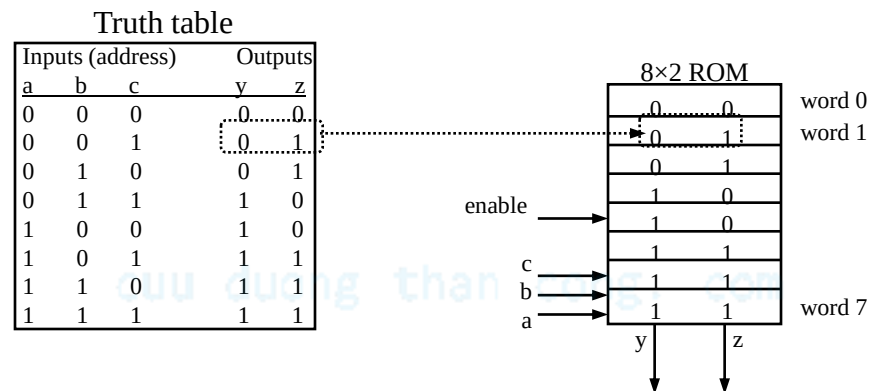
Ví dụ: 8 x 4 ROM

- Hàng ngang = từ
- Hàng đứng = dữ liệu
- Các đường kết nối ở giao điểm
- Bộ giải mã nối đường dẫn của từ số 2 là 1 nếu địa chỉ đầu vào là 010
- Đường dữ liệu Q_3 và Q_1 đặt là 1 bởi vì có một kết nối “được lập trình” với đường của từ số 2
- Từ số 2 không được kết nối với đường dữ liệu Q_2 và Q_0
- Đầu ra là 1010



Thực hiện mạch tổ hợp

- Bất cứ mạch tổ hợp nào với n hàm có cùng k biến đều có thể được thực hiện bằng ROM $2^k \times n$



ROM lập trình bằng mặt nạ

- Các kết nối được “lập trình” khi sản xuất
 - Thiết lập các mặt nạ
- Khả năng ghi thấp nhất
 - Chỉ ghi một lần
- Chất lượng lưu trữ cao nhất
 - Bít không bao giờ thay đổi
- Được sử dụng điển hình cho các thiết kế cuối cùng của hệ với dung lượng lớn
 - Giá NRE cao nếu sản xuất với số lượng ít

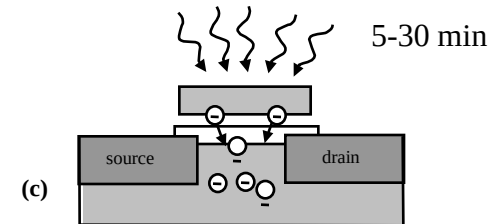
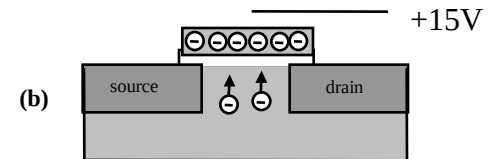
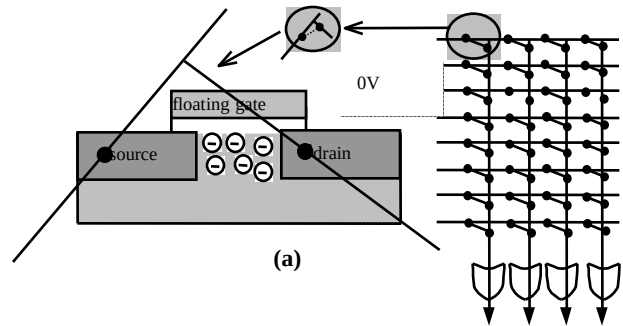
OTP ROM: ROM lập trình một lần

- Các kết nối được “lập trình” sau khi sản xuất bởi người sử dụng
 - Người sử dụng cung cấp nội dung yêu cầu lưu trữ trong ROM
 - Nội dung được đưa vào thiết bị gọi là bộ lập trình ROM
 - Mỗi kết nối có thể lập trình được là một cầu chì
 - Bộ lập trình ROM phá vỡ các cầu chì khi không muốn duy trì kết nối
- Khả năng ghi rất thấp
 - Diễn hình ghi chỉ một lần và yêu cầu thiết bị lập trình ROM
- Chất lượng lưu trữ rất cao
 - Bit không thay đổi trừ khi kết nối với bộ lập trình
- Thường sử dụng trong các sản phẩm cuối cùng
 - Rẻ, khó thay đổi nội dung

EPROM: ROM lập trình ghi xóa

Phần tử có thể lập trình là một transistor MOS

- Transistor có cổng “floating” bao quanh bởi chất cách điện
- (a)** Điện tích âm hình thành một kênh giữa nguồn và máng lưu giữ mức logic “1”
- (b)** Điện áp dương lớn ở “cổng” làm cho các điện tích âm di chuyển ra khỏi kênh và duy trì trong cổng “floating” lưu trữ mức logic 0
- (c)** (Xóa) chiếu tia UV trên bề mặt cổng “floating” làm cho các điện tích âm trở lại kênh từ cổng “floating” lưu trữ mức logic 1
- (d)** Một IC EPROM có một cửa sổ để tia UV có thể chiếu qua



Khả năng ghi tốt

- Có thể tẩy xóa và lập trình hàng nghìn lần

Chất lượng lưu trữ giảm

- Chương trình kéo dài khoảng 10 năm nhưng sẽ bị suy giảm do nhiễu bức xạ và từ trường

Điện hình được sử dụng trong quá trình phát triển sản phẩm

EEPROM: ROM có thể ghi xóa bằng điện

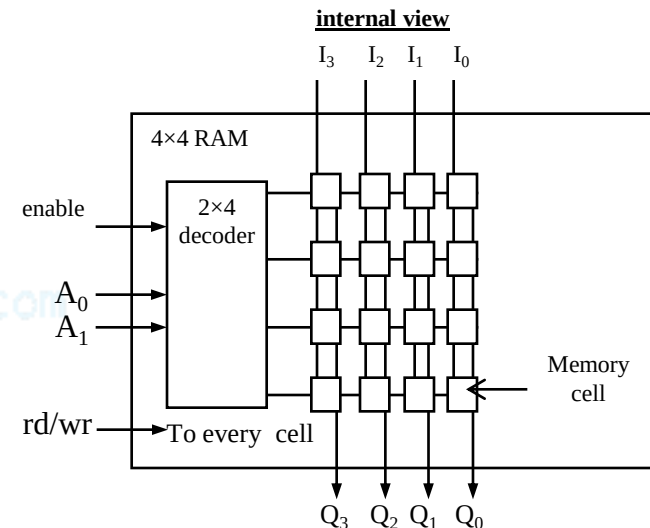
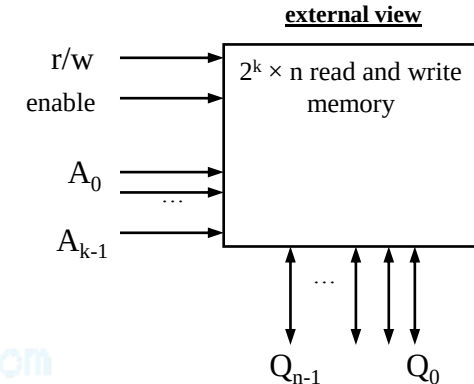
- Lập trình và xóa bằng điện
 - Sử dụng mức điện áp cao hơn bình thường
 - Có thể lập trình và xóa từng từ riêng
 - Khả năng ghi tốt
 - Có thể lập trình trong hệ thống với mạch tích hợp để cung cấp điện áp cao hơn mức thông thường
 - Bộ điều khiển tích hợp trong bộ nhớ thường dùng để ẩn các chi tiết từ người sử dụng bộ nhớ
 - Ghi rất chậm do xóa và lập trình
 - Chân “busy” biểu thị với bộ xử lý là EEPROM vẫn đang trong quá trình ghi
 - Khả năng xóa và lập trình có thể lên tới 10 nghìn lần
 - Chất lượng lưu trữ tương tự như EPROM (khoảng 10 năm)
 - Thuận tiện hơn EPROMs, nhưng đắt hơn
-

Bộ nhớ Flash

- Mở rộng của EEPROM
 - Nguyên lý công “floating” tương tự
 - Chất lượng lưu trữ và khả năng ghi tương tự
 - Xóa nhanh
 - Nhiều ô nhớ được xóa đồng thời, chứ không chỉ một ô nhớ tại một thời điểm
 - Các khối nhớ có kích thước khoảng vài nghìn byte
 - Khả năng ghi từng từ có thể chậm hơn
 - Toàn bộ khối phải được đọc, từ được cập nhật, sau đó toàn bộ khối được ghi
 - Sử dụng với các hệ thống nhúng lưu trữ dữ liệu lớn trong bộ nhớ
 - VD: camera số, điện thoại di động
-

RAM: Bộ nhớ “Truy cập ngẫu nhiên”

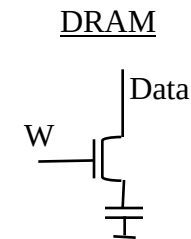
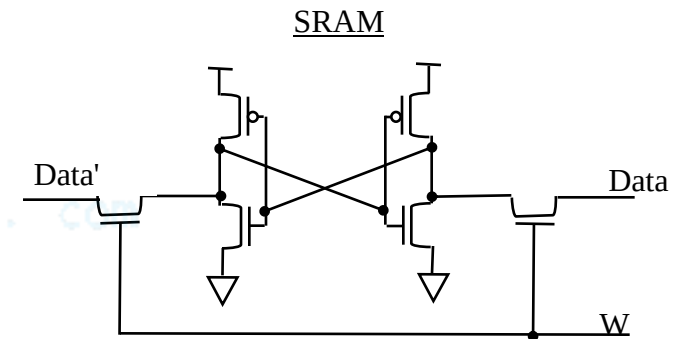
- **Thường là bộ nhớ “volatile”**
 - Dữ liệu bị mất khi không có nguồn cấp
- **Khi và đọc dễ dàng trong hệ thống nhúng trong quá trình làm việc**
- **Cấu trúc bên trong phức tạp hơn ROM**
 - Một từ nhớ có vài ô nhớ, mỗi ô nhớ chứa một bit nhớ
 - Mỗi đường dữ liệu vào và ra kết nối tới mỗi ô nhớ trong cột của nó
 - rd/wr Kết nối tới tất cả các ô nhớ
 - Khi hàng được “enable” bởi bộ giải mã, mỗi ô nhớ có mức logic lưu trữ bit dữ liệu đầu vào khi chân rd/wr biểu thị ghi hoặc đầu ra lưu trữ bit khi chân rd/wr biểu thị đọc



Các kiểu RAM cơ bản

- SRAM: RAM tĩnh
 - Ô nhớ dùng flip-flop để lưu trữ bit
 - Yêu cầu 6 transistors
 - Giữ dữ liệu khi có nguồn cấp
- DRAM: RAM động
 - Ô nhớ dùng transistor MOS và tụ để lưu trữ bit
 - Gọn nhẹ hơn SRAM
 - Yêu cầu “Refresh” do tụ bị dò
 - Các ô nhớ của từ được “refresh” khi đọc
 - Tốc độ “refresh” thường khoảng 15.625 microsec.
 - Truy cập chậm hơn SRAM

memory cell internals



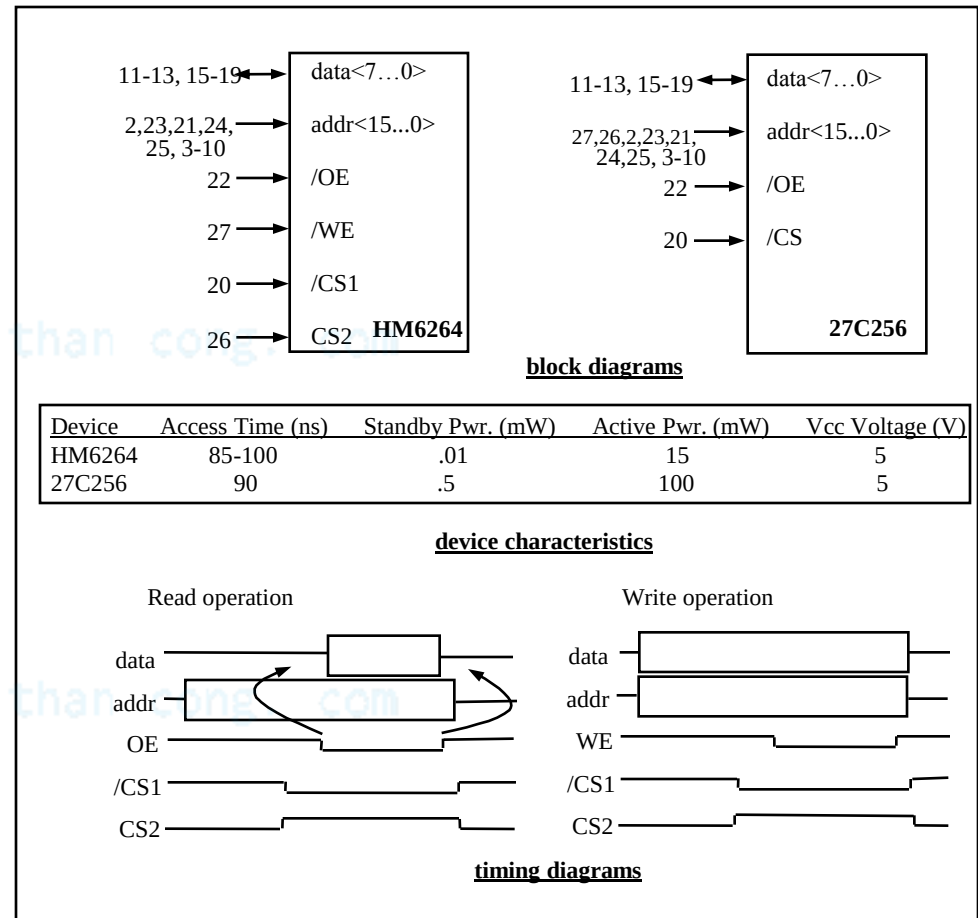
Các biến thể của RAM

- PSRAM: RAM giả tĩnh
 - DRAM với bộ điều khiển “refresh” tích hợp bên trong bộ nhớ
 - Giá thấp và mật độ lưu trữ cao hơn so với SRAM
 - NVRAM: Nonvolatile RAM
 - Lưu trữ giữ liệu ngay cả khi không cấp nguồn
 - RAM có nguồn dự phòng
 - SRAM với battery được kết nối vĩnh cửu
 - Ghi nhanh như đọc
 - Không giới hạn số lần ghi
 - SRAM với EEPROM hoặc flash
 - Lưu trữ toàn bộ nội dung của RAM trên EEPROM hoặc flash trước khi ngắt nguồn
-

Ví dụ:

Thiết bị HM6264 & 27C256 RAM/ROM

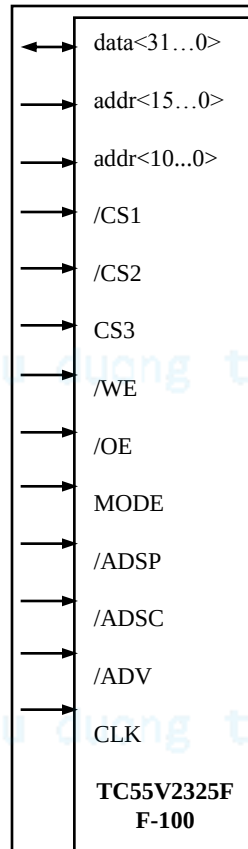
- Là thiết bị giá thấp, mật độ thấp
- Thường dùng trong bộ hệ thống nhúng dựa trên vi điều khiển 8-bit
- Hai số đầu thể hiện kiểu thiết bị
 - RAM: 62
 - ROM: 27
- Các số tiếp theo biểu thị dung lượng tính theo kilobits



Ví dụ:

Thiết bị nhớ TC55V2325FF-100

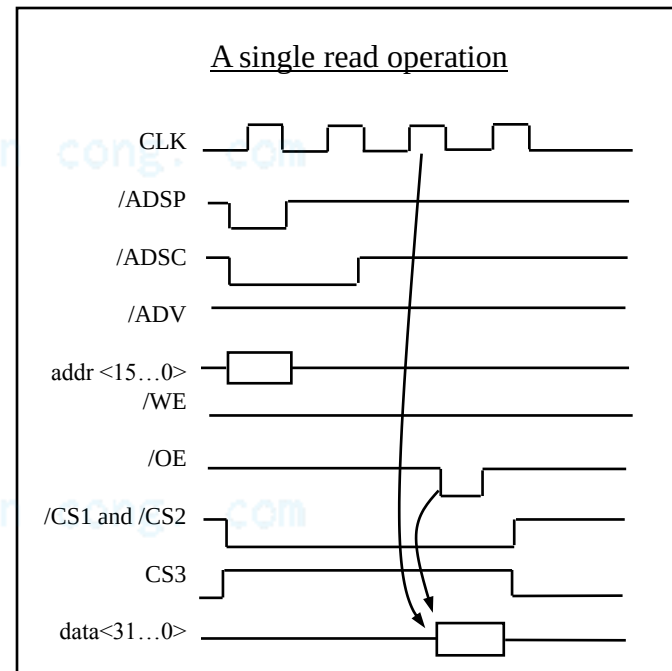
- Thiết bị nhớ SRAM 2-megabit
- Thiết kế để giao tiếp với bộ xử lý 32-bit
- Có khả năng đọc ghi tuần tự nhanh



block diagram

Device	Access Time (ns)	Standby Pwr. (mW)	Active Pwr. (mW)	Vcc Voltage (V)
TC55V2325FF-100	10	na	1200	3.3

device characteristics

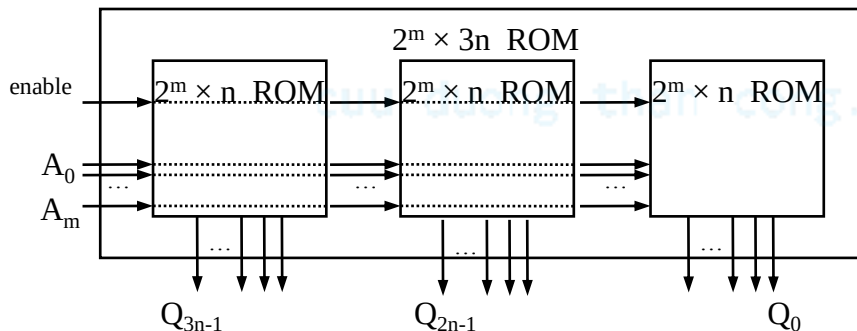


timing diagram

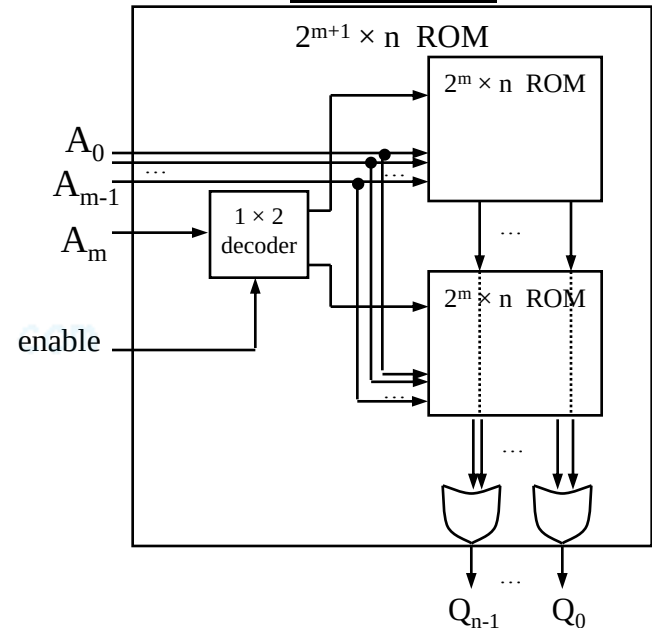
Ghép bộ nhớ

- Kích thước bộ nhớ yêu cầu thường khác với kích thước thiết kế của bộ nhớ
- Khi bộ nhớ thiết kế sẵn lớn hơn yêu cầu, chúng ta chỉ cần bỏ các địa chỉ nhớ ở vùng cao và đường dữ liệu ở vùng cao
- Khi bộ nhớ thiết kế sẵn nhỏ hơn yêu cầu, chúng ta cần ghép một vài bộ nhớ nhỏ hơn thành một bộ nhớ lớn
 - Kết nối kề nhau để tăng độ rộng từ nhớ
 - Kết nối tầng để tăng số từ
 - Dùng các đường địa chỉ vùng cao để lựa chọn bộ nhớ nhỏ hơn sử dụng bộ giải mã
 - Kết hợp cả hai khi cần tăng số từ cũng như độ rộng từ

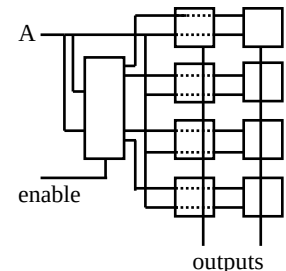
Tăng độ rộng từ nhớ



Tăng số từ nhớ

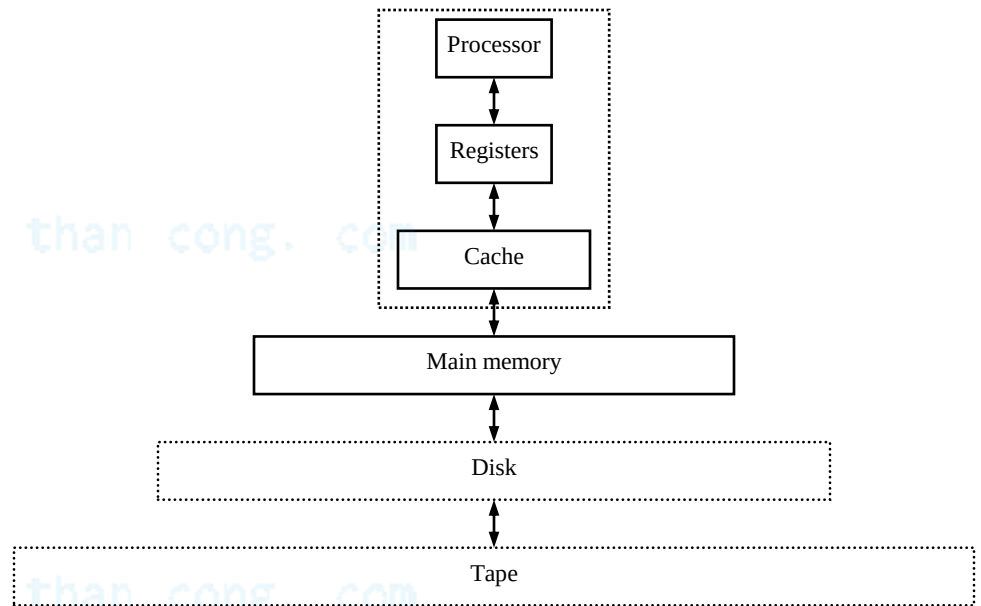


Tăng số từ cũng như độ rộng từ nhớ



Phân cấp bộ nhớ

- Chúng ta muốn bộ nhớ rẻ, truy cập nhanh
- Bộ nhớ chính
 - Dùng bộ nhớ dung lượng lớn, rẻ, chậm để lưu trữ toàn bộ chương trình và dữ liệu
- Cache
 - Dùng bộ nhớ nhỏ, đắt tiền và nhanh để lưu trữ phần “copy” của phần dữ liệu truy cập thuộc bộ nhớ lớn
 - Có thể có nhiều mức cache



Cache

- **Thường được thiết kế dùng SRAM**
 - Nhanh hơn nhưng đắt hơn DRAM
- **Thường đặt trên cùng chip với bộ xử lý**
 - Không gian hạn chế, vì vậy có dung lượng nhỏ hơn nhiều so với bộ nhớ chính bên ngoài chip
 - Truy cập nhanh hơn (thường là 1 chu kỳ đồng hồ so với vài chu kỳ đồng hồ so với bộ nhớ ngoài)
- **Một số thiết kế cache**
 - Bản đồ cache, cơ chế thay thế, và kỹ thuật ghi

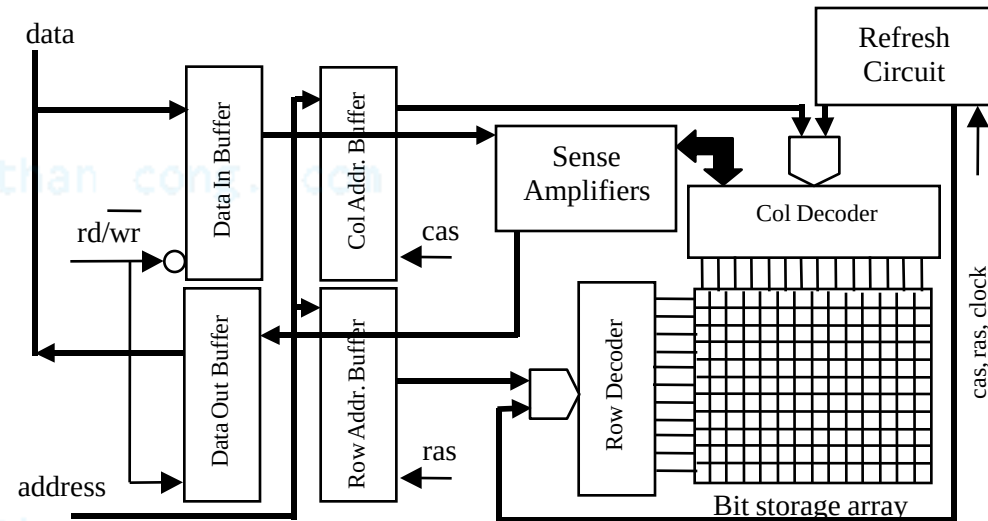
cuu duong than cong. com

RAM cải tiến

- DRAMs thường được sử dụng như bộ nhớ chính trong bộ xử lý của hệ thống nhúng
 - Dung lượng lớn, giá thành thấp
- Các biến thể chính của DRAMs
 - Cần tương thích với tốc độ của bộ xử lý
 - FPM DRAM: DRAM kiểu trang nhanh
 - EDO DRAM: DRAM có đầu ra dữ liệu mở rộng
 - SDRAM/ESDRAM: DRAM đồng bộ và đồng bộ mở rộng

DRAM cơ bản

- Bus địa chỉ ghép giữa các phần tử hàng và cột
- Địa chỉ hàng và cột được chốt, tuần tự, bằng các tín hiệu *ras* và *cas*, tương ứng
- Mạch “refresh” có thể bên trong hoặc bên ngoài DRAM



Vấn đề tích hợp DRAM

- SRAM dễ dàng tích hợp trên một chip như bộ xử lý
- DRAM khó hơn
 - Sự khác biệt giữa thiết kế DRAM và mạch tổ hợp
 - Mục tiêu của người thiết kế mạch tổ hợp:
 - Giảm điện dung ký sinh để giảm trễ truyền lan và mức tiêu thụ công suất
 - Mục tiêu của người thiết kế DRAM:
 - Tạo ra điện dung để lưu trữ thông tin
 - Quá trình tổ hợp gặp khó khăn

Đơn vị quản lý bộ nhớ (MMU)

- Chức năng của MMU
 - Thực hiện “refresh” DRAM, giao tiếp bus và điều phối
 - Thực hiện việc chia sẻ bộ nhớ
 - Chuyển đổi địa chỉ nhớ từ bộ xử lý sang địa chỉ nhớ vật lý của DRAM
- Các CPUs thường có bộ MMU tích hợp sẵn
- Bộ xử lý chức năng đơn có thể sử dụng để xây dựng MMU

Reference

- This document is used for educational purposes only
- This document contains material from other sources (incl. textbook, lecture notes, application notes, ...)
- Our thanks go to the contributors, references

CHƯƠNG 2: CẤU TRÚC PHẦN CỨNG HỆ THỐNG NHÚNG

Bài 5: Giao diện

cuu duong than cong. com

cuu duong than cong. com

Tổng quan

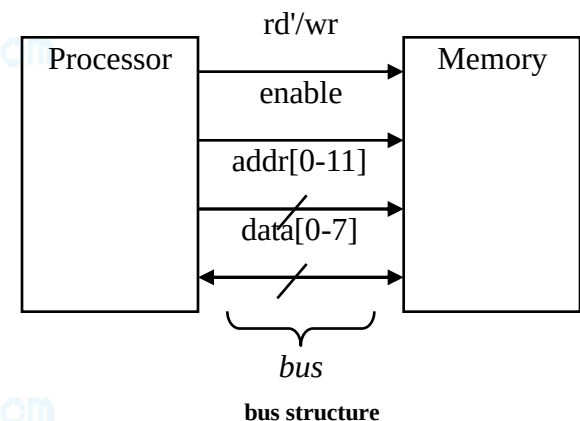
- Khái niệm cơ bản
- Giao diện vi xử lý
 - Địa chỉ I/O
 - Ngắt
 - Truy cập bộ nhớ trực tiếp (DMA)
- Bus phân cấp
- Thủ tục - Protocols
 - Nối tiếp
 - Song song
 - Không dây

Giới thiệu

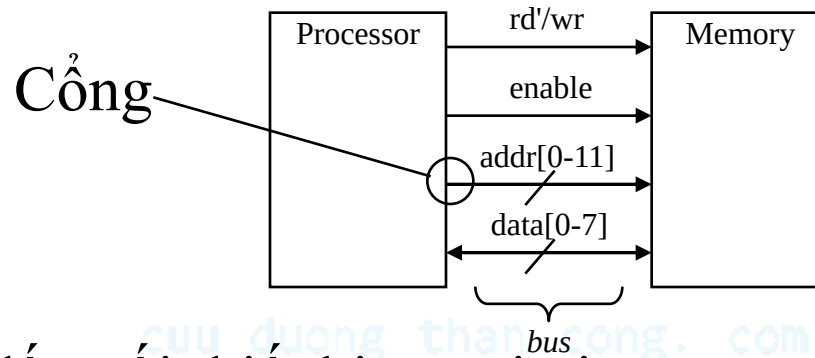
- Chức năng của một hệ thống nhúng
 - Xử lý
 - Biến đổi dữ liệu
 - Thực hiện công việc dùng bộ xử lý
 - Lưu trữ
 - Lưu trữ dữ liệu
 - Thực hiện dùng bộ nhớ
 - Truyền thông
 - Trao đổi dữ liệu giữa bộ xử lý và bộ nhớ
 - Thực hiện sử dụng bus
 - Thường được gọi là giao diện - *interfacing*
-

Một bus đơn giản

- Dây nối:
 - Đơn hướng hay song hướng
 - Một đường truyền có thể có nhiều dây nối
- Bus
 - Một nhóm dây nối có chức năng riêng
 - Bus địa chỉ, bus dữ liệu
 - Hoặc, việc tập hợp các dây nối
 - Địa chỉ, dữ liệu và điều khiển
 - Thủ tục (protocol): quy tắc cho việc trao đổi thông tin



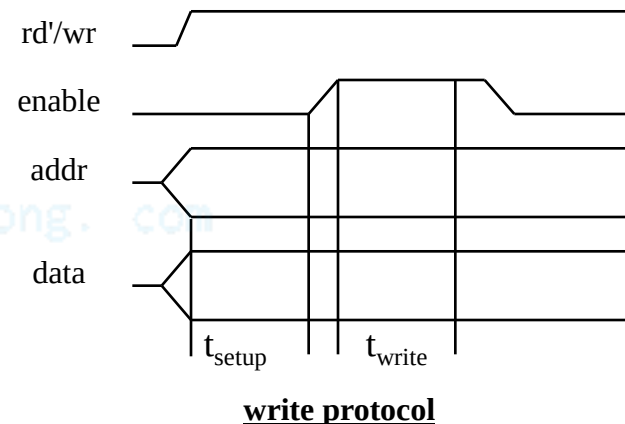
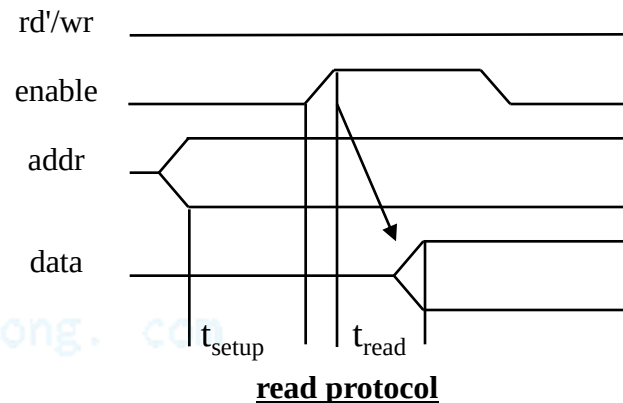
Cổng



- Cổng dùng đầu nối thiết bị ngoại vi
- Kết nối bus tới bộ xử lý và bộ nhớ
- Thường được gọi là “chân” (*pin*)
 - Là chân thực tế của IC cắm vào để cắm trên mạch in
 - Đôi khi các điểm tiếp xúc thay cho chân
 - Ngày nay, các “pads” kim loại kết nối bộ xử lý và bộ nhớ trong một IC
- Cổng gồm một đường hoặc nhiều đường với chức năng riêng
 - VD: cổng địa chỉ 12-đường

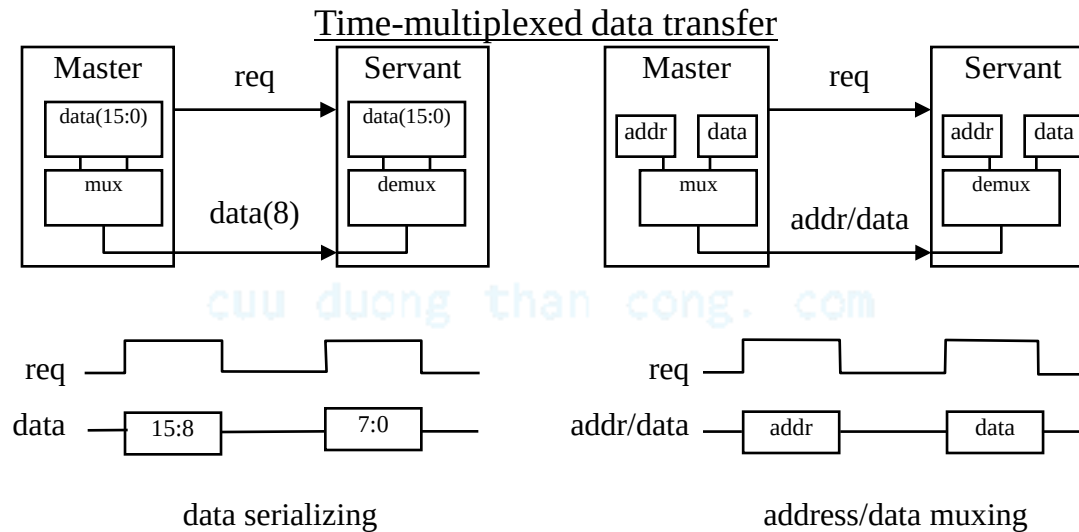
Giản đồ thời gian

- Phương pháp thông thường nhất để mô tả một thủ tục truyền thông trên công (hoặc bus)
- Thời gian biểu thị trên trục x theo chiều sang bên phải
- Tín hiệu điều khiển: thấp hoặc cao
 - Có thể tích cực thấp
 - Sử dụng thuật ngữ *assert* (active) và *deassert*
- Tín hiệu dữ liệu: not valid hoặc valid
- Thủ tục đôi khi có các thủ tục con
 - Chu kỳ bus, VD đọc và ghi
 - Mỗi thao tác có thể gồm nhiều chu kỳ đồng hồ
- Ví dụ quá trình đọc
 - *rd'/wr* ở mức thấp, địa chỉ đặt lên trên *addr* trong khoảng thời gian t_{setup} trước khi chân *enable* được tác động, cho phép bộ nhớ đặt dữ liệu trên chân *data* trong khoảng thời gian t_{read}

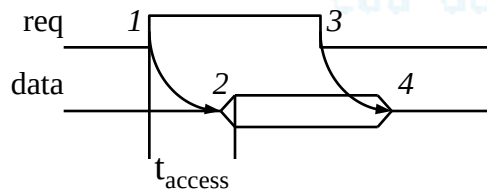
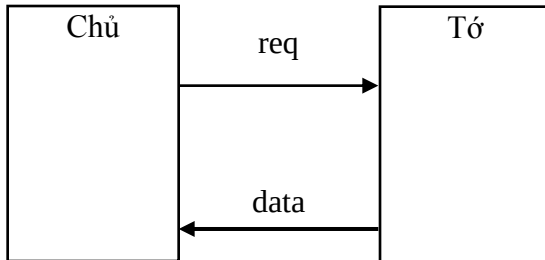


Khái niệm cơ bản về thủ tục

- Kiểu tác động: master khởi đầu, slave đáp ứng
- Hướng truyền: phía gửi, phía nhận
- Địa chỉ: kiểu dữ liệu đặc biệt
 - Quy định một vị trí trên bộ nhớ, một ngoại vi, hoặc một thanh ghi trên ngoại vi
- Ghép thời gian
 - Chia sẻ một bộ kênh truyền cho nhiều dữ liệu khác nhau
 - Tiết kiệm dây truyền, nhưng tốn thời gian

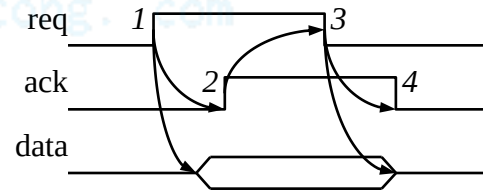
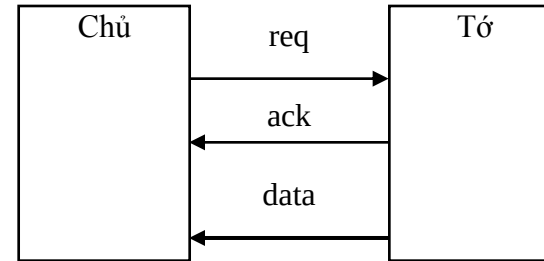


Khái niệm thủ tục đơn giản: các phương pháp điều khiển



1. Chủ phát *req* để nhận dữ liệu
2. Tớ đưa dữ liệu lên trong khoảng t_{access}
3. Chủ nhận dữ liệu và kết thúc *req*
4. Tớ sẵn sàng cho chu kỳ kế tiếp

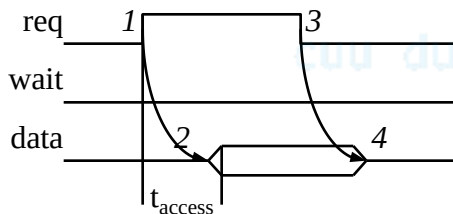
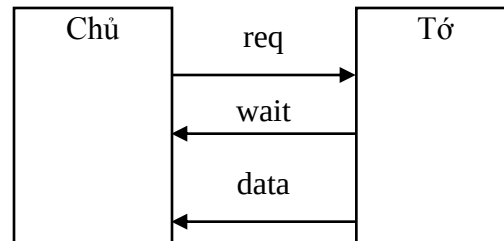
Thủ tục “Strobe”



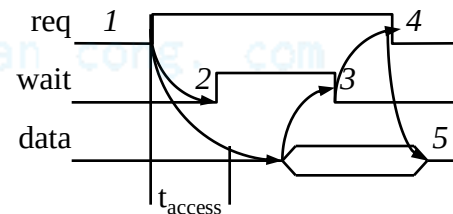
1. Chủ phát *req* để nhận dữ liệu
2. Tớ đưa dữ liệu lên bus và gửi tín hiệu *ack*
3. Chủ nhận dữ liệu và ngắt *req*
4. Tớ sẵn sàng cho chu kỳ kế tiếp

Thủ tục “Handshake”

Thủ tục strobe/handshake có thỏa hiệp



1. Chủ phát *req* để nhận dữ liệu
2. Tớ đưa dữ liệu lên bus *trong khoảng* t_{access}
(đường *wait* không được sử dụng)
3. Chủ nhận dữ liệu và kết thúc *req*
4. Tớ sẵn sàng cho chu kỳ kế tiếp



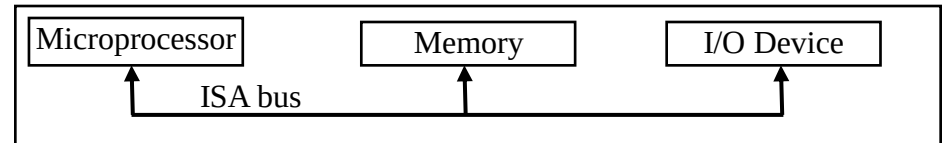
1. Chủ phát *req* để nhận dữ liệu
2. Tớ không đặt *data* t_{access} , đợi *wait*
3. Tớ đặt dữ liệu trên bus và ngắt *wait*
4. Chủ nhận dữ liệu và kết thúc *req*
5. Tớ sẵn sàng cho chu kỳ kế tiếp

Trường hợp đáp ứng nhanh

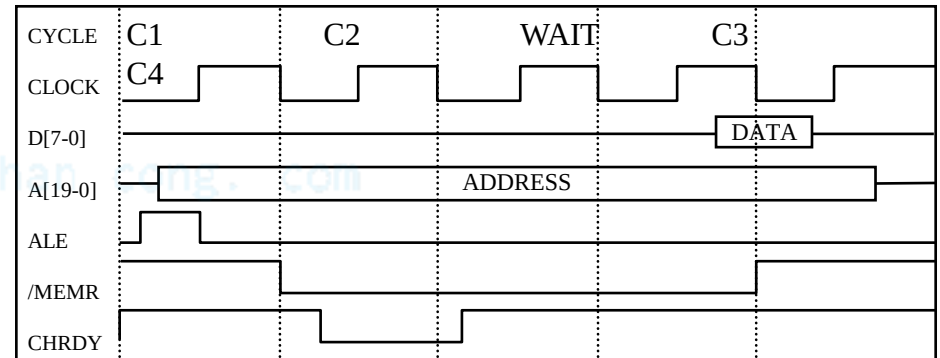
Trường hợp đáp ứng chậm

Thủ tục bus ISA bus – truy cập bộ nhớ

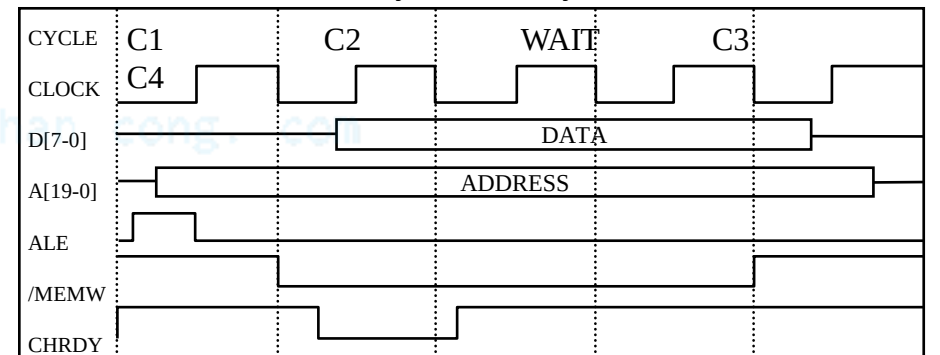
- ISA: Kiến trúc công nghiệp tiêu chuẩn
 - Sử dụng cho các họ 80x86's
- Đặc điểm
 - Đường địa chỉ 20-bit
 - Điều khiển strobe/handshake thỏa hiệp
 - Mặc định 4 chu kỳ máy



memory-read bus cycle



memory-write bus cycle

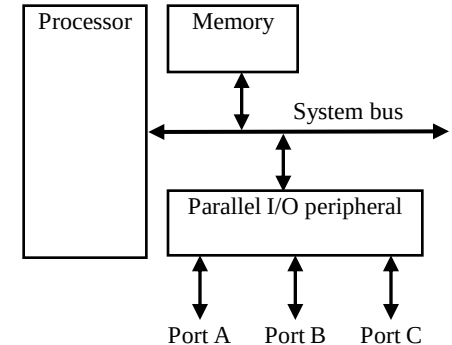


Giao tiếp vi xử lý: Địa chỉ I/O

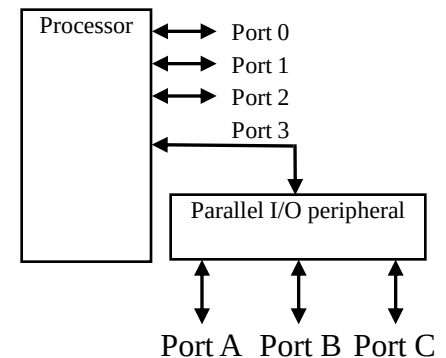
- Một bộ vi xử lý giao tiếp với các thiết bị khác qua chân của nó
 - I/O dựa trên cổng (I/O song song)
 - Bộ xử lý có một hoặc nhiều cổng N-bit
 - Chương trình của bộ xử lý đọc và ghi một cổng như đối với một thanh ghi
 - VD: $P0 = 0xFF$; $v = P1.2$; P0 và P1 là cổng 8-bit
 - I/O dựa trên bus
 - Bộ xử lý có cổng địa chỉ, dữ liệu và điều khiển trên một bus đơn
 - Thủ tục truyền thông được xây dựng bên trong bộ xử lý
 - Một lệnh sẽ thực hiện việc ghi hay đọc trên bus

Thỏa hiệp/mở rộng

- Ngoại vi I/O song song
 - Khi bộ xử lý chỉ có I/O dựa trên bus nhưng chúng ta muốn I/O song song
 - Mỗi cổng trên ngoại vi kết nối với một thanh ghi trong ngoại vi được read/written bởi bộ xử lý
- I/O song song mở rộng
 - Khi bộ xử lý chỉ có I/O dựa trên cổng nhưng chúng ta cần nhiều cổng hơn
 - Một hoặc nhiều cổng của bộ xử lý giao tiếp với ngoại vi I/O song song sẽ mở rộng tổng số cổng I/O
 - VD: mở rộng 4 cổng thành 6 cổng như hình bên



Adding parallel I/O to a bus-based I/O processor



Extended parallel I/O

Các kiểu I/O dựa trên bus:

I/O bản đồ nhớ và I/O tiêu chuẩn

- Bộ xử lý trao đổi với cả bộ nhớ và ngoại vi sử dụng chung bus – có hai cách để trao đổi với ngoại vi
 - I/O bản đồ nhớ
 - Thanh ghi của ngoại vi chiếm địa chỉ trong cùng không gian địa chỉ của bộ nhớ
 - VD: Bus có địa chỉ 16-bit
 - Địa chỉ 32k vùng thấp dùng cho bộ nhớ
 - Địa chỉ 32k vùng cao dùng cho ngoại vi
 - I/O tiêu chuẩn (I/O-mapped I/O)
 - Các chân bổ sung (*M/IO*) trên bus biểu thị bộ nhớ hoặc ngoại vi được truy cập
 - VD: Bus có địa chỉ 16-bit
 - Tất cả 64K địa chỉ dùng cho bộ nhớ khi chân *M/IO* ở mức 0
 - Tất cả 64K địa chỉ dùng cho ngoại vi khi chân *M/IO* ở mức 1

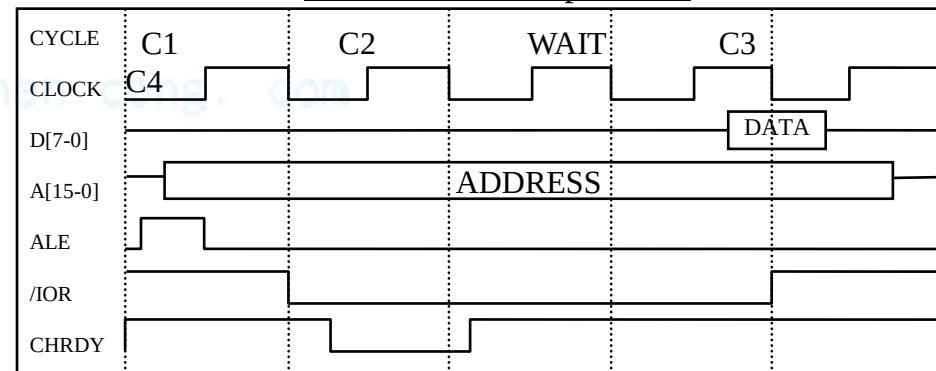
So sánh I/O bản đồ nhớ và I/O tiêu chuẩn

- I/O bản đồ nhớ
 - Không yêu cầu lệnh đặc biệt
 - Lệnh assembly có các lệnh như MOV và ADD cngx cho phép làm việc với ngoại vi
 - I/O tiêu chuẩn yêu cầu các lệnh đặc biệt (VD: IN, OUT) để di chuyển dữ liệu giữa thanh ghi của ngoại vi và bộ nhớ
- I/O tiêu chuẩn
 - Không mất địa chỉ nhớ cho ngoại vi
 - Bộ giải mã địa chỉ đơn giản hơn
 - Khi số ngoại vi phải nhỏ hơn không gian địa chỉ thì các bit địa chỉ vùng cao có thể bỏ trống
 - Bộ so sánh nhỏ hơn và/hoặc nhanh hơn

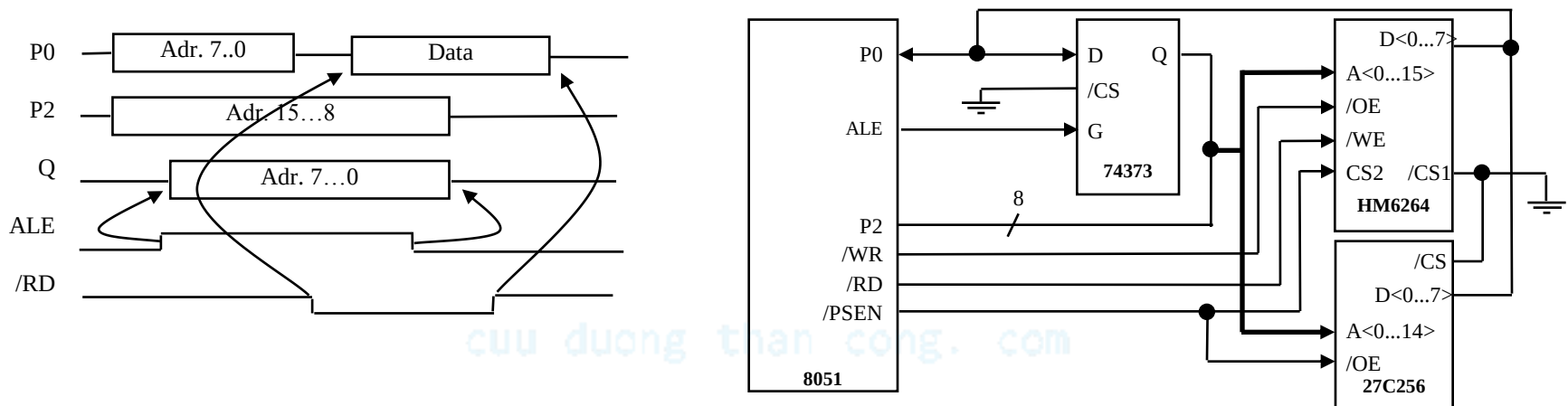
Bus ISA

- ISA cung cấp I/O tiêu chuẩn
 - /IOR khác với /MEMR để cho phép đọc từ thiết bị ngoại vi
 - /IOW sử dụng cho ghi
 - Không gian địa chỉ 16-bit cho I/O vs. không gian địa chỉ 20-bit cho bộ nhớ
 - Ngoài ra nó tương tự với thủ tục truy cập bộ nhớ

ISA I/O bus read protocol



Một thủ tục truy cập bộ nhớ đơn giản



- Giao tiếp 8051 với bộ nhớ ngoài
 - Cổng P0 và P2 hỗ trợ I/O dựa trên công khi bộ nhớ trong của 8051 được sử dụng
 - Các cổng này phục vụ như bus dữ liệu/địa chỉ khi bộ nhớ ngoài được sử dụng
 - Địa chỉ 16-bit và dữ liệu 8-bit được ghép theo thời gian; 8-bits thấp của địa chỉ phải được chốt nhờ tín hiệu ALE

Giao tiếp vi xử lý: ngắt

- Giả sử một thiết bị ngoại vi yêu cầu được thu nhận dữ liệu lập tức bởi bộ xử lý
 - Bộ xử lý có thể kiểm tra liên tục ngoại vi xem dữ liệu đã đến hay chưa – rất kém hiệu quả
 - Ngoại vi có thể *ngắt* bộ xử lý khi nó có dữ liệu
- Yêu cầu phải có thêm chân: Int
 - Nếu Int là 1, bộ xử lý dừng chương trình đang thực hiện, nhảy tới một Interrupt Service Routine, hoặc ISR
 - Được biết như đầu vào điều khiển ngắt I/O

Giao tiếp vi xử lý: ngắt

- Địa chỉ của ISR (interrupt address vector) là gì?
 - Ngắt cố định
 - Địa chỉ ngắt được thiết lập sẵn trong bộ xử lý, không thể thay đổi
 - Ngắt vector
 - Ngoại vi phải cung cấp địa chỉ
 - Thường sử dụng khi bộ xử lý có nhiều ngoại vi kết nối chung một bus
 - Phối hợp: bảng địa chỉ ngắt

Truyền thông song song

- Nhiều chân dữ liệu, điều khiển, và có thể cả công suất
 - Mỗi dây một bit
- Tốc độ dữ liệu cao, cự ly truyền dẫn ngắn
- Thường sử dụng khi kết nối các thiết bị trên cùng IC hoặc trên cùng bo mạch
 - Bus phải đủ ngắn
 - Dây dài dẫn đến điện dung ký sinh cao
 - Nhiều xuyên kênh tăng khi dây dài
- Giá cao hơn

Truyền thông nối tiếp

- Dây dữ liệu đơn, có thể bao gồm dây điều khiển và công suất
- Từ được truyền đi từng bit một
- Truyền dữ liệu tốt hơn với cự ly dài
 - Điện dung ký sinh ít
- Rẻ hơn
- Thủ tục truyền thông và giao tiếp linh hoạt hơn
 - Phía gửi phải tách từ thành các bit
 - Phía thu làm ngược lại
 - Tín hiệu điều khiển thường gửi cùng trên một dây với dữ liệu làm cho thủ tục truyền phức tạp hơn

Truyền thông không dây

- Hồng ngoại (IR)
 - Tần số dưới phổ ánh sáng nhìn thấy
 - Đi ốt phát ánh sáng hồng ngoại để tạo tín hiệu
 - Transistor hồng ngoại xác định tín hiệu, dẫn khi có ánh sáng hồng ngoại
 - Giá rẻ
 - Cần đường truyền thẳng, phạm vi hẹp
- Sóng vô tuyến (RF)
 - Sóng điện từ trong phổ radio
 - Mạch tương tự và anten ở cả hai phía phát và thu
 - Không cần đường truyền thẳng, phạm vi hoạt động phụ thuộc vào công suất phát

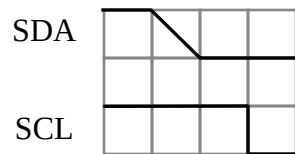
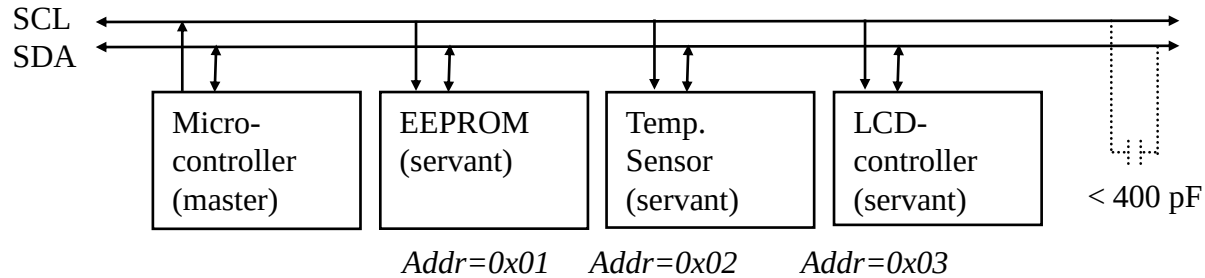
Xác định lỗi và sửa lỗi

- Thường là một phần của thủ tục truyền
 - Xác định lỗi: là khả năng của bộ thu xác định lỗi trong quá trình truyền
 - Sửa lỗi: khả năng phối hợp giữa bộ thu và bộ phát để khôi phục lỗi
 - bit lỗi đơn: sử dụng một bit
 - Bit lỗi nhóm: sử dụng nhóm bit
 - Chẵn lẻ: bit bổ sung cho việc xác định lỗi
 - Kiểm tra tổng: từ bổ sung với gói dữ liệu nhiều từ
-

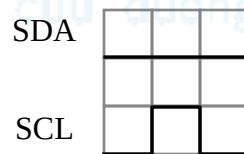
Thủ tục nối tiếp: I²C

- I²C (Inter-IC)
 - Thủ tục bus nối tiếp 2 dây phát triển bởi Philips Semiconductors gần 20 năm trước
 - Cho phép các ngoại vi của ICs giao tiếp với nhau sử dụng phần cứng truyền thông đơn giản
 - Tốc độ truyền dữ liệu lên tới 100 kbits/s và địa chỉ 7-bit ở chế độ hoạt động thông thường
 - 3.4 Mbits/s và địa chỉ 10-bit ở chế độ nhanh
 - Các thiết bị có thể giao tiếp với bus I²C:
 - EPROMs, Flash, và một vài bộ nhớ RAM, đồng hồ thời gian thực và vi điều khiển,...

Kiến trúc bus I2C



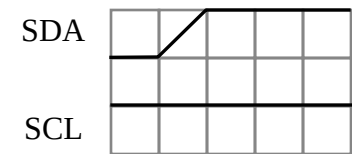
Start condition



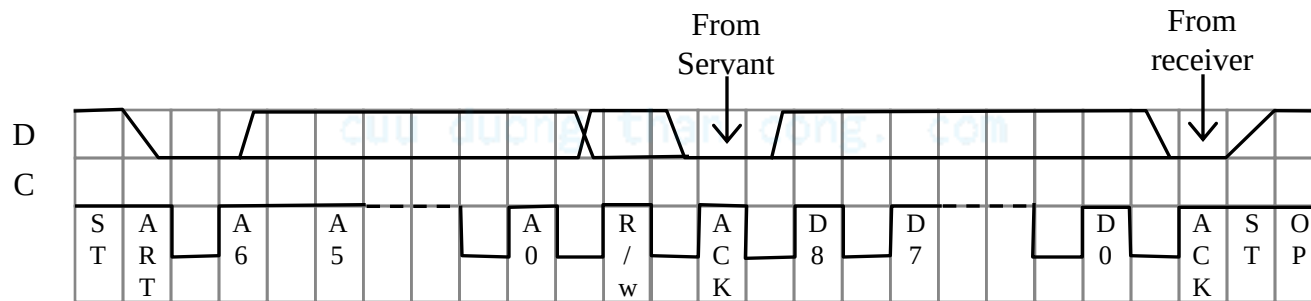
Sending 0



Sending 1



Stop condition



Typical read/write cycle

Thủ tục nối tiếp: CAN

- CAN (Controller area network)
 - Thủ tục cho ứng dụng thời gian thực
 - Phát triển bởi Robert Bosch GmbH
 - Thường sử dụng trong các hệ thống thông tin trong oto
 - Các ứng dụng sử dụng CAN ngày nay bao gồm:
 - Điều khiển thang máy, máy photo, hệ thống điều khiển tự động, thiết bị y tế
 - Tốc độ truyền dữ liệu lên tới 1 Mbit/s và địa chỉ 11-bit
 - Các thiết bị có thể giao tiếp với CAN:
 - 8051-tương thích bộ xử lý 8592 và bộ điều khiển CAN
 - Thiết kế vật lý thực tế của bus CAN không quy định trong thủ tục
-

Thủ tục nối tiếp: FireWire

- FireWire (a.k.a. I-Link, Lynx, IEEE 1394)
 - Bus nối tiếp chất lượng cao phát triển bởi Apple Computer Inc.
 - Thiết kế cho việc giao tiếp các thiết bị điện tử độc lập
 - e.g., Desktop, scanner
 - Tốc độ truyền dữ liệu từ 12.5 tới 400 Mbits/s, 64-bit địa chỉ
 - Khả năng Plug-and-play
 - Thiết kế theo cấu trúc góc dữ liệu
 - Các ứng dụng sử dụng FireWire bao gồm:
 - disk drives, printers, scanners, cameras

Thủ tục nối tiếp: USB

- USB (Universal Serial Bus)
 - Dễ kết nối hơn giữa PC và màn hình, máy in, modem, máy scanner, camera, vv...
 - Có hai tốc độ dữ liệu:
 - 12 Mbps đối với thiết bị băng rộng
 - 1.5 Mbps đối với thiết bị tốc độ thấp
 - Cấu hình sao có thể được sử dụng
 - Một thiết bị USB (hub) kết nối với PC
 - Hub có thể nhúng trong thiết bị như màn hình, máy in, hoặc bàn phím
 - Nhiều thiết bị USB có thể kết nối với hub
 - Tối đa 127 thiết bị có thể kết nối kiểu này
 - Bộ điều khiển USB host
 - Quản lý và điều khiển độ rộng băng truyền và phần mềm điều khiển cần thiết cho mỗi ngoại vi
 - Điều khiển công suất tùy theo thiết bị được kết nối
-

Thủ tục song song: bus PCI

- PCI Bus (Peripheral Component Interconnect)
 - Bus chất lượng cao được đưa ra bởi Intel vào đầu những năm 1990's
 - Tiêu chuẩn được lựa chọn cho công nghiệp và được quản lý bởi PCISIG (PCI Special Interest Group)
 - Liên kết chíp, board mở rộng, bộ nhớ con của hệ vi xử lý
 - Tốc độ truyền dữ liệu từ 127.2 đến 508.6 Mbits/s và 32-bit địa chỉ
 - Sau đó được mở rộng tới 64-bit trong khi đó vẫn cho phép tương hợp với cấu trúc 32-bit
 - Kiến trúc bus đồng bộ
 - Đường địa chỉ/dữ liệu ghép

Thủ tục song song: bus ARM

- ARM Bus
 - Thiết kế và sử dụng bởi tập đoàn ARM
 - Giao tiếp với vi xử lý ARM
 - Nhiều công ty thiết kế IC có chuẩn bus riêng
 - Tốc độ truyền dữ liệu là một hàm số của chu kỳ đồng hồ
 - Nếu tốc độ đồng hồ của bus là X, tốc độ truyền là $= 16 \times X$ bits/s
 - 32-bit địa chỉ

Thủ tục không dây: IrDA

- IrDA
 - Thủ tục cho phép truyền dữ liệu bằng sóng hồng ngoại điểm tới điểm với cự ly ngắn
 - Được tạo ra bởi Infrared Data Association (IrDA)
 - Tốc độ truyền dữ liệu 9.6 kbps và 4 Mbps
 - Phần cứng IrDA có trong máy notebook, máy in, PDAs, cameras số, điện thoại

Thủ tục không dây: Bluetooth

- Bluetooth
 - Mới, chuẩn quốc tế cho kết nối không dây
 - Dựa trên vô tuyến cự ly ngắn, giá thấp
 - Kết nối trong phạm vi 10m
 - Không yêu cầu đường truyền thẳng
 - VD: kết nối với máy in của phòng khác

Thủ tục không dây: IEEE 802.11

- IEEE 802.11
 - Tiêu chuẩn cho mạng LANs không dây
 - Các thông số cụ thể cho các lớp PHY và MAC của mạng
 - Lớp PHY
 - Lớp vật lý
 - Xử lý việc truyền dữ liệu giữa các lớp
 - Cung cấp tốc độ truyền 1 hoặc 2 Mbps
 - Oạt động trong băng tần từ 2.4 đến 2.4835 GHz (RF)
 - Hoặc 300 đến 428,000 GHz (IR)
 - Lớp MAC
 - Lớp điều khiển truy cập
 - Thủ tục cho việc duy trì thứ tự trong truyền thông nhiều điểm (nút)
 - Tránh xung đột dữ liệu

Tóm tắt

- Các khái niệm về thủ tục
 - Hướng truyền, ghép kênh theo thời gian, phương pháp điều khiển
 - Bộ xử lý chức năng chung
 - I/O dựa trên cổng hoặc dựa trên bus
 - Địa chỉ I/O: I/O bản đồ nhớ hoặc I/O tiêu chuẩn
 - Xử lý ngắt: cố định hay vector
 - Truy cập bộ nhớ trực tiếp
 - Phân cấp bus
 - Thông tin
 - Song song hay nối tiếp, hữu tuyến hay không dây, xác định và sửa lỗi, các lớp
 - Thủ tục nối tiếp: I²C, CAN, FireWire, và USB;
 - Song song: PCI và ARM.
 - Thủ tục không dây nối tiếp: IrDA, Bluetooth, và IEEE 802.11.
-

Reference

- This document is used for educational purposes only
- This document contains material from other sources (incl. textbook, lecture notes, application notes, ...)
- Our thanks go to the contributors, references

CHƯƠNG 2: CẤU TRÚC PHẦN CỨNG HỆ THỐNG NHÚNG

Bài 6: Công nghệ IC

cuu duong than cong. com

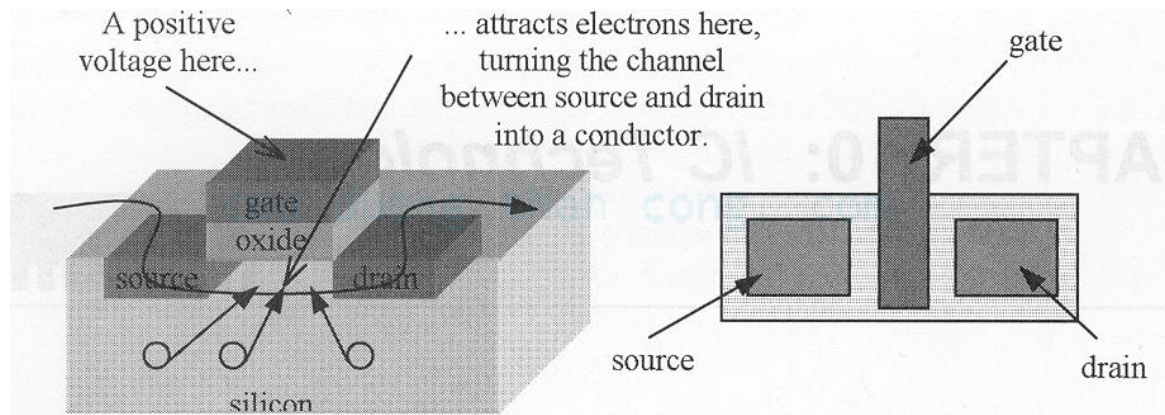
cuu duong than cong. com

Tổng quan

- Cấu trúc IC
- Công nghệ IC chức năng chung (VLSI)
- Công nghệ IC chức năng chuyên biệt (ASIC)
- Công nghệ IC có thể lập trình (PLD)

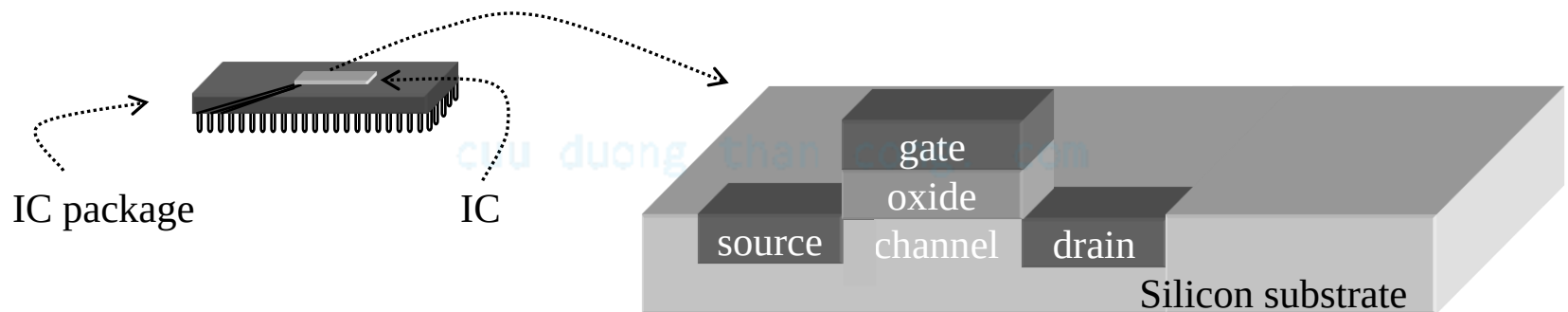
Transistor CMOS

- Nguồn, máng
 - Vùng khuếch tán nơi electrons có thể đi qua
 - Có thể kết nối nhờ tiếp xúc kim loại
- Cổng
 - Vùng Polysilicon nơi đặt điện áp điều khiển
- Oxide
 - Chất cách điện SiO_2 chống rò điện áp cổng



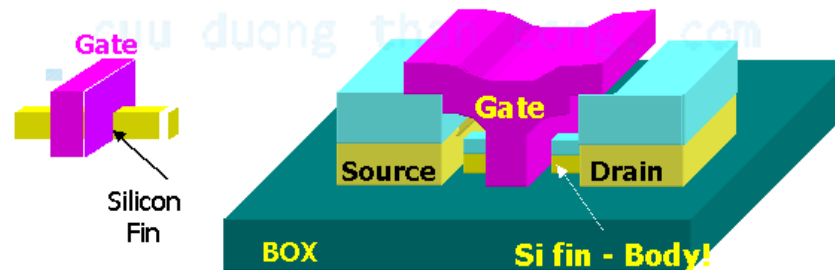
Kết thúc luật Moore?

- Mọi kích thước của MOSFET đã được thay đổi
 - (PMOS) đã giảm xuống
 - Tăng điện dung cổng
 - Giảm dòng rò từ S sang D
 - Độ dày cổng oxide hiện tại là 2.5-3nm
- **Khoảng 25 atoms!!!**



Proposed Structures: FinFET

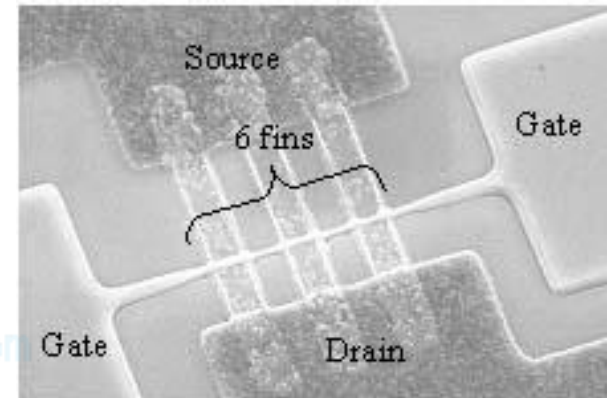
Body is a Thin Silicon Film
Double Gate Structure + Raised Source Drain



X. Huang, et al, 1999 IEDM, p.67-70

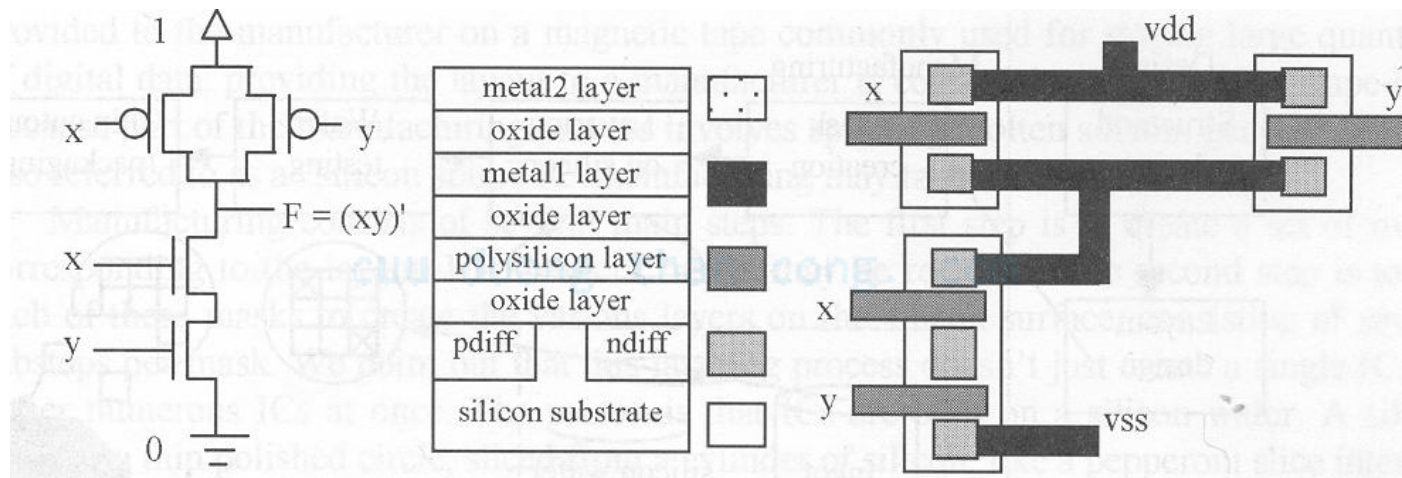
20Ghz +

- FinFET đã được sản xuất với độ dày 18nm
 - Vẫn hoạt động rất tốt
- Mô phỏng chỉ ra rằng nó có thể đạt tới 10nm
 - Hiệu ứng Quantum bắt đầu xảy ra
 - Giảm độ di chuyển điện tích ~10%
 - Truyền dẫn ngược bắt đầu đáng kể
 - Tăng dòng khoảng ~20%



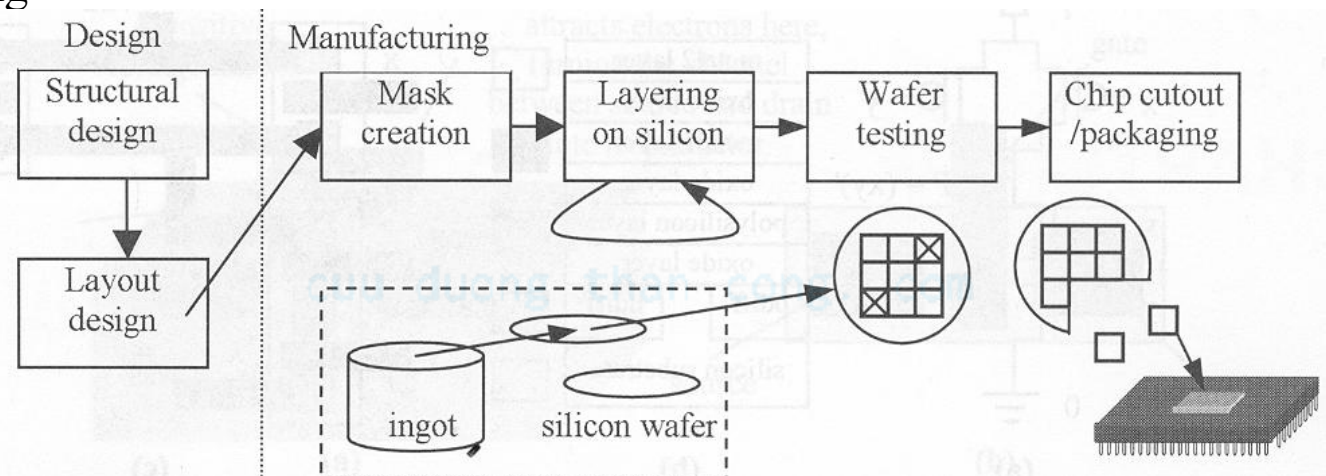
NAND

- Các lớp kim loại cho việc định tuyến (~10)
- PMOS không phù hợp với mức 0
- NMOS không phù hợp với mức 1
- Một sơ đồ cơ bản cho việc hình thành cổng



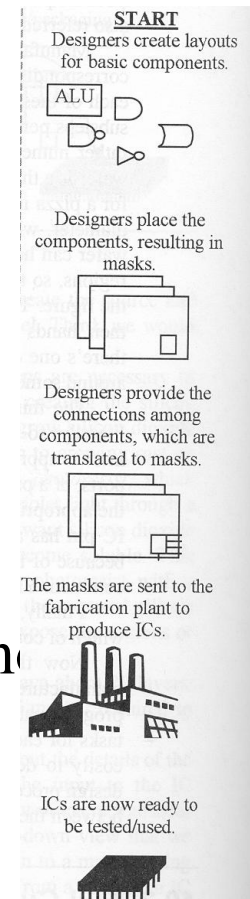
Các bước sản xuất Silicon

- Tape out
 - Gửi thiết kế đến khu vực sản xuất
- Quay
 - Thực hiện một lần trong quá trình sản xuất
- Quang khắc
 - Vẽ mẫu bằng cách sử dụng căn quang để hình thành rào chắn cho quá trình lắng đọng



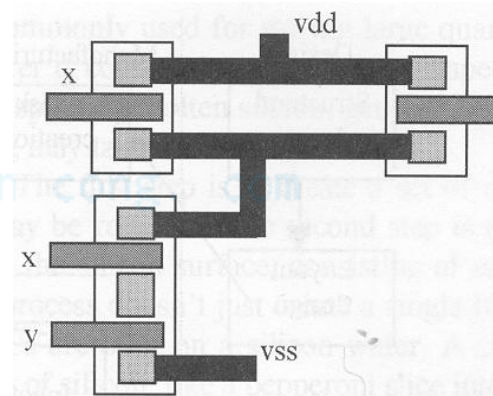
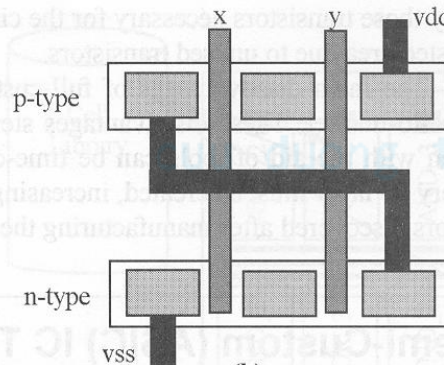
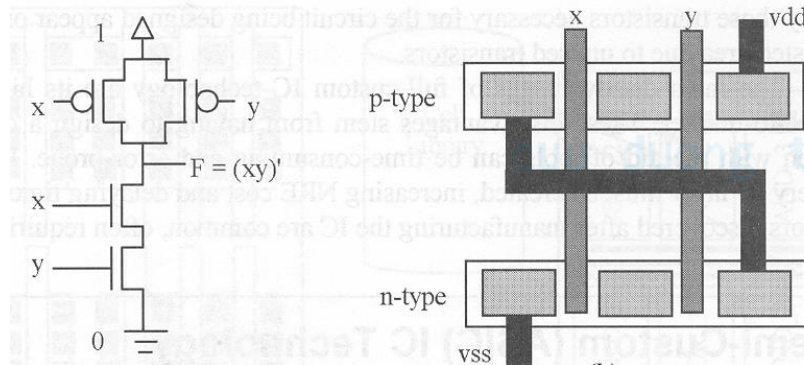
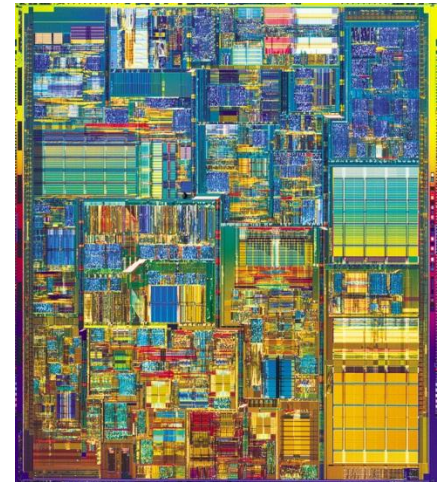
Chuyên dụng đầy đủ

- Mạch tích hợp mật độ rất lớn (VLSI)
- Bố trí
 - Đặt và định hướng transistors
- Kết nối
 - Kết nối transistors
- Điều chỉnh kích thước
 - Tạo ra các dây nối kích thước dày hoặc mỏng, nhanh hay chậm
 - Cũng có thể cần thay đổi kích thước bộ đệm



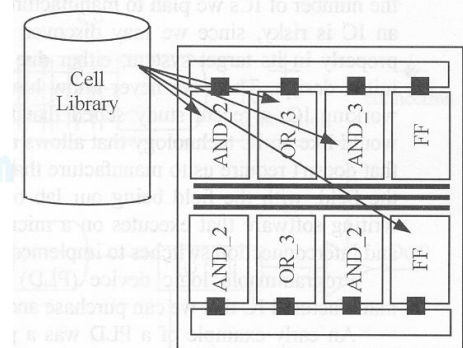
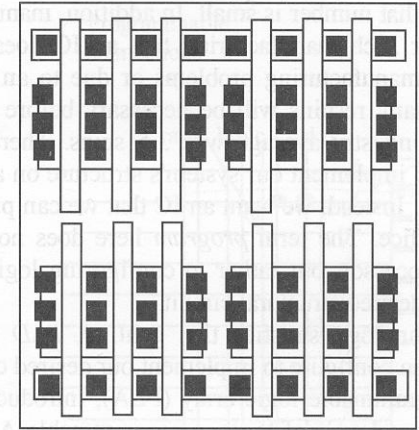
Chuyên dụng đầy đủ

- Kích thước, công suất và chất lượng tốt nhất
- Thiết kế thủ công
 - Tốn nhiều thời gian/độ linh hoạt cao/giá NRE cao...
 - Dành cho các bộ phận quan trọng nhất trong bộ xử lý
 - ALU, đọc mã lệnh...
- Công cụ thiết kế vật lý
 - Không tối ưu, nhưng nhanh hơn...



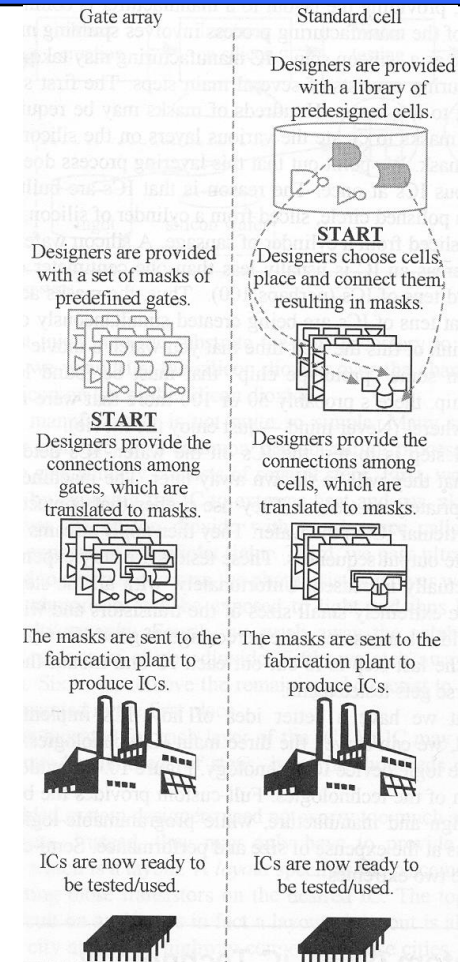
Bán chuyên dụng

- Mảng các cổng logic
 - Gồm một mảng các cổng được chế tạo sẵn
 - “bố trí” và kết nối
 - Mật độ cao hơn, thời gian đưa ra thị trường nhanh hơn
 - Không tích hợp cao như các vi mạch chức năng chuyên dụng đầy đủ
- Các “Ô” tiêu chuẩn
 - Một thư viện các ô được thiết kế trước
 - Bố trí và kết nối
 - Mật độ thấp hơn, độ phức tạp cao hơn
 - Tích hợp tốt nhất với chuyên dụng đầy đủ



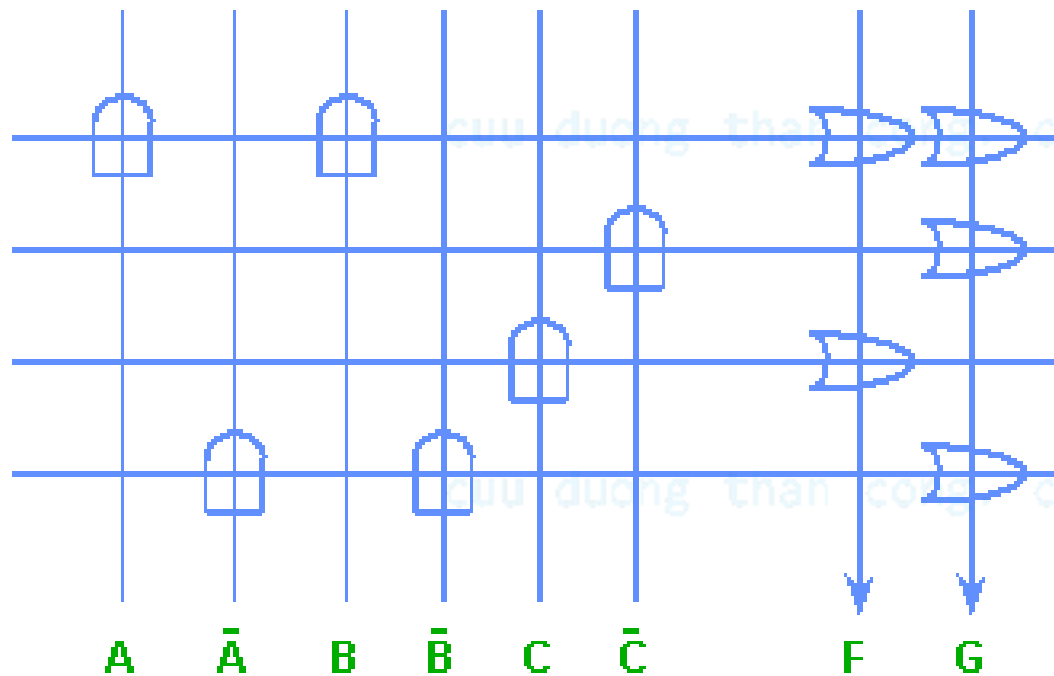
Bán chuyên dụng

- Kiểu thiết kế phổ biến nhất
- Phù hợp với các loại
 - Tốt
 - Công suất, thời gian đưa ra thị trường, chất lượng, giá NRE, giá đơn chiếc...
- Tích hợp
 - Tích hợp với thiết bị chuyên dụng đầy đủ cho các vùng thiết kế quan trọng



Programmable Logic Array (PLA)

*PLA exploits
structure of
expression.*



$$F = AB + C$$

$$G = AB + \bar{C} + \bar{A}\bar{B}$$

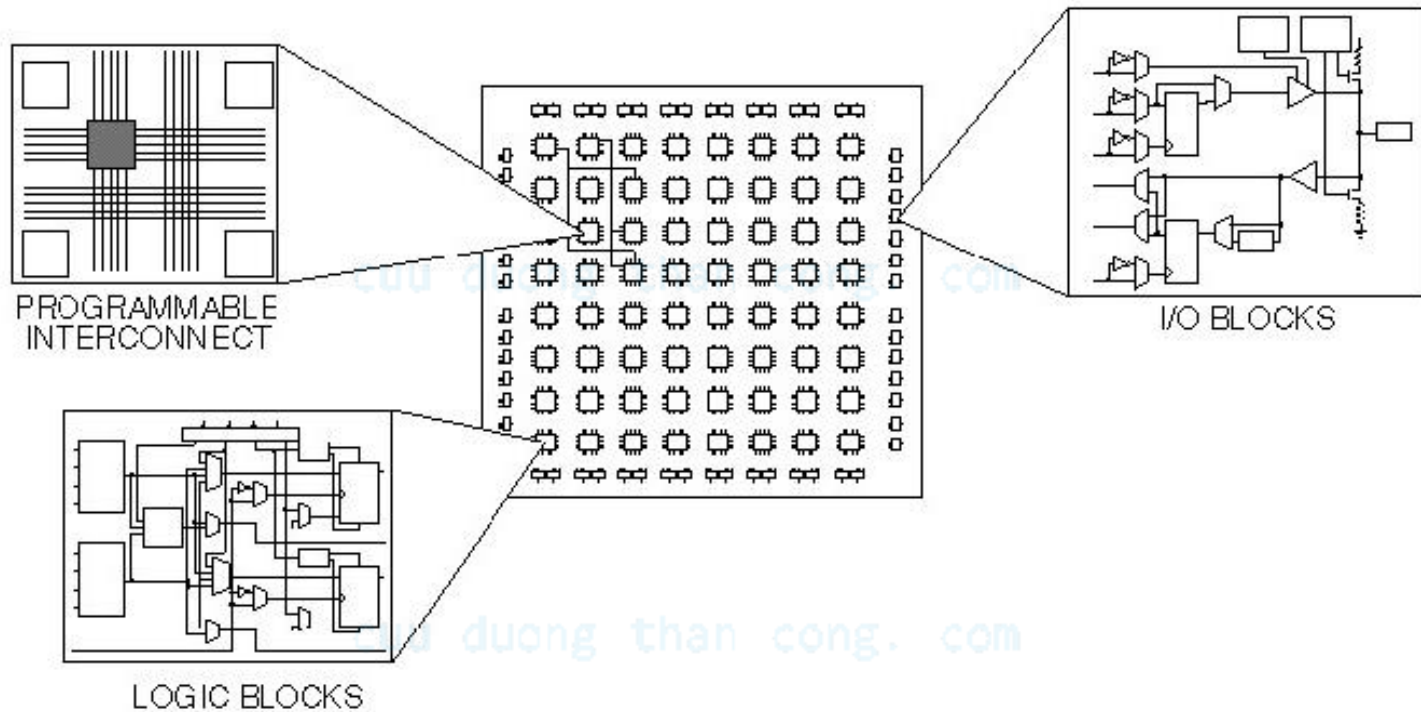
PLD

- Thiết bị logic khả trình
 - Programmable Logic Array (PLA), Programmable Array Logic (PAL), Field Programmable Gate Array (FPGA)
- Tất cả các lớp đã có sẵn
 - Người thiết kế có thể mua một IC
 - Để thực hiện chức năng mong muốn
 - Các kết nối trên IC được tạo ra hoặc hủy để thực hiện chức năng
- Lợi ích
 - Giá NRE rất thấp
 - Thời gian đưa ra thị trường ngắn
- Hạn chế
 - Giá cao, không tốt cho sản xuất hàng loạt
 - Công suất
 - Ngoại trừ PLA loại đặc biệt
 - Chậm hơn



1600 usable gate, 7.5 ns
\$7 list price

Xilinx FPGA



Khối logic có thể cấu hình được (CLB)

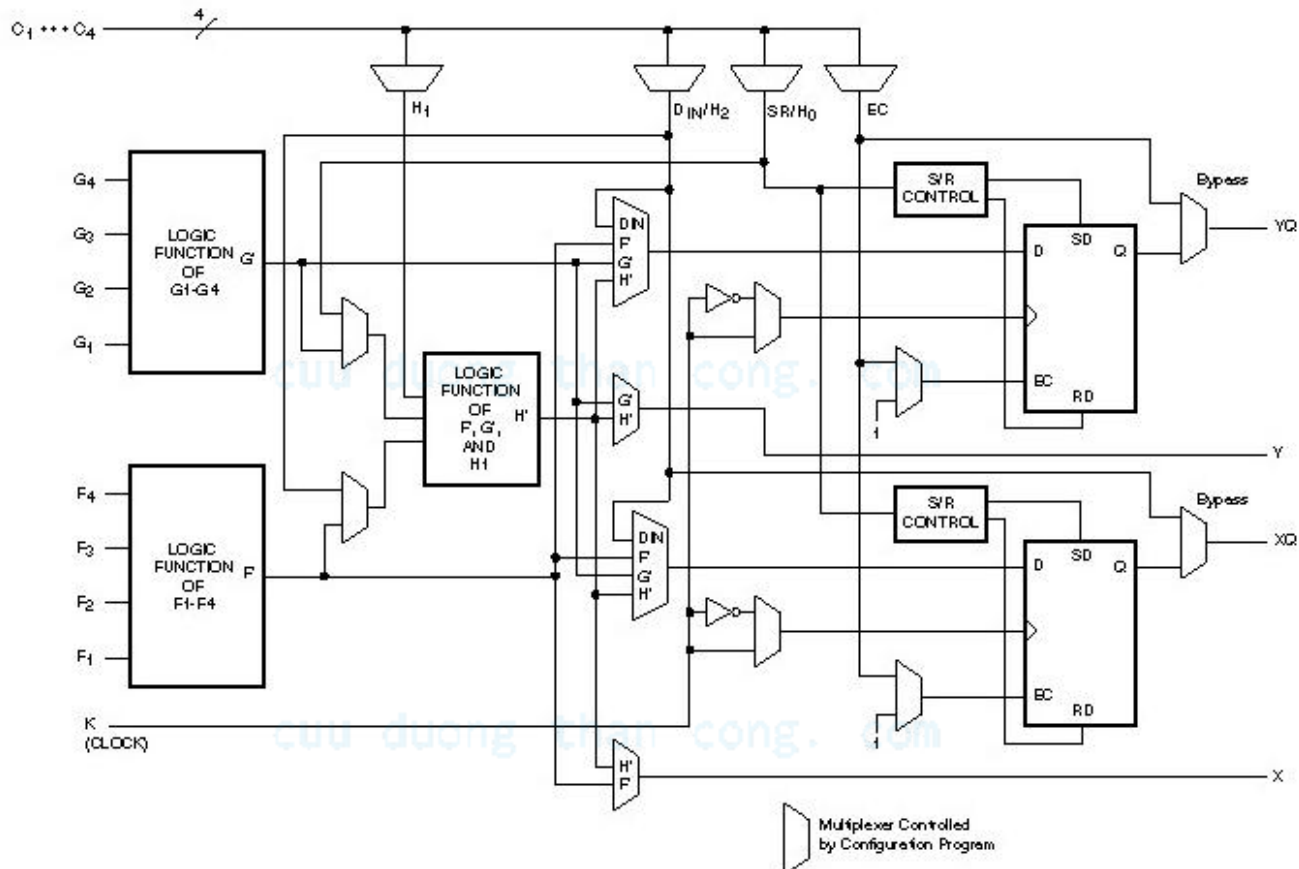
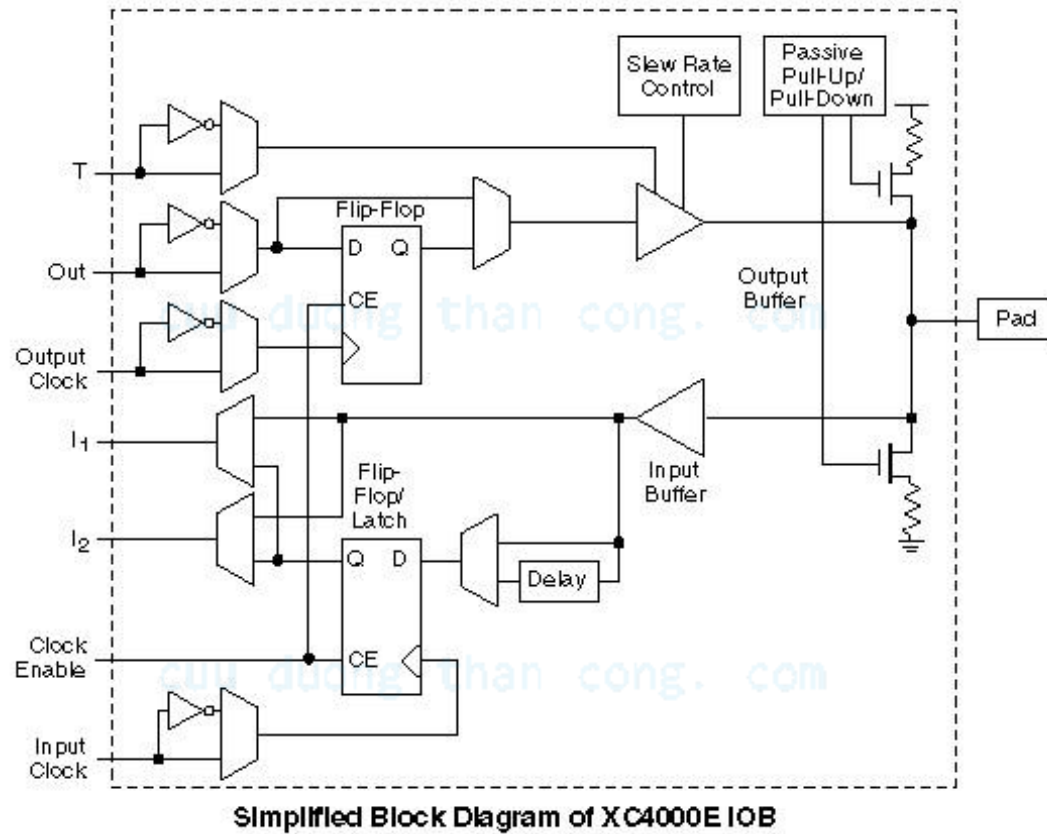


Figure 1: Simplified Block Diagram of XC4000-Series CLB (RAM and Carry Logic functions not shown)

Khối I/O



CHƯƠNG 4: KỸ THUẬT LẬP TRÌNH NHÚNG

Bài 8: Biểu diễn trạng thái và mô hình hóa quá trình

cuu duong than cong. com

cuu duong than cong. com

Tổng quan

- Mô hình vs Ngôn ngữ
 - Mô hình trạng thái
 - FSM/FSMD
 - HCFSM và ngôn ngữ biểu đồ
 - Mô hình trạng thái lập trình (Program-State Machine (PSM) Model)
 - Mô hình quá trình đồng thời
 - Truyền thông
 - Đồng bộ
 - Thực hiện
 - Mô hình luồng dữ liệu
 - Các hệ thời gian thực
-

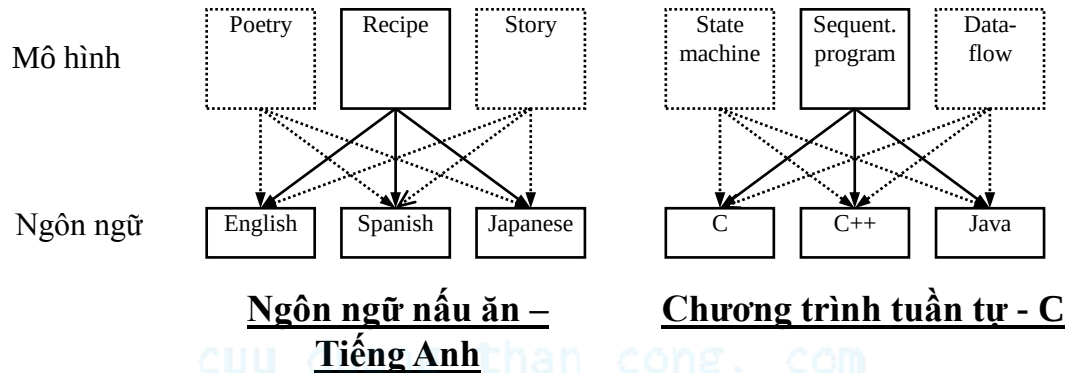
Giới thiệu

- Mô tả trạng thái xử lý của hệ thống nhúng
 - Đôi khi là rất khó
 - Độ phức tạp tăng khi khả năng của IC tăng
 - Trong quá khứ: máy giặt, games etc.
 - Vài trăm dòng lệnh
 - Ngày nay: Đầu TV kỹ thuật số, điện thoại di động etc.
 - Vài trăm nghìn dòng lệnh
 - Trạng thái yêu cầu thường không được hiểu đầy đủ khi bắt đầu
 - Nhiều quá trình thực hiện lỗi do mô tả sự kiện thiếu, ko chính xác
 - Tiếng Anh (hoặc ngôn ngữ khác) – điểm khởi đầu chung
 - Khó mô tả chính xác hoặc đôi khi không thể
 - Ví dụ: Mã điều khiển cho một ô tô – dài hàng nghìn trang...

Mô hình và ngôn ngữ

- Làm thế nào chúng ta ghi nhận hành vi (chính xác)?
 - Chúng ta có thể nghĩ đến ngôn ngữ (C, C++), nhưng mô hình tính toán là mấu chốt
- Mô hình tính toán cơ bản:
 - Mô hình lập trình tuần tự
 - Các câu lệnh, quy tắc ghép câu lệnh, cơ chế thực hiện chúng
 - Mô hình xử lý thông tin
 - Nhiều mô hình tuần tự chạy đồng thời
 - Mô hình trạng thái
 - Cho các hệ riêng, giám sát đầu vào điều khiển, thiết lập đầu ra điều khiển
 - Mô hình luồng dữ liệu
 - Cho các hệ dữ liệu riêng, biến dòng dữ liệu đầu vào thành dòng dữ liệu đầu ra
 - Mô hình hướng đối tượng
 - Để tách phần mềm phức tạp thành đơn giản, các mục được định nghĩa

Mô hình vs ngôn ngữ

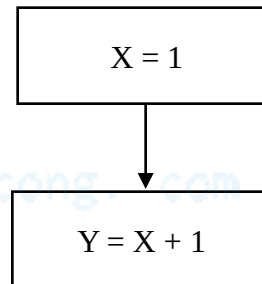


- Mô hình tính toán mô tả trạng thái của hệ
 - Ghi chú khái niệm, vd công thức hay chương trình tuần tự
- Ngôn ngữ để thể hiện mô hình
 - Dạng duy nhất, ví dụ tiếng Anh, C
- Hiểu ngôn ngữ được dùng để thể hiện một mô hình
 - VD mô hình lập trình tuần tự → C, C++, Java
- Một ngôn ngữ có thể thể hiện nhiều mô hình
 - VD C++ → mô hình lập trình tuần tự, mô hình hướng đối tượng, mô hình trạng thái
- Các ngôn ngữ nhất định thể hiện tốt các mô hình tính toán nhất định

Chữ vs Đồ họa

- Mô hình và ngôn ngữ không được nhầm lẫn với “chữ và đồ họa”
 - “Chữ và đồ họa” chỉ là hai kiểu ngôn ngữ
 - Chữ: ký tự, số
 - Đồ họa: vòng tròn, mũi tên (với một số ký tự, số)

$X = 1;$
 $Y = X + 1;$



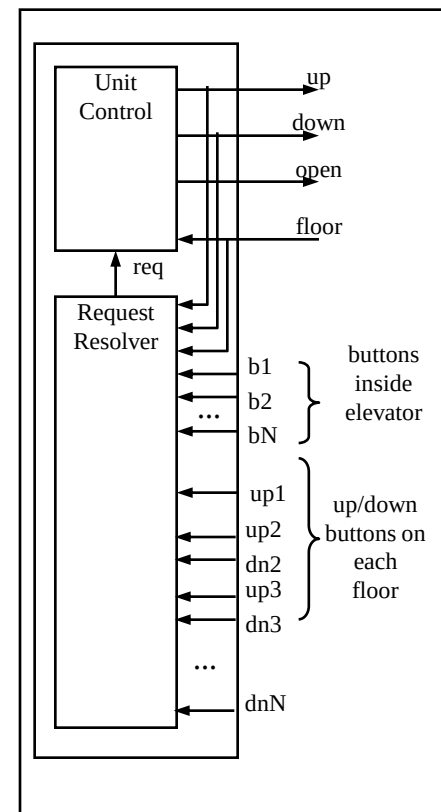
Ví dụ: Bộ điều khiển thang máy

- Bộ điều khiển thang máy đơn giản
 - *Bộ phận yêu cầu* chuyển các yêu cầu khác nhau thành yêu cầu của một tầng duy nhất
 - *Đơn vị điều khiển* di chuyển thang máy tới tầng yêu cầu
- Thử thể hiện bằng C...

Mô tả tiếng Anh một phần

“Di chuyển thang máy lên hoặc xuống để đến tầng yêu. Một khi ở tầng yêu cầu, mở cửa ít nhất 10 giây, và duy trì nó đến khi tầng được yêu cầu thay đổi. Đảm bảo cửa không bao giờ mở khi di chuyển. Không thay đổi hướng trừ khi có yêu cầu ở tầng cao hơn khi đi lên hoặc tầng thấp hơn khi đi xuống...”

Giao diện hệ thống



Bộ điều khiển thang máy sử dụng mô hình lập trình tuần tự

Mô hình chương trình tuần tự

```
Inputs: int floor; bit b1..bN; up1..upN-1; dn2..dnN;
Outputs: bit up, down, open;
Global variables: int req;

void UnitControl()
{
    up = down = 0; open = 1;
    while (1) {
        while (req == floor);
        open = 0;
        if (req > floor) { up = 1; }
        else { down = 1; }
        while (req != floor);
        up = down = 0;
        open = 1;
        delay(10);
    }
}

void RequestResolver()
{
    while (1)
        ...
        req = ...
        ...
}

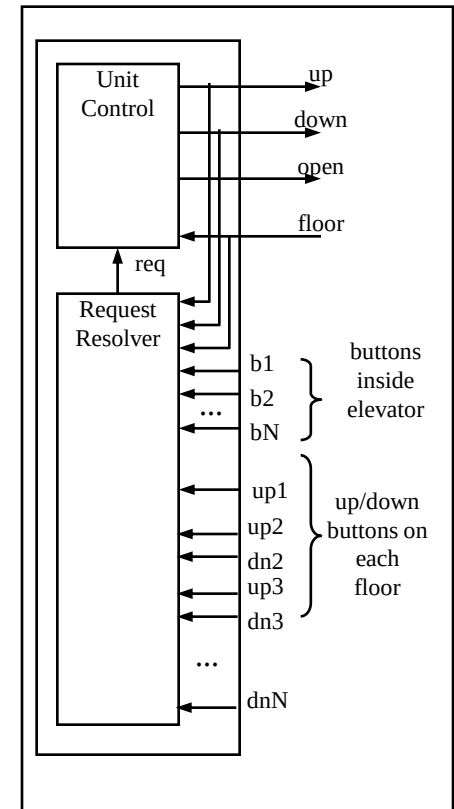
void main()
{
    Call concurrently:
        UnitControl() and
        RequestResolver()
}
```

Có thể thực hiện chương trình với nhiều câu lệnh “if” hơn.

Mô tả tiếng Anh một phần

“Di chuyển thang máy lên hoặc xuống để đến tầng yêu cầu. Một khi ở tầng yêu cầu, mở cửa ít nhất 10 giây, và duy trì nó đến khi tầng được yêu cầu thay đổi. Đảm bảo cửa không bao giờ mở khi di chuyển. Không thay đổi hướng đi lên hoặc xuống khi đi xuống...”

Giao diện hệ thống

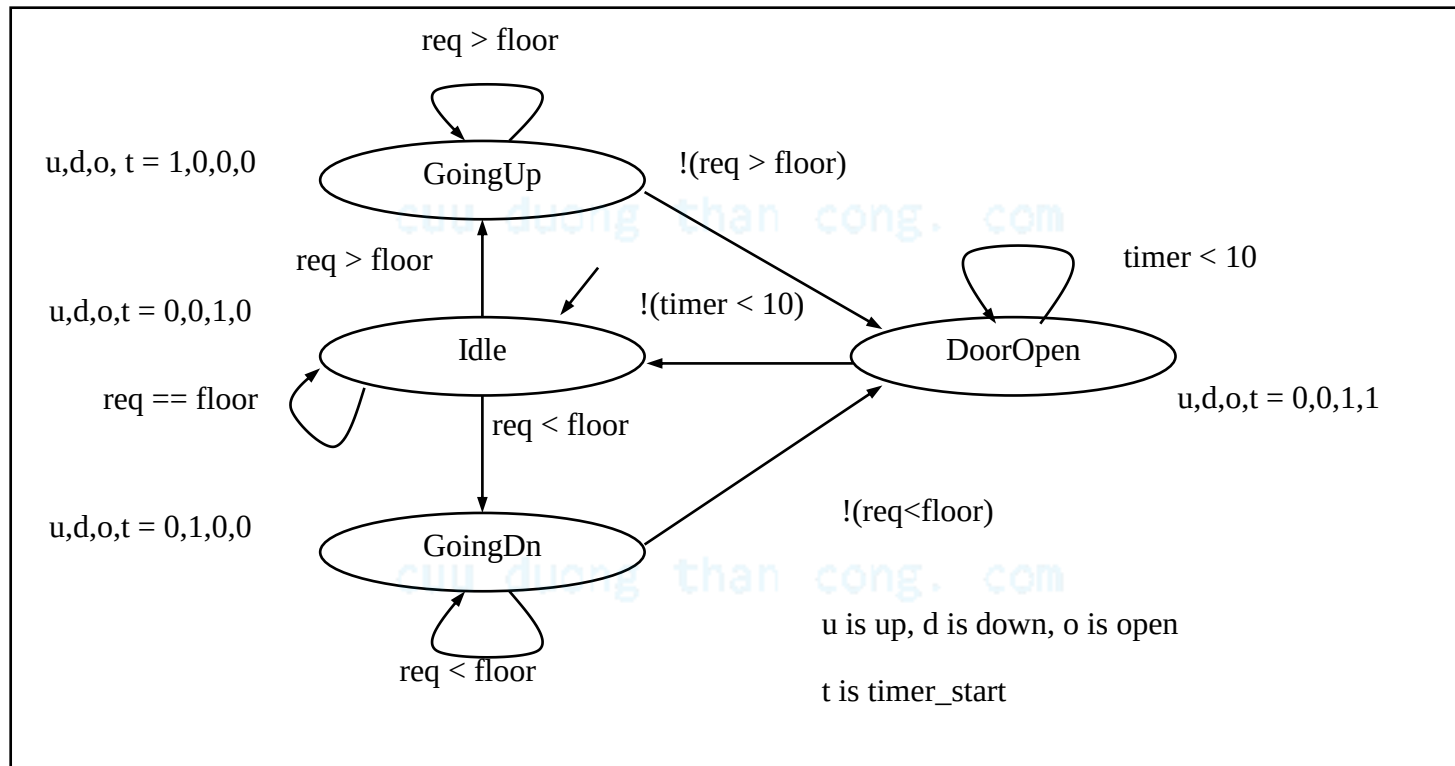


Mô hình trạng thái máy hữu hạn (Finite-state machine model – FSM)

- Cố gắng thể hiện trạng thái này như một chương trình tuần tự là đôi khi không đầy đủ hoặc khó
- Thay vào đó, chúng ta có thể xem xét như một mô hình FSM, mô tả hệ như sau:
 - Các trạng thái có thể
 - VD, nghỉ, đi lên, đi xuống, mở cửa
 - Chuyển đổi có thể từ trạng thái này đến trạng thái khác dựa trên các đầu vào
 - VD yêu cầu $\rightarrow$ tầng
 - Các hoạt động xảy ra trong mỗi trạng thái
 - VD trong trạng thái đi lên, $u,d,o,t = 1,0,0,0$ (up = 1, down, open, and timer_start = 0)
- Thử...

Mô hình trạng thái máy hữu hạn (FSM)

Quá trình của đơn vị điều khiển sử dụng máy trạng thái



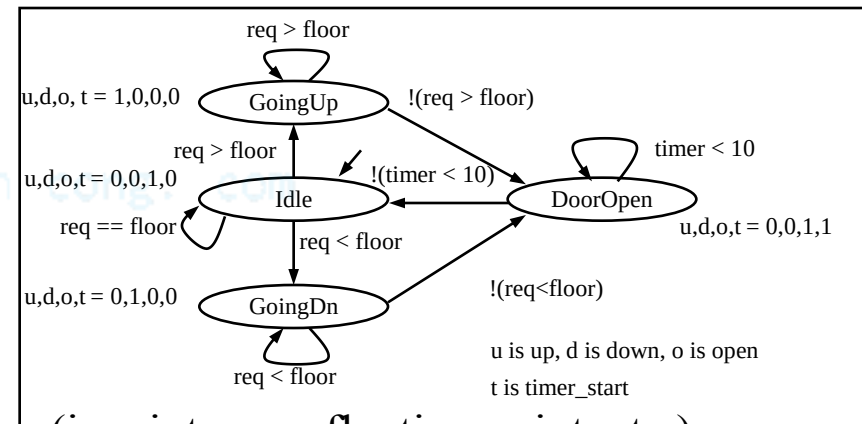
Định nghĩa chuẩn²

- Một FSM gồm 6-phần tử $F = \langle S, I, O, F, H, s_0 \rangle$
 - S là tập tất cả các trạng thái $\{s_0, s_1, \dots, s_l\}$
 - I là tập đầu vào $\{i_0, i_1, \dots, i_m\}$
 - O là tập đầu ra $\{o_0, o_1, \dots, o_n\}$
 - F là hàm trạng thái tiếp theo ($S \times I \rightarrow S$)
 - H là hàm đầu ra ($S \rightarrow O$)
 - s_0 là trạng thái đầu
- Kiểu Moore
 - Liên kết các đầu vào với các trạng thái (như trên, H liên kết $S \rightarrow O$)
- Kiểu Mealy
 - Liên kết đầu ra với chuyển trạng thái (H liên kết $S \times I \rightarrow O$)
- Viết tắt để đơn giản hóa các mô tả
 - Gán 0 cho tất cả các đầu ra, không gán giá trị trong một trạng thái
 - AND tất cả các điều kiện chuyển với xung đồng hồ (FSM là quá trình đồng bộ)

Trạng thái máy hữu hạn với mô hình tuyến dữ liệu (FSMD)

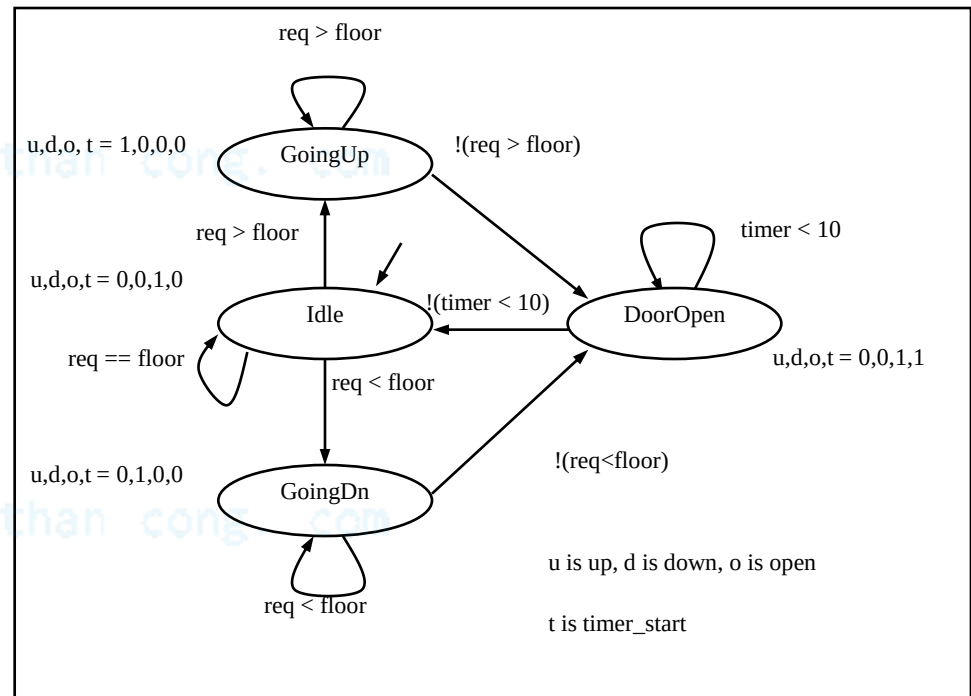
- FSMD mở rộng FSM: các kiểu dữ liệu và biến phức tạp để lưu trữ dữ liệu
 - FSMs chỉ sử dụng kiểu dữ liệu và toán hạng Boolean, không có biến
- FSMD: 7-phần tử $\langle S, I, O, \underline{V}, F, H, s_0 \rangle$
 - S là tập các trạng thái $\{s_0, s_1, \dots, s_l\}$
 - I là tập các đầu vào $\{i_0, i_1, \dots, i_m\}$
 - O là tập các đầu ra $\{o_0, o_1, \dots, o_n\}$
 - $\underline{V}$ là tập các biến $\{v_0, v_1, \dots, v_n\}$**
 - F là hàm của trạng thái tiếp ($S \times I \times V \rightarrow S$)
 - H là một hàm **tác động** ($S \rightarrow O + V$)
 - s_0 là trạng thái đầu
- I, O, V có thể diễn tả các kiểu dữ liệu phức tạp (i.e., integers, floating point, etc.)
- F, H có thể bao gồm các toán hạng
- H là một hàm tác động, không chỉ là một hàm ra
 - Mô tả các biến cập nhật cũng như đầu ra
- Trạng thái hoàn thiện của hệ bao gồm trạng thái hiện tại, s_i , và các giá trị của tất cả các biến

We described UnitControl as an FSMD



Mô tả hệ theo trạng thái máy

1. Liệt kê tất cả các trạng thái có thể
2. Khai báo tất cả các biến
3. Với mỗi trạng thái, liệt kê các chuyển trạng thái có thể, với các điều kiện, sang các trạng thái khác
4. Với mỗi trạng thái/chuyển, liệt kê các hoạt động liên quan
5. Với mỗi trạng thái, đảm bảo loại trừ và các điều kiện chuyển đã có
 - Không tồn tại hai điều kiện đúng tại một thời điểm
 - Nếu không sẽ trở thành trạng thái máy không xác định
 - Một điều kiện phải đúng tại mọi thời điểm



Trạng thái máy vs. Mô hình lập trình tuần tự

- Trạng thái máy:
 - Khuyến khích người thiết kế nghĩ đến tất cả các trạng thái có thể và chuyển trạng thái dựa trên tất cả các điều kiện đầu vào có thể
- Mô hình lập trình tuần tự:
 - Được thiết kế để chuyển dữ liệu thông qua chuỗi các lệnh mà có thể lặp lại hoặc thực hiện có điều kiện

Thử mô tả các hành vi khác với một mô hình FSM

- VD: Máy trả lời nhấp nháy đèn khi có bản tin
- VD: Một máy trả lời điện thoại đơn giản mà trả lời sau 4 hồi chuông
- VD: Một hệ thống đèn giao thông đơn giản
- Nhiều ví dụ khác

cuu duong than cong. com

Mô tả trạng thái máy trong ngôn ngữ lập trình tuần tự

- Mặc dù mô hình trạng thái máy có nhiều lợi ích, hầu hết các công cụ phát triển phổ biến sử dụng ngôn ngữ lập trình tuần tự
 - C, C++, Java, Ada, VHDL, Verilog HDL, vv....
 - Công cụ phát triển đắt và phức tạp, bởi vậy không dễ để thích nghi hay thay đổi
 - Phải đầu tư
- Hai phương pháp để mô tả mô hình trạng thái máy với ngôn ngữ lập trình tuần tự
 - Phương pháp công cụ hỗ trợ
 - Công cụ bổ sung được cài đặt để hỗ trợ ngôn ngữ trạng thái máy
 - Đồ họa
 - Có thể hỗ trợ mô phỏng đồ họa
 - Tự động tạo ra code trong ngôn ngữ lập trình tuần tự là đầu vào cho các công cụ phát triển chính
 - Hạn chế: phải hỗ trợ các công cụ bổ sung (giá bản quyền, nâng cấp, đào tạo, vv.)
 - Phương pháp ngôn ngữ tập con
 - Phương pháp thông dụng nhất...

Phương pháp ngôn ngữ tập con

- Tuân theo các quy tắc (mẫu) để mô tả cấu trúc trạng thái máy trong cấu trúc ngôn ngữ tuần tự tương đương
- Được sử dụng với phần mềm (VD: C) và ngôn ngữ phần cứng (VD: VHDL)
- Mô tả trạng thái máy *UnitControl* bằng C
 - Liệt kê các trạng thái (#define)
 - Khai báo các biến trạng thái, khởi tạo giá trị đầu (IDLE)
 - Câu lệnh chuyển mạch đơn rẽ nhánh tới trạng thái hiện tại
 - Mỗi trường hợp có các hoạt động
 - up, down, open, timer_start
 - Mỗi trường hợp kiểm tra điều kiện chuyển để xác định trạng thái tiếp theo
 - if(...) {state = ...;}

```
#define IDLE0
#define GOINGUP1
#define GOINGDN2
#define DOOROPEN3
void UnitControl() {
    int state = IDLE;
    while (1) {
        switch (state) {
            IDLE: up=0; down=0; open=1; timer_start=0;
                if (req==floor) {state = IDLE;}
                if (req > floor) {state = GOINGUP;}
                if (req < floor) {state = GOINGDN;}
                break;
            GOINGUP: up=1; down=0; open=0; timer_start=0;
                if (req > floor) {state = GOINGUP;}
                if (!(req>floor)) {state = DOOROPEN;}
                break;
            GOINGDN: up=1; down=0; open=0; timer_start=0;
                if (req < floor) {state = GOINGDN;}
                if (!(req<floor)) {state = DOOROPEN;}
                break;
            DOOROPEN: up=0; down=0; open=1; timer_start=1;
                if (timer < 10) {state = DOOROPEN;}
                if (!(timer<10)){state = IDLE;}
                break;
        }
    }
}
```

Trạng thái máy *UnitControl* trong ngôn ngữ lập trình tuần tự

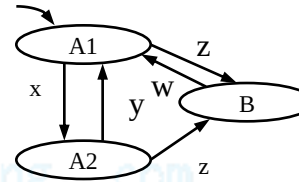
Mẫu chung

```
#define S0  0
#define S1  1
...
#define SN  N
void StateMachine() {
    int state = S0; // or whatever is the initial state.
    while (1) {
        switch (state) {
            S0:
                // Insert S0's actions here & Insert transitions  $T_i$  leaving S0:
                if(  $T_0$ 's condition is true ) {state =  $T_0$ 's next state; /*actions*/ }
                if(  $T_1$ 's condition is true ) {state =  $T_1$ 's next state; /*actions*/ }
                ...
                if(  $T_m$ 's condition is true ) {state =  $T_m$ 's next state; /*actions*/ }
                break;
            S1:
                // Insert S1's actions here
                // Insert transitions  $T_i$  leaving S1
                break;
            ...
            SN:
                // Insert SN's actions here
                // Insert transitions  $T_i$  leaving SN
                break;
        }
    }
}
```

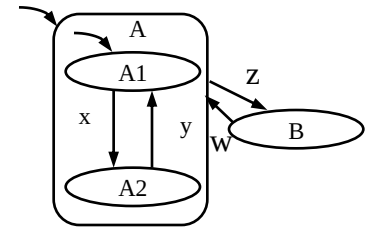
HCFSM và ngôn ngữ biểu đồ trạng thái

- Mô hình trạng thái máy phân cấp/đồng thời (Hierarchical/concurrent state machine model - HCFSM)
 - Mở rộng mô hình trạng thái máy để hỗ trợ phân cấp và đồng thời
 - Các trạng thái có thể tách thành các trạng thái máy khác
 - Các trạng thái có thể thực hiện đồng thời
- Biểu đồ trạng thái
 - Ngôn ngữ đồ họa để mô tả HCFSM
 - *timeout*: chuyển trạng thái với giới hạn thời gian như là một điều kiện

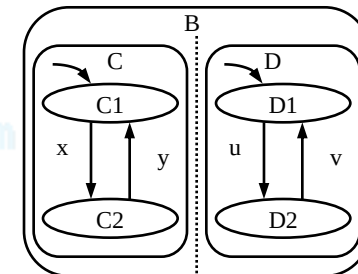
Without hierarchy



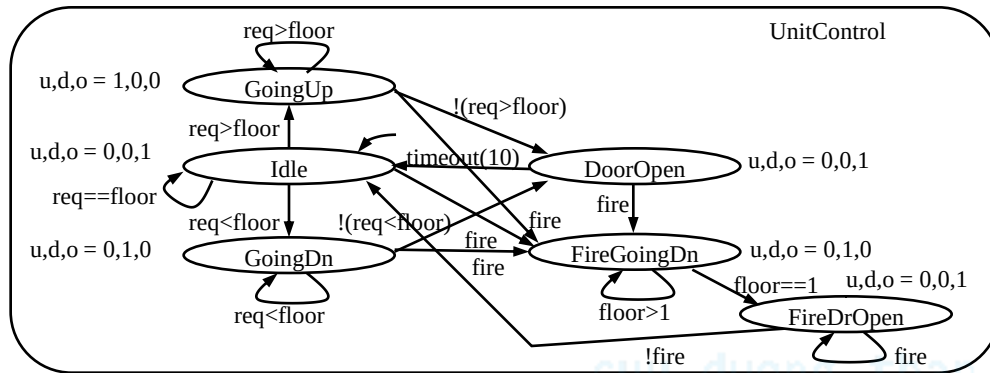
With hierarchy



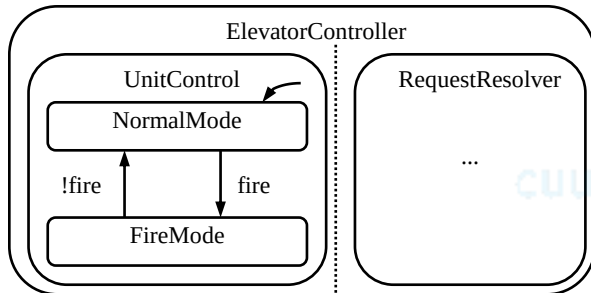
Concurrency



UnitControl với FireMode



Không phân cấp

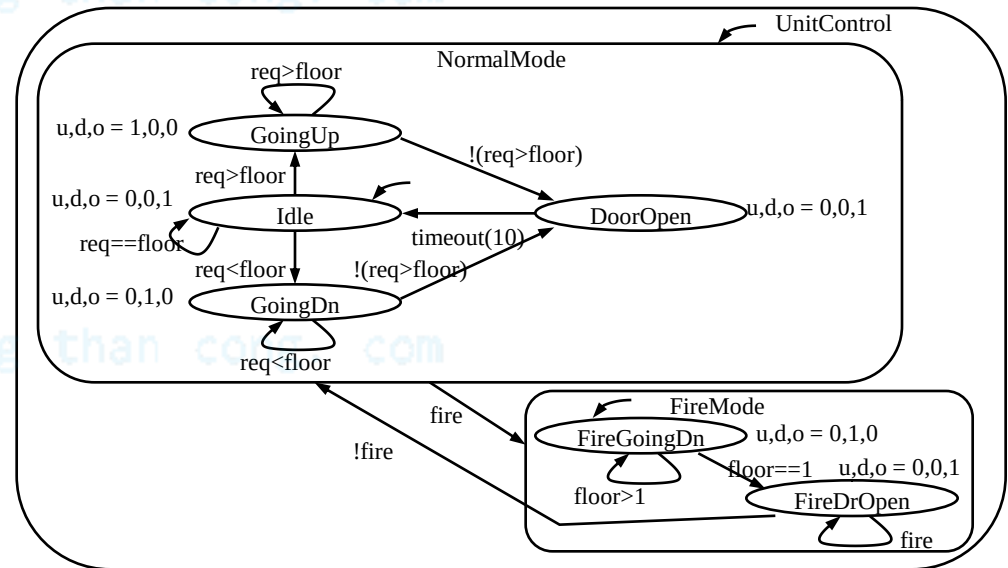


Xử lý đồng thời với RequestResolver

• FireMode

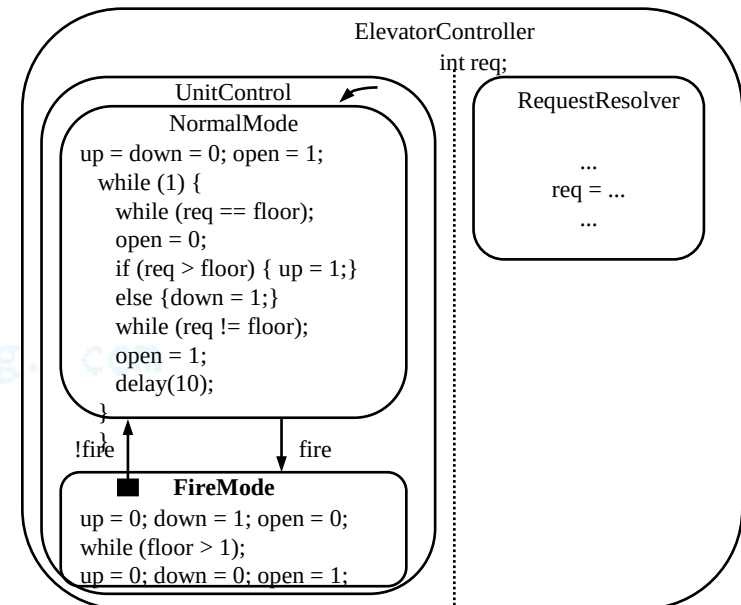
- Khi *fire* đúng, di chuyển thang máy tới tầng 1st và mở cửa
- w/o hierarchy: Getting messy!
- w/ hierarchy: Simple!

Có phân cấp



Mô hình trạng thái máy – chương trình (PSM): HCFSM + mô hình lập trình tuần tự

- Các hoạt động của trạng thái chương trình có thể là FSM hoặc chương trình tuần tự
 - Người thiết kế chọn kiểu thích hợp nhất
- Phân cấp hạn chế hơn HCFSM sử dụng trong biểu đồ trạng thái
 - Chỉ chuyển trạng thái giữa các trạng thái cận kề, đầu vào đơn
 - Trạng thái – chương trình có thể “hoàn thiện”
 - Đạt đến cuối của chương trình tuần tự, hoặc
 - FSM chuyển tới trạng thái con hoàn thiện
 - PSM có hai kiểu chuyển
 - Chuyển trực tiếp (TI): xảy ra bất kể trạng thái của chương trình nguồn
 - Chuyển khi hoàn thành (TOC): xảy ra nếu điều kiện đúng và chương trình nguồn kết thúc
 - Biểu đồ đặc biệt: mở rộng của VHDL để mô tả mô hình PSM
 - C đặc biệt: mở rộng của C để mô tả mô hình PSM



Vai trò của việc chọn ngôn ngữ và mô hình thích hợp

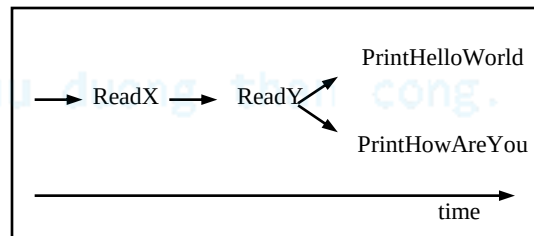
- Tìm mô hình thích hợp để biểu diễn hệ nhúng là một bước quan trọng
 - Mô hình sẽ ảnh hưởng đến cách chúng ta nhìn hệ thống
 - Ban đầu chúng ta nghĩ về chuỗi các hoạt động, viết chương trình tuần tự
 - Trước tiên đợi lệnh gọi tầng
 - Sau đó, đóng cửa
 - Sau đó, di chuyển lên hay xuống đến tầng yêu cầu
 - Rồi mở cửa
 - Rồi lặp lại tuần tự này
 - Để tạo ra trạng thái máy, chúng ta nghĩ theo khía cạnh các trạng thái và sự chuyển đổi giữa chúng
 - Khi hệ phải phản ứng lại với các đầu vào thay đổi, trạng thái máy là mô hình tốt nhất
 - HCFSM mô tả *FireMode* một cách dễ dàng và rõ ràng
- Ngôn ngữ nên mô tả mô hình dễ dàng
 - Về lý tưởng, nên có các đặc điểm mô tả trực tiếp cấu trúc của mô hình
 - *FireMode* sẽ rất phức tạp trong chương trình tuần tự
 - Xem lại code
 - Các yếu tố khác có thể ảnh hưởng đến việc chọn lựa mô hình
 - Các kỹ thuật cấu trúc có thể sử dụng để thay thế
 - VD: Các nhãn để mô tả trạng thái máy trong chương trình tuần tự

Mô hình quá trình đồng thời

```
ConcurrentProcessExample() {  
  x = ReadX()  
  y = ReadY()  
  Call concurrently:  
    PrintHelloWorld(x) and  
    PrintHowAreYou(y)  
}  
PrintHelloWorld(x) {  
  while( 1 ) {  
    print "Hello world."  
    delay(x);  
  }  
}  
PrintHowAreYou(x) {  
  while( 1 ) {  
    print "How are you?"  
    delay(y);  
  }  
}
```

Ví dụ quá trình đồng thời đơn giản

- Mô tả chức năng của hệ theo khía cạnh của hai hoặc nhiều tác vụ thực hiện đồng thời
- Nhiều hệ thống dễ hơn để mô tả với mô hình quá trình đồng thời bởi vì tính chất đa tác vụ của nó
- Ví dụ đơn giản:
 - Đọc hai số X và Y
 - Hiện thị “Hello world.” sau mỗi X giây
 - Hiện thị “How are you?” sau mỗi Y giây



Chương trình thực hiện

```
Enter X: 1  
Enter Y: 2  
Hello world. (Time = 1 s)  
Hello world. (Time = 2 s)  
How are you? (Time = 2 s)  
Hello world. (Time = 3 s)  
How are you? (Time = 4 s)  
Hello world. (Time = 4 s)  
...
```

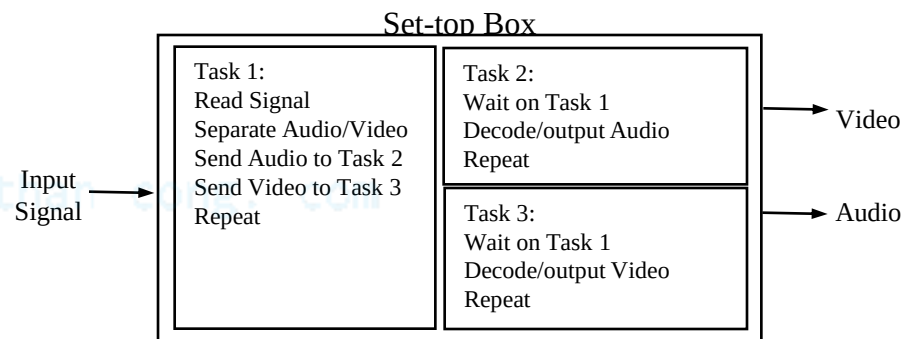
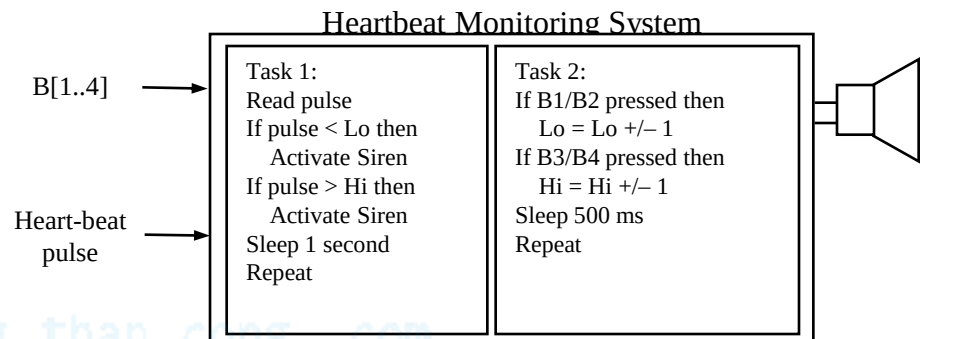
Đầu vào và đầu ra mẫu

Quá trình đồng thời và các hệ thời gian thực

cuu duong than cong. com

Quá trình đồng thời

- Xem xét hai ví dụ có các tác vụ chạy độc lập nhưng chia sẻ dữ liệu
- Khó để viết sử dụng mô hình lập trình tuần tự
- Mô hình quá trình đồng thời dễ hơn
 - Các chương trình tuần tự riêng cho các tác vụ
 - Các chương trình trao đổi thông tin với nhau

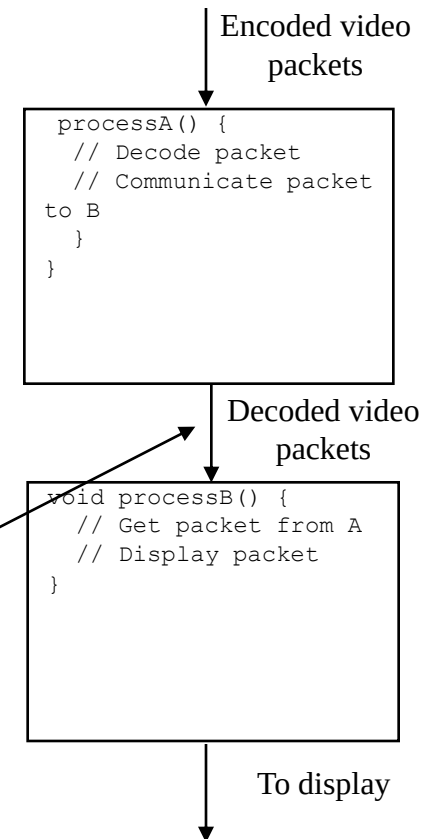


Quá trình

- Một chương trình tuần tự, thường là vòng lặp vô hạn
 - Thực hiện đồng thời với các quá trình khác
 - Chúng ta đang bước vào thế giới “lập trình đồng thời”
- Các hoạt động chính của quá trình
 - Khởi tạo và kết thúc
 - Khởi tạo giống một thủ tục gọi, nhưng bên gọi không đợi
 - Quá trình được khởi tạo có thể chính nó tạo ra các quá trình mới
 - Kết thúc chấm dứt một quá trình, loại bỏ các dữ liệu
 - Trong ví dụ HelloWorld/HowAreYou, chúng ta chỉ khởi tạo quá trình
 - Dừng và phục hồi
 - Dừng là đưa quá trình vào trạng thái giữ, lưu trữ trạng thái cho việc thực hiện sau đó
 - Phục hồi khởi đầu lại quá trình từ điểm nó được dừng
 - Liên kết
 - Một quá trình dừng cho đến khi một quá trình con được kết thúc

Thông tin giữa các quá trình

- Các quá trình cần trao đổi dữ liệu và tín hiệu để giải quyết vấn đề tính toán của chúng
 - Các quá trình không thông tin là các chương trình độc lập giải quyết các vấn đề độc lập
- Ví dụ cơ bản: producer/consumer
 - Quá trình A tạo ra dữ liệu, quá trình B sử dụng chúng
 - VD: A giải mã các gói video, B hiển thị các gói được giải mã trên màn hình
- Làm thế nào để đạt được quá trình thông tin này?
 - Hai phương pháp cơ bản
 - Chia sẻ bộ nhớ
 - Chuyển tin



Chia sẻ bộ nhớ

- Các quá trình đọc và ghi các biến được chia sẻ
 - Không mất thời gian, dễ thực hiện
 - Nhưng hay bị lỗi
- Ví dụ: Producer/consumer với một lỗi
 - Chia sẻ *buffer[N]*, *count*
 - *count* = # số dữ liệu trong *buffer*
 - *processA* tạo dữ liệu và lưu trong *buffer*
 - Nếu *buffer* đầy, phải đợi
 - *processB* sử dụng dữ liệu trong *buffer*
 - Nếu *buffer* trống, phải đợi
 - Lỗi xảy ra khi cả hai quá trình cập nhật *count* đồng thời (dòng 10 và 19) và tuần tự thực hiện sau xảy ra. “count” là 3.
 - A nạp *count* (*count* = 3) từ bộ nhớ vào thanh ghi R1 (*R1* = 3)
 - A tăng *R1* (*R1* = 4)
 - B nạp *count* (*count* = 3) từ bộ nhớ vào thanh ghi *R2* (*R2* = 3)
 - B giảm *R2* (*R2* = 2)
 - A lưu *R1* trở lại *count* trong bộ nhớ (*count* = 4)
 - B lưu *R2* trở lại *count* trong bộ nhớ (*count* = 2)
 - *count* có giá trị không đúng là 2

```
01: data_type buffer[N];
02: int count = 0;
03: void processA() {
04:     int i;
05:     while( 1 ) {
06:         produce(&data);
07:         while( count == N ); /*loop*/
08:         buffer[i] = data;
09:         i = (i + 1) % N;
10:         count = count + 1;
11:     }
12: }
13: void processB() {
14:     int i;
15:     while( 1 ) {
16:         while( count == 0 ); /*loop*/
17:         data = buffer[i];
18:         i = (i + 1) % N;
19:         count = count - 1;
20:         consume(&data);
21:     }
22: }
23: void main() {
24:     create_process(processA);
25:     create_process(processB);
26: }
```

Chuyển tin

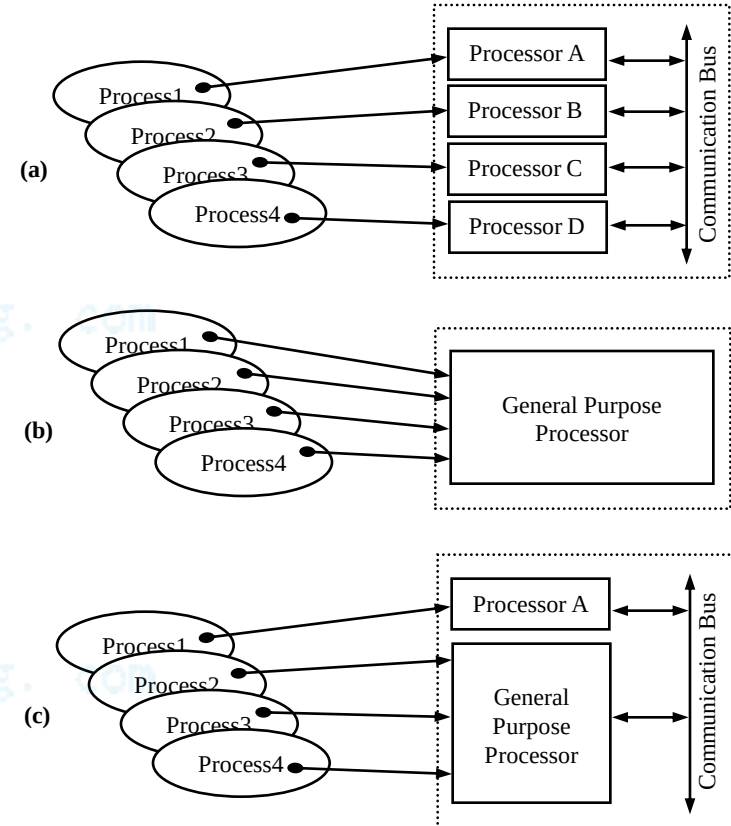
- Chuyển tin
 - Dữ liệu được gửi thẳng từ một quá trình tới quá trình khác
 - Quá trình gửi thực hiện hoạt động đặc biệt, *send*
 - Quá trình thu thực hiện hoạt động đặc biệt, *receive*, để thu dữ liệu
 - Cả hai hoạt động phải chỉ ra quá trình nào gửi hoặc nhận
 - Thu theo gói, gửi có thể hoặc không thể theo gói
 - Mô hình này an toàn hơn, nhưng kém linh hoạt

```
void processA() {  
    while( 1 ) {  
        produce(&data)  
        send(B, &data);  
        /* region 1 */  
        receive(B, &data);  
        consume(&data);  
    }  
}
```

```
void processB() {  
    while( 1 ) {  
        receive(A, &data);  
        transform(&data)  
        send(A, &data);  
        /* region 2 */  
    }  
}
```

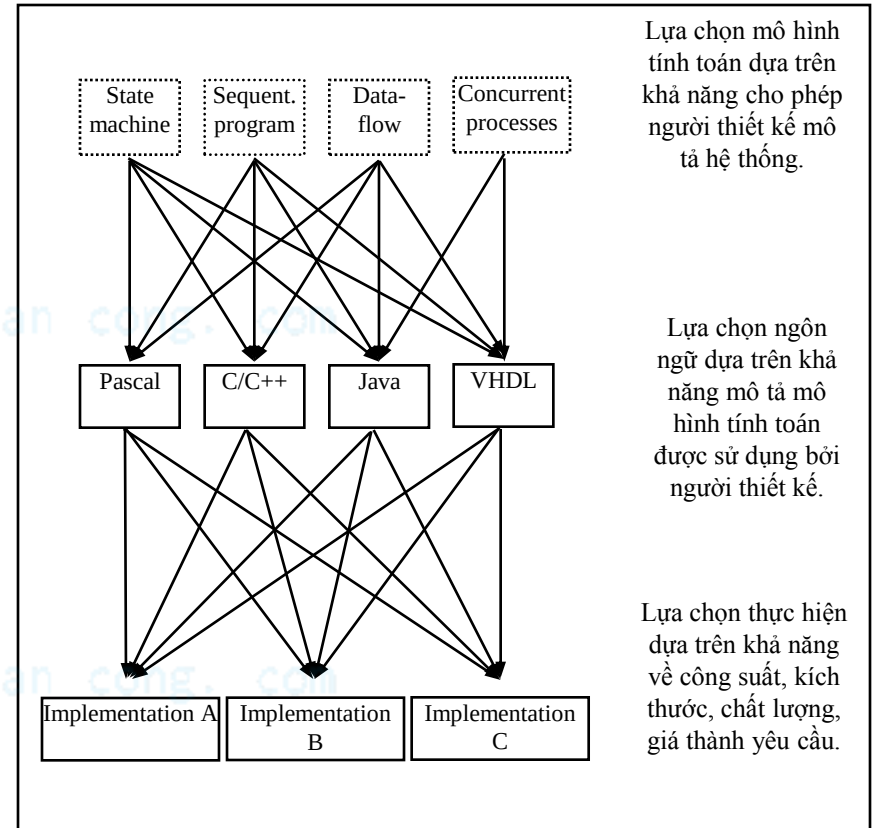
Mô hình quá trình đồng thời: Thực hiện

- Có thể sử dụng bộ xử lý chức năng đơn hay chung (SPP/GPP)
- (a) Nhiều bộ xử lý, mỗi bộ xử lý thực hiện một quá trình
 - Đa nhiệm đúng nghĩa (xử lý song song)
 - Bộ xử lý chức năng chung
 - Sử dụng ngôn ngữ lập trình như C và biên dịch thành các lệnh cho bộ xử lý
 - Đắt, trong nhiều trường hợp không cần thiết
 - Bộ xử lý chức năng đơn chuyên dụng
 - Hay dùng hơn
- (b) Một bộ xử lý chức năng chung chạy tất cả các quá trình
 - Đa số các quá trình không sử dụng 100% thời gian của bộ xử lý
 - Có thể chia sẻ thời gian của bộ xử lý và vẫn đạt được mục đích
- (c) Kết hợp (a) và (b)
 - Nhiều quá trình chạy trên một bộ xử lý chức năng chung trong khi đó một vài quá trình sử dụng bộ xử lý chức năng đơn chuyên dụng



Thực hiện

- Quá trình đưa các chức năng của hệ lên phần cứng của bộ xử lý:
 - Mô tả sử dụng mô hình tính toán(s)
 - Viết bằng một số ngôn ngữ(s)
- Lựa chọn thực hiện độc lập với lựa chọn ngôn ngữ
- Lựa chọn thực hiện dựa trên công suất, kích thước, chất lượng, thời gian và giá thành yêu cầu
- Thực hiện cuối cùng cần được kiểm tra tính khả thi
 - Là bản thử nghiệm cho việc sản xuất hàng loạt sản phẩm cuối cùng



Thực hiện:

Nhiều quá trình chia sẻ một bộ xử lý

- Có thể viết lại các quá trình như là một chương trình tuần tự
 - Thực hiện được với các trường hợp đơn giản, nhưng rất khó với các trường hợp phức tạp
 - Kỹ thuật tự động đã ra đời nhưng không thông dụng
 - VD: chương trình đồng thời Hello World có thể viết:
I = 1; T = 0;
while (1) {
 Delay(I); T = T + 1;
 if X modulo T is 0 then call PrintHelloWorld
 if Y modulo T is 0 then call PrintHowAreYou
}
- Có thể dùng hệ điều hành đa nhiệm
 - Thông dụng hơn
 - Hệ điều hành lập lịch cho các quá trình, định vị bộ nhớ, giao diện ngoại vi, etc.
 - Hệ điều hành thời gian thực (RTOS) có thể thực hiện điều đó
 - Mô tả các quá trình đồng thời với ngôn ngữ có các quá trình built-in (Java, Ada, etc.) hoặc một ngôn ngữ lập trình tuần tự với các thư viện hỗ trợ các quá trình (C, C++, etc)
- Có thể biến đổi các quá trình thành chương trình tuần tự với lập lịch trong code
 - Ít tốn bộ nhớ (không cần OS)
 - Chương trình phức tạp hơn

Tóm tắt

- Mô hình tính toán khác với ngôn ngữ
 - Mô hình lập trình tuần tự là phổ biến
 - Ngôn ngữ phổ thông nhất như là C
 - Mô hình trạng thái máy tốt cho điều khiển
 - Các mở rộng như HCFSM cung cấp thêm nhiều chức năng
 - PSM kết hợp trạng thái máy và chương trình tuần tự
 - Mô hình quá trình đồng thời sử dụng cho các hệ thống nhiều tác vụ
 - Tồn tại truyền thông và phương pháp đồng bộ
 - Lập lịch là rất quan trọng
 - Mô hình tuyến dữ liệu tốt cho xử lý tín hiệu
-

CHƯƠNG 5: RTOS – HỆ ĐIỀU HÀNH THỜI GIAN THỰC

Bài 9: RTOS và Kỹ thuật lập lịch

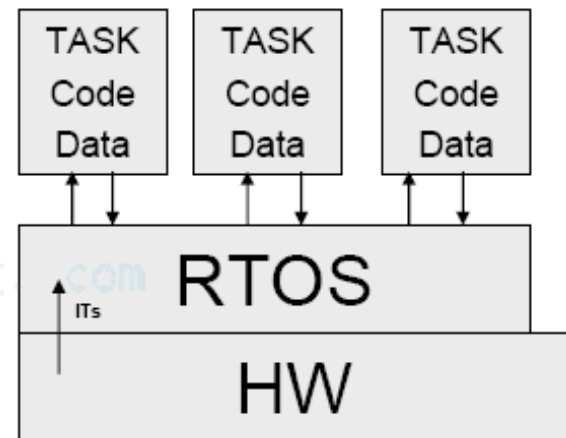
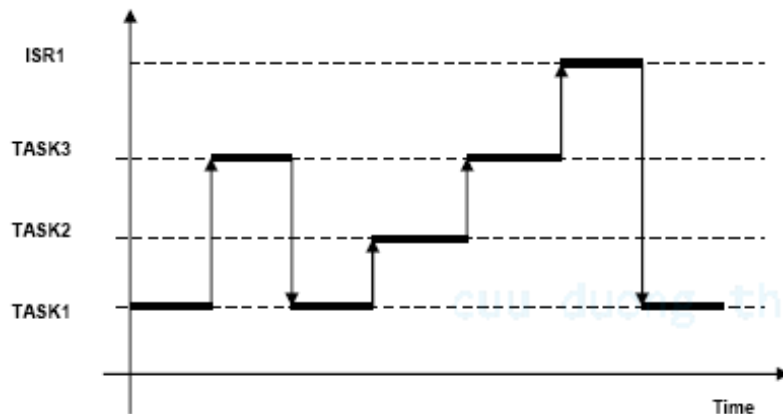
cuu duong than cong. com

cuu duong than cong. com

RTOS

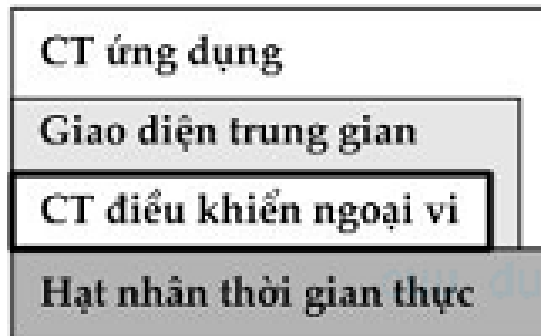
- Phần lõi (Kernel): Thực hiện việc lập lịch (schedules tasks)
- Tác vụ (Tasks): Là các hoạt động hiện tại với các trạng thái riêng của nó (PC, registers, stack, etc.)

cuu duong than cong. com

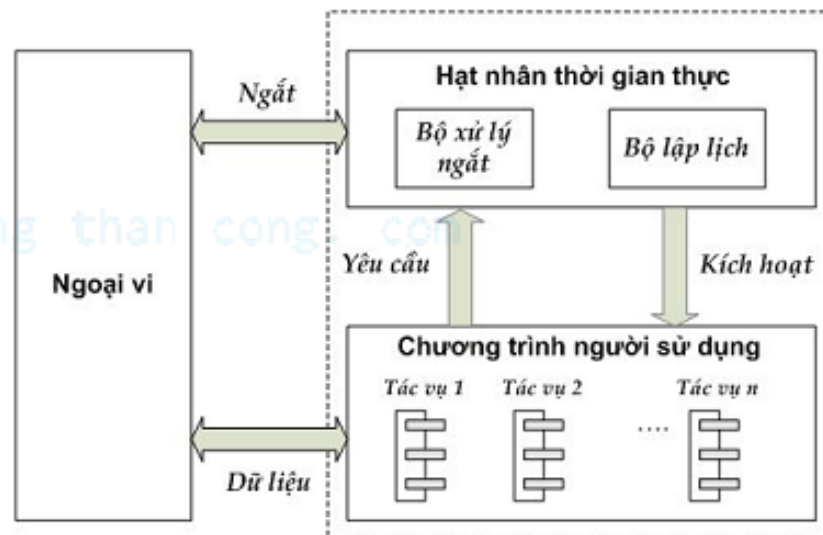


RTOS

RTOS



OS chuẩn



CÁC YÊU CẦU VỚI RTOS

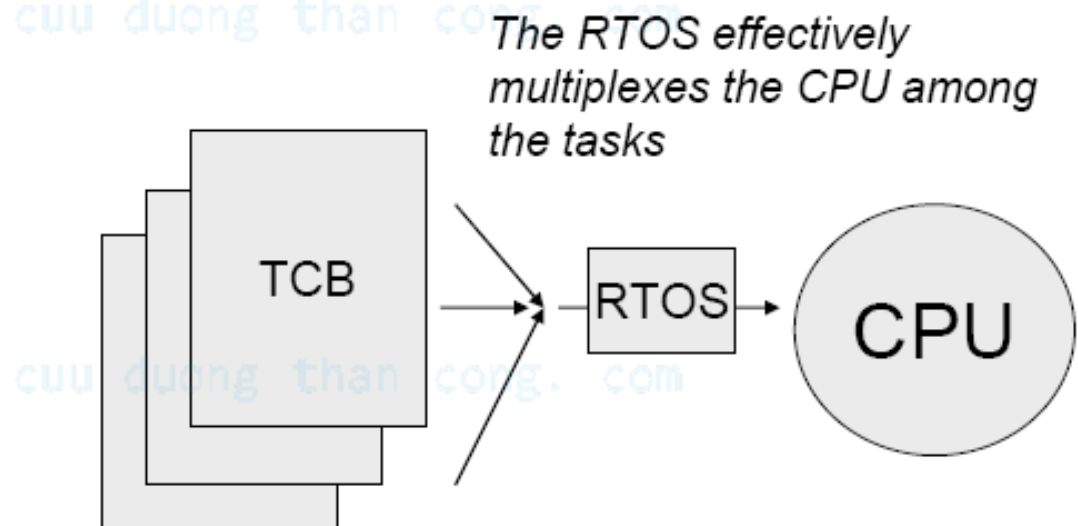
- Kích thước nhỏ (lưu trữ toàn bộ trong ROM)
- Sử dụng hệ thống ngắt
- Không nhất thiết phải có các cơ chế bảo vệ
- Tăng tốc độ truyền thông giữa các quá trình
- Khi các quá trình ứng dụng đang thực hiện thì các yêu cầu hệ thống điều hành có thể được thực hiện thông qua các lời gọi hàm thay vì sử dụng cơ chế ngắt mềm

cuu duong than cong. com

CÁC NHIỆM VỤ (Tasks)

- Các nhiệm vụ = Code + Data + State (trạng thái)
- Trạng thái của nhiệm vụ được lưu trữ trong khối điều khiển nhiệm vụ (Task Control Block - TCB) khi nhiệm vụ không được thực hiện trên CPU
- Một TCB điển hình:

ID
Priority
Status
Registers
Saved PC
Saved SP

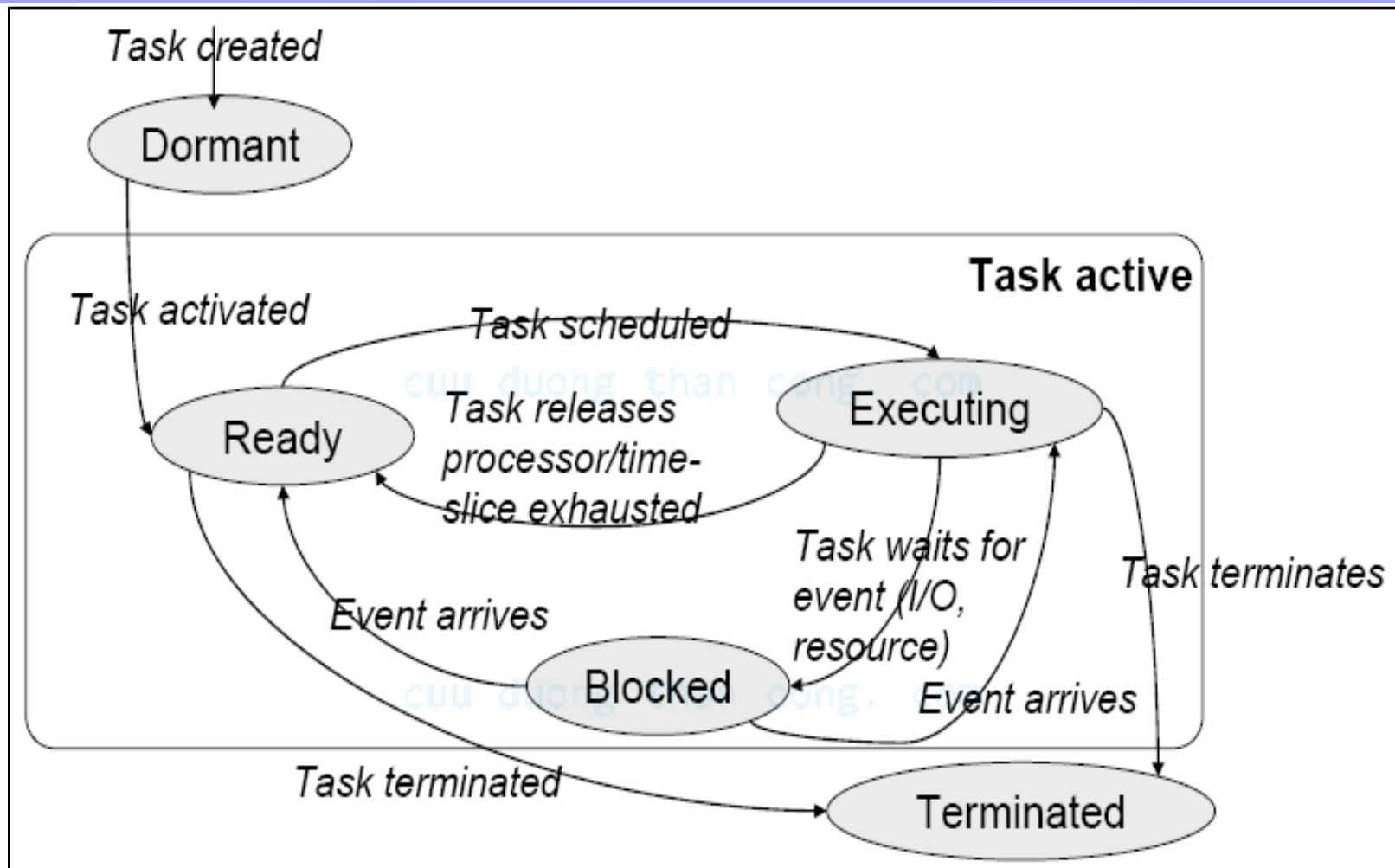


Các trạng thái của nhiệm vụ

- Executing: Đang thực hiện trên CPU
- Ready: Có thể chạy trên CPU nhưng một nhiệm vụ khác đang sử dụng CPU
- Blocked: Đợi sự kiện (I/O, signal, resource, etc.)
- Dormant: Tạo ra nhưng chưa được thực hiện
- Terminated: Không còn tác động nữa

RTOS thực hiện một cơ chế chuyển trạng thái cho mỗi nhiệm vụ và quản lý quá trình chuyển trạng thái.

Task States Transitions



Bộ lập lịch RTOS

- Thực hiện cơ chế chuyển trạng thái
- Chuyển giữa các nhiệm vụ
- Thuật toán chuyển:
 1. Lưu trữ trạng thái hiện tại vào TCB
 2. Tìm TCB mới
 3. Khôi phục trạng thái từ TCB mới
 4. Tiếp tục
- Chuyển đổi giữa trạng thái EXECUTING -> READY:
 1. Nhiệm vụ được thực hiện tuần tự: **NON-PREEMPTIVE**
 2. RTOS chuyển trạng thái cho các nhiệm vụ ưu tiên cao hơn: **PREEMPTIVE**

Quá trình lập lịch

Mục đích: Đảm bảo yêu cầu về thời gian

- Lập lịch trước khi chạy (*static*): Xác định chính xác gián đồ thời gian cho các nhiệm vụ tại thời điểm thiết kế
- Lập lịch khi chạy chương trình (*dynamic*): Lập lịch được thực hiện tự động bởi RTOS, dựa trên sự ưu tiên.

Phân bố các nhiệm vụ (1)

Một chu kỳ của nhiệm vụ (Đối với các nhiệm vụ có chu kỳ - periodic tasks):

- Các ràng buộc ưu tiên: Xét xem có bất cứ nhiệm vụ nào cần được ưu tiên không.
- Thời gian xuất hiện-*ai* (*arrival time*): Là khoảng thời gian khi sự kiện xảy ra và nhiệm vụ tương ứng được kích hoạt.
- Thời điểm bắt đầu thực thi *ri* (*release time*): Thời điểm sớm nhất khi việc xử lý đã sẵn sàng và có thể bắt đầu.
- Thời điểm bắt đầu thực hiện - *si* (*starting time*): Là thời điểm mà tại đó tác vụ bắt đầu việc thực hiện của mình.
- Thời gian tính toán/thực thi - *ci* (*Computation time*): Là khoảng thời gian cần thiết để bộ xử lý thực hiện xong nhiệm vụ của mình mà không bị ngắt.
- Thời điểm hoàn thành - *fi* (*finishing time*): Là thời điểm mà tại đó tác vụ hoàn thành việc thực hiện của mình.
- Thời gian rủi ro/xấu nhất - *wi* (*worst case time*): khoảng thời gian thực hiện lâu nhất có thể xảy ra.
- Thời điểm kết thúc - *di* (*due time*): Thời điểm mà tác vụ phải hoàn thành.

Model nhiệm vụ

Mô hình nhiệm vụ đơn giản:

- Tất cả các nhiệm vụ phải có yêu cầu khắt khe về chu kỳ thực hiện.
- Thời gian để hoàn thành nhiệm vụ chính là chu kỳ của nhiệm vụ.
- Tất cả các nhiệm vụ độc lập – không có ràng buộc về quyền ưu tiên.
- Không có nhiệm vụ nào có bộ phận không ưu tiên
- Chỉ có nhiệm vụ xử lý.

Các kỹ thuật lập lịch

1. First Come First Serve (FCFS)

- Các quá trình được xử lý theo thứ tự mà nó xuất hiện yêu cầu và cho đến khi hoàn thành
- Cơ chế lập lịch này thuộc loại không ngắt được và có ưu điểm là dễ dàng thực thi
- Không phù hợp cho hoạt động đáp ứng thời gian thực

2. Shortest Job First (SJF)

- Tác vụ có thời gian thực thi ngắn nhất sẽ có quyền ưu tiên cao nhất và sẽ được phục vụ trước
- Không biết trước được thời gian thực thi của các tác vụ tham gia trong chương trình và thông thường phải áp dụng cơ chế tiên đoán và đánh giá dựa vào kinh nghiệm về các tác vụ thực thi trong hệ thống
- Có thể áp dụng cho các tác vụ cả loại ngắt được và không ngắt được

Các kỹ thuật lập lịch

3. Rate Monotonic (RM)

- Phương pháp này dựa trên một số giả thiết sau:
 - Tất cả các tác vụ tham gia hệ thống phải có *deadline* kiểu chu kỳ
 - Tất cả các tác vụ độc lập với nhau
 - Thời gian thực hiện của các tác vụ biết trước và không đổi
 - Thời gian chuyển đổi ngữ cảnh thực hiện là rất nhỏ và có thể bỏ qua
- Thuật toán RM được thực thi theo nguyên lý ***gán mức ưu tiên cho các tác vụ dựa trên chu kỳ của chúng*** (chu kỳ nhỏ thì mức ưu tiên cao)
- Với các tác vụ chu kỳ không thay đổi thì RM sẽ là phương pháp lập lịch cho phép ngắn và mức ưu tiên cố định

Các kỹ thuật lập lịch

3. Earliest Deadline First (EDF)

- Sử dụng *deadline* của tác vụ như điều kiện ưu tiên để xử lý điều phối hoạt động
- Tác vụ có *deadline* gần nhất sẽ có mức ưu tiên cao nhất và các tác vụ có *deadline* xa nhất sẽ nhận mức ưu tiên thấp nhất

4. Minimum Laxity First (MLF)

- Cơ chế lập lịch này sẽ ưu tiên tác vụ nào còn ít thời gian còn lại để thực hiện nhất trước khi nó phải kết thúc để đảm bảo yêu cầu thực thi đúng
- Cơ chế lập lịch gán quyền ưu tiên động và dễ đạt được sự tối ưu về hiệu suất thực hiện và sự công bằng trong hệ thống

Các kỹ thuật lập lịch

5. Round Robin (RR)

- Mỗi một tác vụ được xử lý/phục vụ trong một khoảng thời gian nhất định và lặp lại theo một chu trình xuyên suốt toàn bộ các tác vụ tham gia trong hệ thống
- Khoảng thời gian phục vụ cho mỗi tác vụ trong quá trình là một sự thoả hiệp giữa thời gian thực hiện của các tác vụ và thời gian thực hiện một chu trình

CHƯƠNG 6: TỔNG HỢP PHẦN CỨNG VÀ PHẦN MỀM

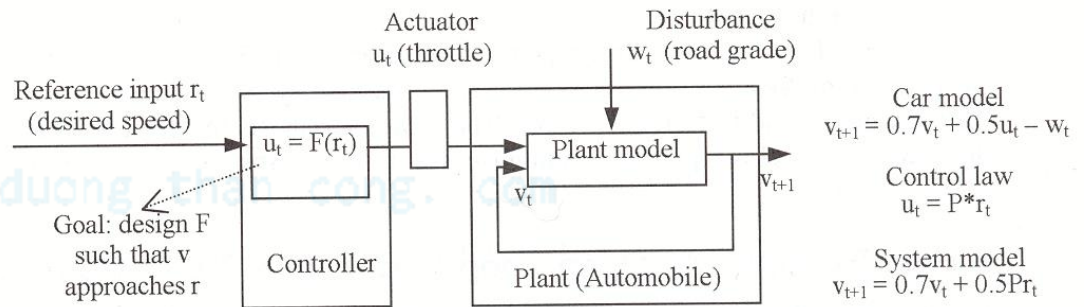
Bài 10: Các hệ thống điều khiển

cuuduongthancong.com

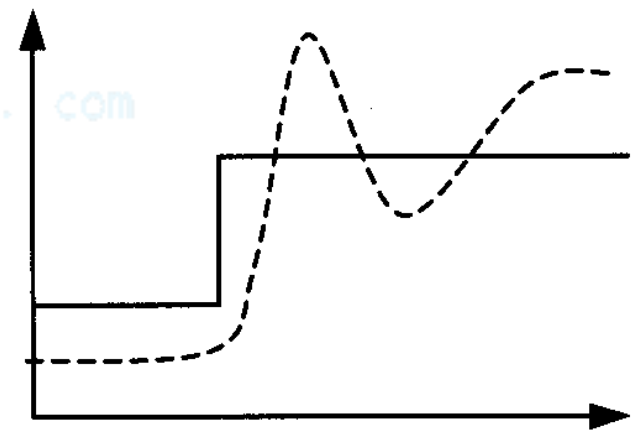
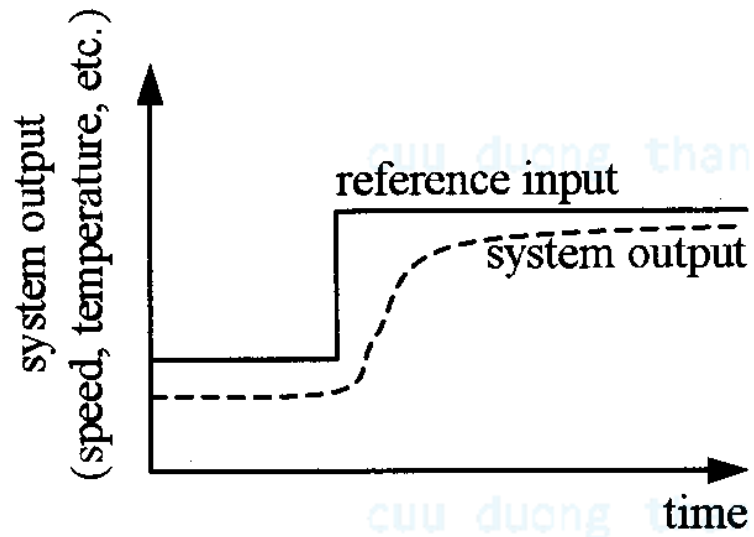
cuuduongthancong.com

Hệ thống điều khiển

- Điều khiển đầu ra của hệ vật lý
 - Bằng cách thiết lập đầu vào của hệ vật lý
- Điều khiển bám
- VD
 - Điều khiển lái
 - Điều khiển nhiệt độ
 - Điều khiển ổ đĩa
 - Điều khiển bay
- Khó, do các nguyên nhân:
 - Nhiều: gió, mặt đường, lốp xe, phanh...
 - Tương tác với con người

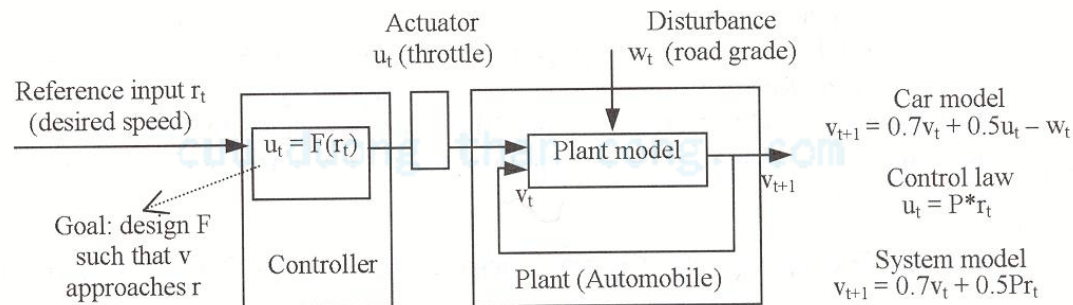


Bám



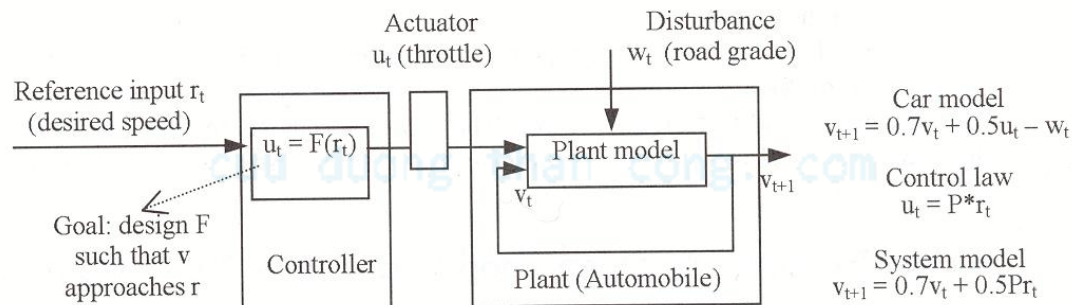
Các hệ thống điều khiển vòng hở

- Đối tượng
 - Các hệ vật lý được điều khiển
 - Ô tô, máy bay, lò nhiệt...
- Cơ cấu chấp hành
 - Thiết bị để điều khiển đối tượng
 - van, động cơ...
- Bộ điều khiển
 - Sản phẩm được thiết kế để điều khiển đối tượng



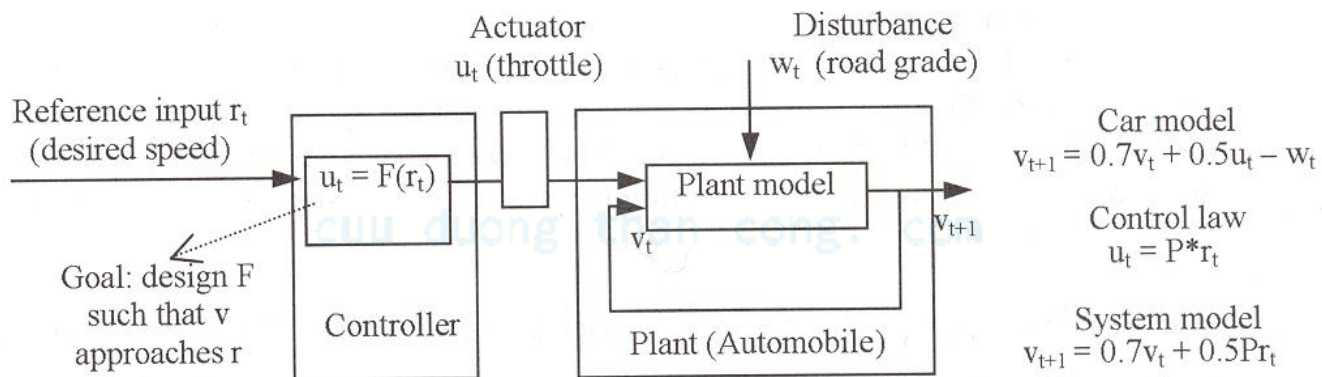
Các hệ điều khiển vòng hở

- Đầu ra
 - Các thông số của đối tượng mà ta quan tâm
 - Tốc độ, vị trí, nhiệt độ,...
- Giá trị tham chiếu
 - Giá trị chúng ta muốn đạt được ở đầu ra
 - Tốc độ yêu cầu, vị trí mong muốn, nhiệt độ mong muốn
- Nhiễu
 - Đầu vào không điều khiển được
 - Gió, lực ngoài



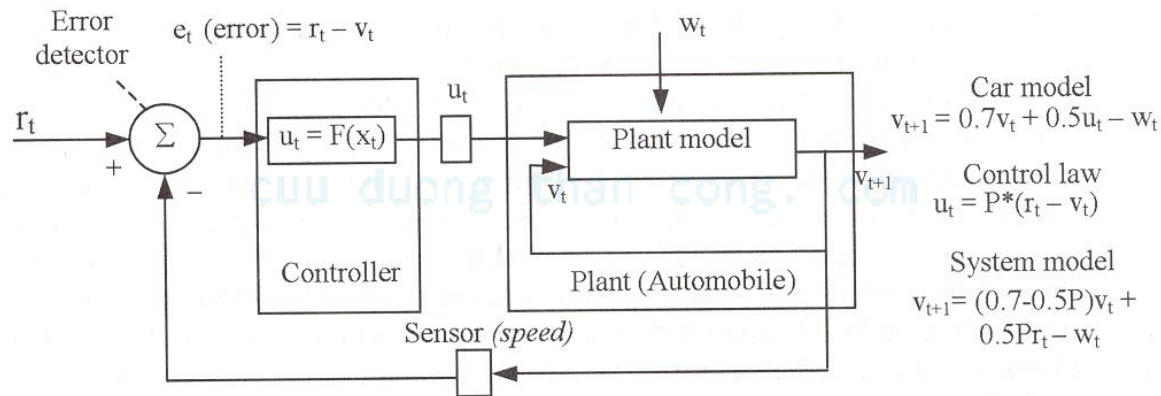
Các đặc tính khác của vòng hở

- Điều khiển Feed-forward
- Có trễ ở đầu ra
- Bộ điều khiển không biết đầu ra có đáp ứng theo yêu cầu
- Đơn giản
- Sử dụng tốt nhất cho các hệ thống có thể dự đoán đầu ra



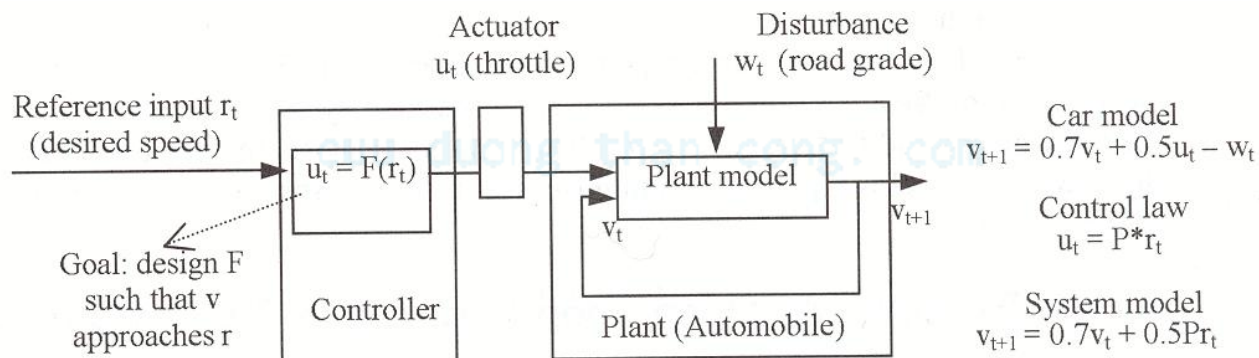
Các hệ điều khiển vòng kín

- Cảm biến
 - Đo đầu ra của đối tượng
- Bộ xác định lỗi
 - Xác định sai số
- Hệ điều khiển vòng kín
- Giảm sai số bám



Thiết kế hệ điều khiển vòng hở

- Xây dựng mô hình đối tượng
- Xây dựng bộ điều khiển
- Phân tích bộ điều khiển
- Xem xét nhiễu
- Xác định chất lượng
- Ví dụ: Hệ điều khiển lái vòng hở



Mô hình đối tượng

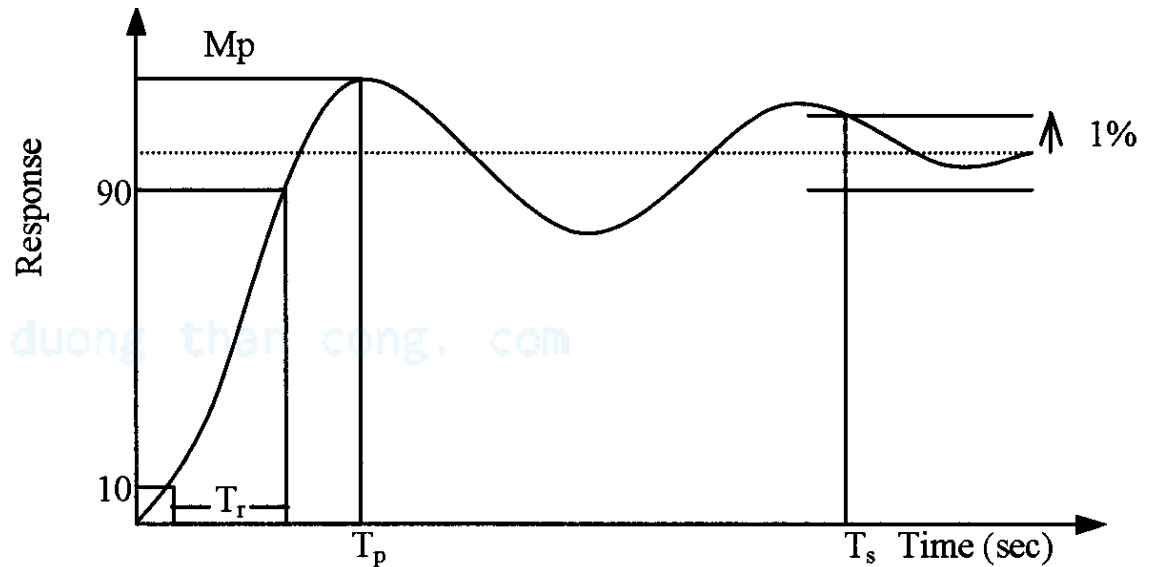
- Có thể không cần thiết
 - Có thể được thực hiện thông qua thực nghiệm hoặc dò
- Tuy nhiên,
 - Có thể giúp quá trình thiết kế đơn giản hơn
 - Có thể sử dụng để tính toán bộ điều khiển
- Ví dụ: điều khiển góc mở từ 0 đến 45 độ
 - Trên mặt phẳng ở tốc độ 50 mph, mở van tới 40 độ
 - Đợi 1 khoảng thời gian nhất định
 - Đo tốc độ, ví dụ 55 mph
 - Dựa vào phương trình sau
 - $v_{t+1} = 0.7 \cdot v_t + 0.5 \cdot u_t$
 - $55 = 0.7 \cdot 50 + 0.5 \cdot 40$
 - Nếu phương trình đúng cho mọi trường hợp
 - Như vậy chúng ta có một mô hình của đối tượng

Hệ điều khiển chung

- Mục tiêu
 - Làm cho đầu ra bám theo giá trị đặt ngay cả khi
 - Nhiễu đo lường
 - Sai số của mô hình
 - Nhiễu ngoài
- Các thông số
 - Tính ổn định
 - Đầu ra nằm trong một giới hạn
 - Chất lượng
 - Đầu ra bám theo giá trị đặt
 - Loại bỏ nhiễu
 - Tính bền vững
 - Khả năng đáp ứng khi mô hình thay đổi

Chất lượng (nói chung)

- Thời gian tăng
 - Thời gian từ 10% tới 90%
- Thời gian đỉnh
- Quá điều chỉnh
 - Phần trăm mà giá trị đỉnh đạt quá giá trị đặt
- Thời gian xác lập
 - Thời gian cần thiết để đạt 1% giá trị cuối

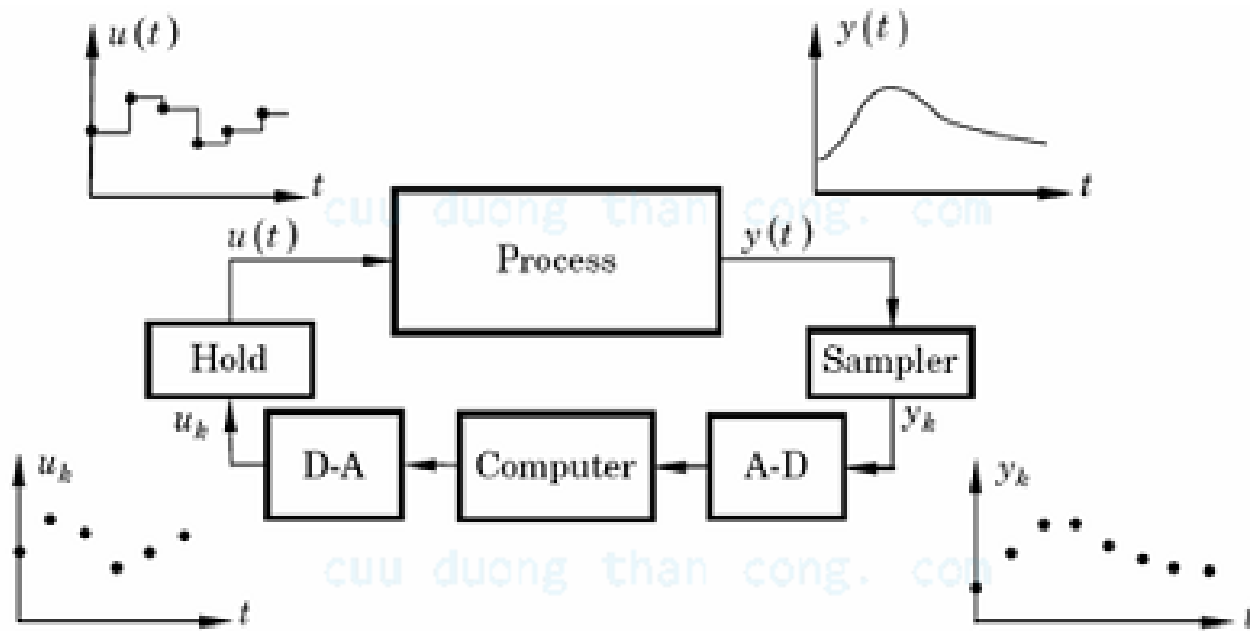


Xây dựng mô hình của đối tượng là khó

- Cần được thực hiện trước
- Đối tượng thường liên tục theo thời gian
 - Không rời rạc
 - VD tốc độ xe thay đổi liên tục so với vị trí góc mở của van
 - Thời gian lấy mẫu phải được lựa chọn cẩn thận
- Đối tượng thường phi tuyến
- Mỗi tương tác giữa mô hình đối tượng và bộ điều khiển
 - Mô hình “đủ tốt” cho việc thiết kế bộ điều khiển

Thực hiện bộ điều khiển số

- Hệ thống điều khiển số



Thực hiện bộ điều khiển số

- Triển khai hàm điều khiển dưới dạng biến đổi z

$$G_R(z) = \frac{U(z)}{E(z)} = \frac{b_0 + b_1 z^{-1} + \dots + b_m z^{-m}}{a_0 + a_1 z^{-1} + \dots + a_n z^{-n}}$$

- Biểu diễn dạng rời rạc

$$a_0 u^*(t) + \sum_{k=1}^n a_k u^*(t - kT) = \sum_{k=0}^m b_k e^*(t - kT)$$

- Hoặc

$$u^*(t) = \frac{1}{a_0} \sum_{k=0}^m b_k e^*(t - kT) - \frac{1}{a_0} \sum_{k=1}^n a_k u^*(t - kT)$$

Bộ điều khiển PID số

- Phương trình bộ điều khiển PID

$$\begin{aligned}u(t) &= u_p(t) + u_i(t) + u_d(t) \\&= K \left[e(t) + \frac{1}{T_i} \int_0^t e(\tau) d\tau + T_D \frac{de(t)}{dt} \right]\end{aligned}$$

- Biểu diễn xấp xỉ dạng rời rạc

$$u(k) = K_p \left[e(k) + \frac{T_s}{T_i} \sum_{i=0}^{k-1} e(i) + \frac{T_D}{T_s} (e(k) - e(k-1)) \right]$$

- Suy ra

$$u(k-1) = K \left[e(k-1) + \frac{T_s}{T_i} \sum_{i=0}^{k-2} e(i) + \frac{T_D}{T_s} (e(k-1) - e(k-2)) \right]$$

Bộ điều khiển PID số

- Từ hai phương trình trên ta thu được

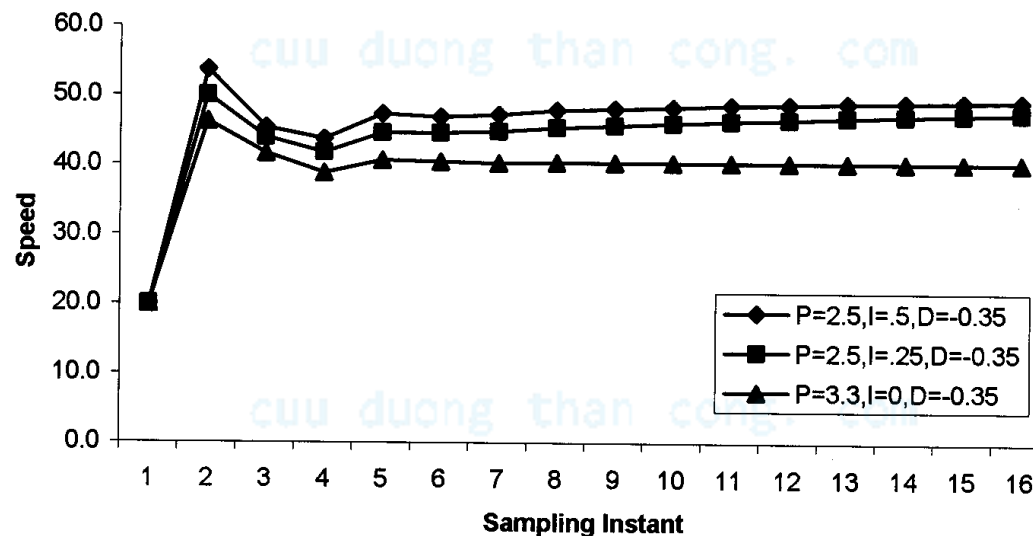
$$u(k) - u(k-1) = a_0 e(k) + a_1 e(k-1) + a_2 e(k-2)$$

trong đó,
$$a_0 = K \left(1 + \frac{T_D}{T_s} \right), \quad a_1 = -K \left(1 + 2 \frac{T_D}{T_s} - \frac{T_s}{T_I} \right), \quad a_2 = K \frac{T_D}{T_s}$$

cuu duong than cong. com

Bộ điều khiển PID số dạng xấp xỉ

- Kết hợp điều khiển tỷ lệ, vi phân và tích phân
 - $u_t = P \cdot e_t + I \cdot (e_0 + e_1 + \dots + e_t) + D \cdot (e_t - e_{t-1})$
- Có sẵn trong công nghiệp



Mã chương trình

- Vòng lặp chương trình chính
 - Đọc đầu ra cảm biến của đối tượng điều khiển
 - Có thể yêu cầu A2D
 - Đọc đầu vào đặt hiện tại
 - Gọi hàm PidUpdate, xác định giá trị của cơ cấu chấp hành
 - Thiết lập giá trị của cơ cấu chấp hành
 - Có thể yêu cầu D2A

```
void main()
{
    double sensor_value, actuator_value, error_current;
    PID_DATA pid_data;
    PidInitialize(&pid_data);
    while (1) {
        sensor_value = SensorGetValue();
        reference_value = ReferenceGetValue();
        actuator_value =
            PidUpdate(&pid_data, sensor_value, reference_value);
        ActuatorSetValue(actuator_value);
    }
}
```

Mã chương trình (cont)

- Pgain, Dgain, Igain là hằng số
- sensor_value_previous
 - Cho khâu D
- error_sum
 - Cho khâu I

```
typedef struct PID_DATA {  
    double Pgain, Dgain, Igain;  
    double sensor_value_previous; // find the derivative  
    double error_sum; // cumulative error  
}
```

Tính toán

- $$u_t = P * e_t + I * (e_0 + e_1 + \dots + e_t) + D * (e_t - e_{t-1})$$

```
double PidUpdate(PID_DATA *pid_data, double sensor_value,
                 double reference_value)
{
    double Pterm, Iterm, Dterm;
    double error, difference;
    error = reference_value - sensor_value;
    Pterm = pid_data->Pgain * error; /* proportional term*/
    pid_data->error_sum += error; /* current + cumulative*/
    // the integral term
    Iterm = pid_data->Igain * pid_data->error_sum;
    difference = pid_data->sensor_value_previous -
                 sensor_value;
    // update for next iteration
    pid_data->sensor_value_previous = sensor_value;
    // the derivative term
    Dterm = pid_data->Dgain * difference;
    return (Pterm + Iterm + Dterm);
}
```

1

Dò giá trị PID

- Xác định giá trị P, I, D theo tính toán có thể không áp dụng được
 - VD không có mô hình của đối tượng, hoặc phức tạp
- Phương pháp đặc biệt để xác định giá trị P, I, D “hợp lý”
 - Bắt đầu với P, I=D=0
 - Tăng D, đến khi thấy dao động
 - Giảm D một ít
 - Tăng P, đến khi thấy dao động
 - Giảm D một ít
 - Tăng I, đến khi thấy dao động
- Lặp lại cho đến khi hết dao động

Trễ tính toán

- Luôn có trễ trong quá trình xử lý
 - Cơ cấu chấp hành tác động chậm hơn so với mong muốn
- Cần đánh giá trễ để đảm bảo có thể bỏ qua chúng mà không làm ảnh hưởng hệ thống
- Trễ phần cứng thường dễ xác định
 - Thiết kế đồng bộ
- Trễ phần mềm dễ xác định hơn
 - Tổ chức chương trình cẩn thận sao cho trễ có thể dự đoán được và tối thiểu
 - Viết chương trình có khả năng xác định thời gian
 - Watchdog timer
 - Synchronous Software Language

Lợi ích của điều khiển bằng máy tính

- Giá!!!
 - Giá thành bộ điều khiển sử dụng thiết bị tương tự đắt
 - Bộ điều khiển máy tính thay thế phần cứng analog phức tạp với mã phức tạp
- Khả năng lập trình!!!
 - Điều khiển bằng máy tính có thể được “nâng cấp”
 - Thay đổi kiểu điều khiển, chế độ điều khiển
 - Điều khiển bằng máy tính có thể thích nghi với thay đổi của đối tượng điều khiển
 - Do sự thay đổi của nhiệt độ, áp suất,...
 - “có tính mở”
 - Dễ thích nghi với các chuẩn mới

Reference

- This document is used for educational purposes only
- This document contains material from other sources (incl. textbook, lecture notes, application notes, ...)
- Our thanks go to the contributors, references